

TEST AUTOMATION FRAMEWORK

Playwright UI Framework

Complete design, file and change-
management documentation

VERSION

1.0

GENERATED

2026-09-04

FILES DOCUMENTED

139

EXPORTS EXTRACTED

171

Contents

- 1 **Overview**
What this framework is and how to read this document
- 2 **Architecture**
Layers, execution flow and the reasoning behind each decision
- 3 **Worked example**
The suite against a real banking application, and the defects it found
- 4 **Project structure**
Every folder and file, with its purpose
- 5 **Reference: configuration & tooling**
Root config, CI, containers, editor and docs tooling
- 6 **Reference: config, core & types**
The foundation every other layer is built on
- 7 **Reference: component library**
All 37 element types, grouped by family
- 8 **Reference: fixtures, hooks, pages & API**
How the framework reaches your tests
- 9 **Reference: utilities, reporters & data**
Cross-cutting helpers and outputs
- 10 **Reference: tests & repository docs**
Where specs live and the conventions they follow
- 11 **Playbooks**
I need to change something — where do I go, what do I do, and why there?
- 12 **Conventions**
Rules, naming, tags, anti-patterns and the review checklist
- 13 **Troubleshooting**
Failure modes, flakiness and debugging tools
- 14 **API index**
Every exported symbol and where it lives
- 15 **Glossary**
Terms used in the code and in this document

Overview

What this framework is and how to read this document

This is the complete reference for the **playwright-ui-framework**: what every folder and file does, how the layers fit together, and — for any change you might need to make — where to go, what to do and why that is the right place for it.

The framework is a TypeScript foundation for testing every kind of user-interface element: forms, tables and enterprise data grids, dropdowns, date pickers, modals, toasts, tabs, trees, carousels, canvases, iframes, shadow DOM, rich-text editors, drag-and-drop, infinite scroll and charts.

How to use this document

IF YOU WANT TO...	GO TO
See the framework actually working	Worked example — the suite against a real bank, and the defects it found
Understand the design before touching anything	Architecture
Find what a specific file does	Project structure , then the matching reference page
Make a change and not sure where	Playbooks — task-oriented runbooks
Know the rules before opening a pull request	Conventions
Diagnose a failure or a flaky test	Troubleshooting
Look up a class, method or type	API index

Quick start

FROM CLONE TO FIRST RUN

Shell

```
npm ci # install dependencies
npx playwright install --with-deps # install browsers

npm test # 110 tests against a real application
npm run validate # typecheck + lint + format check
```

NOTE No configuration is required. The suite ships pointed at **SecureBank** — a real banking application published at qaplayground.com for automation practice — with its published demo credentials, so a fresh clone produces a meaningful result immediately. Copy `.env.example` to `.env` when you point it at your own application.

It ships with **110 working tests**: the application's own catalogue of 42 test cases, plus authentication, navigation, responsive behaviour, accessibility scans and visual baselines. Three of them record real defects the suite found.

NOTE Requires Node 20, 22 or 24 (LTS); the version is pinned in `.nvmrc`. Playwright supports LTS releases, and odd-numbered Node versions regularly break module resolution in tooling.

The three things to know first

1. **Tests import from `@fixtures/index`, never from `@playwright/test`**. That one import is what gives a spec the logger, component factory, network control, console capture, generated data and custom matchers.
2. **Selectors live in components, components are wired up in page objects, and tests call page-object methods**. A selector in a spec is a defect waiting to happen.
3. **`src/config/env.config.ts` is the only file that reads `process.env`**. Everything else receives validated, typed configuration.

What a test will look like

TESTS/UI/CHECKOUT/DISCOUNT.SPEC.TS

TypeScript

```
import { test, expect } from '@fixtures/index';
import { CheckoutPage } from '@pages/checkout.page';

test('applies a discount code @smoke', async ({ page, api }) => {
  const order = await api.post('/orders', { items: [{ sku: 'A1', qty: 2 }] }); // seed fast

  const checkout = new CheckoutPage(page);
  await checkout.goto({ query: { order: order.id } });

  const message = await checkout.applyDiscount('SAVE20');

  expect(message).toContain('Discount applied');
  await checkout.total.expectText('$80.00');
  expect(await checkout.items.getColumnValues('Price')).toBeSorted('desc');
});
```

Regenerating this documentation

ONE COMMAND REBUILDS EVERYTHING

Shell

```
npm run docs      # extract API + build HTML site + render PDF
npm run docs:api   # refresh docs/generated/api.json only
npm run docs:html  # rebuild docs/site/ only
npm run docs:pdf   # re-render the PDF only
```

NOTE The build **fails** if a file in the repository has no documentation entry. That is deliberate: it is the mechanism that stops this document falling behind the code.

Architecture

Layers, execution flow and the reasoning behind each decision

The framework is five layers, and each layer may only reach downwards. A component never knows which page it is on; a page never knows which test is running.

```
tests/           What the business cares about. No selectors, no waits.
  | calls
src/pages/       Page objects: URL, ready state, business actions.
  | composed of
src/components/  Element behaviour: how a table sorts, how a dropdown opens.
  | built on
src/core/        BaseComponent / BasePage: logging, steps, waits, diagnostics.
  | uses
src/utils/       Cross-cutting: logging, network, files, ally, visual, data.
src/config/      Validated environment configuration.
```

Why there is a component layer at all

Most Playwright suites put selectors in page objects and call raw locator methods from there. That works until the twentieth table — at which point every page object contains its own slightly different "read all rows" loop, and a change in the grid library means editing twenty files.

Here, `Table` knows how to read rows by column name, sort, filter, select and verify ordering — once. `DataGrid` extends it with the three things virtualised grids add: asynchronous loading, rows that exist only while scrolled into view, and inline editing. A page object then becomes a short, declarative list of components.

A PAGE OBJECT IS A DECLARATION, NOT A PROCEDURE

TypeScript

```
readonly results  = this.factory.dataGrid('[data-testid="results"]', { name: 'Results' });
readonly filters  = this.factory.form('[data-testid="filters"]', { name: 'Filters' });
readonly pagination = this.factory.pagination('[data-testid="pager"]', { name: 'Pager' });
```

BaseComponent: what every element gets for free

Because all 37 component types extend one class, these behaviours are uniform and cost nothing to add to a new component:

- **Reported steps** — every action runs inside `test.step`, so a trace reads `Table "Results": sort by "Amount" desc` rather than an anonymous click.
- **Contextual failures** — errors name the component, the selector and the original cause.
- **Pre-action hygiene** — visibility wait, scroll into view, optional stability wait, before every interaction.
- **State inspection** — `getState()` returns visibility, enablement, checked state, focus, text, value, box and every attribute in one call.

- **Named assertions** — web-first assertions wrapped with messages that identify the component.

NOTE This is the leverage point of the framework: one method added here becomes available to every element type, and one improvement to error messages improves every failure in the suite.

How a run actually executes

- 1 `playwright.config.ts` is loaded in the parent process. Importing it triggers `env.config.ts`, which loads the `.env` files and validates every variable with zod. A bad variable fails here, before a browser exists.
- 2 `globalSetup` creates the artifact directories, clears the previous HTML report and logs the resolved configuration.
- 3 The `setup` project runs `auth.setup.ts`, signing in once per role and writing `storage/<role>.json`.
- 4 Worker processes start. Each worker builds worker-scoped fixtures once — notably the `api` client.
- 5 For each test, Playwright builds the test-scoped fixtures: browser context (with storage state), page, `ui`, `log`, `network`, `consoleErrors`, `testData`.
- 6 The test body runs. Component actions log, report steps and take screenshots on failure.
- 7 Teardown runs in reverse order. If the test failed, the `page` fixture attaches the URL, a DOM snapshot and captured browser errors.
- 8 Reporters write the HTML report, JUnit XML, JSON, CTRF and `reports/summary.json`. `globalTeardown` removes scratch files and logs the duration.

Locator resolution

`resolveLocator()` accepts a CSS or XPath string, an existing Locator, or a function that derives one from a scope — and applies the `iframe` and `shadow-DOM` options a component declares. That is why an `iframe` needs no special handling anywhere else:

THE SAME COMPONENT API, INSIDE A FRAME

TypeScript

```
const frame = new Frame(page, '#payment');
const cardNumber = new TextInput(frame.locator, '#card-number');
```

Design decisions and their rationale

DECISION	RATIONALE	WHAT IT WOULD COST TO REVERSE
A component layer between pages and locators	Element behaviour is written once instead of per page.	High — every page object would absorb the logic.
<code>test.step</code> around every action	Traces read as prose; failures point at a business step.	Low, but debugging gets materially worse.
zod validation of the environment	Misconfiguration fails in one second with a precise message.	Low, but silent misconfiguration returns.
Storage-state authentication	Removes a login from every test.	Medium — suite time grows by minutes per hundred tests.
Test data seeded from the test title	Retries reproduce the original inputs.	Low, but retries stop being diagnostic.
Named timeout budgets	Flakiness is tuned in one file.	Medium — magic numbers spread quickly.
Dependency-free logger	Logging libraries break across Node releases.	Low.
Three drag-and-drop strategies	No single approach works across DnD libraries.	High — discovered mid-sprint at the worst time.

Deliberate constraints

RULE No `waitForTimeout` in test code. No selectors above the component layer. No `process.env` outside `env.config.ts`. No business assertions inside page objects. No hand-maintained data rows that parallel workers can collide on.

Worked example

The suite against a real banking application, and the defects it found

The framework drives **SecureBank**, a real banking application published at qaplayground.com/bank for automation practice. Nothing is mocked. This page explains how the suite is built against it, what the code does, and the three defects it found.

If you read one page of this document, read this one. Everything else is reference; this is the framework doing its job.

Why this application

PROPERTY	WHY IT MATTERS
Thorough <code>data-testid</code> coverage	Almost every meaningful element carries one, which is why <code>testIdAttribute: 'data-testid'</code> does so much work here. The handful that do not are the interesting cases — see below.
State lives in <code>localStorage</code>	Playwright gives every test a fresh browser context, so each test starts from the seeded dataset automatically. No cleanup, no shared account, no ordering constraints — the suite is fully parallel by construction.
Seven personas with different behaviour	A locked account, a frozen account, an overdraft, a slow loader and one carrying a planted defect. Negative paths are reachable by logging in as somebody else rather than by mocking a failure.
It publishes its own test catalogue	The app lists 42 intended test cases at <code>/bank/test-cases</code> . The suite implements them and each test names the case it satisfies, so coverage is checkable against the app's own specification.

RULE The third property is the one worth stealing. Most UI suites test the happy path because the unhappy paths need a backend nobody controls. Here they are one login away.

How a test is put together

TESTS/UI/TRANSFER.SPEC.TS — THE WHOLE SHAPE

TypeScript

```
import { test, expect } from '../src/fixtures';

test('TC-XFER-004 – a valid transfer completes and issues a reference @smoke', async ({ signedIn })
  await signedIn.transfer.goto();

  const reference = await signedIn.transfer.completeTransfer({
    from: 'Everyday Checking',
    to: 'High-Yield Savings',
    amount: '25.50',
  });

  expect(reference).toMatch(/TXN-\d{8}-\d+/);
  await expect(signedIn.transfer.confirmedAmount.locator).toContainText('25.50');
});
```

PIECE	WHAT IT DOES	WHY NOT THE OBVIOUS ALTERNATIVE
<code>from</code> <code>'../src/fixtures'</code>	Gets the framework's <code>test</code> and <code>expect</code>	Importing <code>@playwright/test</code> gives you neither the page objects nor the custom matchers, and nothing warns you
<code>{ signedIn }</code>	Signs in as the standard persona and hands over every page object	A <code>beforeEach</code> would also run for the login specs, which must start signed out
<code>completeTransfer(...)</code>	Drives all three screens — form, confirmation modal, success page	A test stopping at "the form submitted" has not tested a transfer
<code>SEED</code>	Dataset facts in one file	Nine specs hard-coding \$17,050 is nine edits when the app reseeds

Three layers, and what belongs in each

```
tests/ui/transfer.spec.ts      WHAT should be true – assertions only
|
src/pages/bank/transfer.page.ts  WHERE things are and HOW to drive them
|
src/components/form/dropdown.ts  HOW a *kind* of widget behaves, for any app
```

RULE A page object returns values and never asserts business outcomes. That is what lets a negative-path test reuse it instead of fighting it, and it is why `fillForm()` and `completeTransfer()` are separate methods.

The component layer is where the framework earns its keep. SecureBank's dropdowns are custom listboxes, its terms checkbox is a `span[role="checkbox"]`, its two-factor control is a `button[role="switch"]`. The `Dropdown`, `Checkbox` and `Toggle` components already handle all three — the page object only has to say where they are.

The details that took a real run to discover

Every item below was found by running against the application, not by reading its markup. They are the reason exploring first and writing second is worth the time.

1. A closed dropdown leaves its panel in the DOM

SRC/PAGES/BANK/DROPDOWN-PANEL.TS

TypeScript

```
export const VISIBLE_LISTBOX = '[role="listbox"]:visible';
```

On the transfer page, once "From" has been used there are two `[role="listbox"]` elements — the closed one and the one being opened. A plain selector resolves to the first, which is hidden, and the component waits for it until it times out. The failure reads `"waiting for [role=listbox] to be visible"` while a perfectly good panel is open a few pixels away.

2. Reading state after the click instead of before

THE BUG, AND THE FIX

TypeScript

```
// Broken: reads the ALREADY-CHANGED value, so the condition can never hold.
await this.themeToggle.click();
await this.page.waitForFunction((wasDark) => isDark() !== wasDark, await this.isDarkMode());

// Correct:
const wasDark = await this.isDarkMode();
await this.themeToggle.click();
await this.page.waitForFunction((previous) => isDark() !== previous, wasDark);
```

Cheap mistake, expensive symptom: it looked exactly like the application failing to switch theme.

3. Absence is a state

The unread-notification badge is **removed from the DOM** at zero rather than showing "0". A test waiting for the text "0" hangs until its timeout. `unreadCount()` reads absence as zero, so no test has to know.

4. `sr-only` is visible to Playwright

The skip link is clipped to a 1x1 box rather than hidden, because a `display: none` element cannot be focused. Playwright therefore reports it as `*visible*` and `toBeHidden()` fails for the wrong reason. The correct assertion is about the **bounding box**, which is what actually distinguishes "clipped" from "on screen".

5. Sorting applies to the dataset, not the page

An early draft asserted that clicking sort twice produced the reverse of the first page. It failed against entirely correct behaviour: sorting covers all 18 transactions while 10 are shown, so descending page 1 holds the largest ten and ascending page 1 the smallest — two different slices. Comparing the `*directions*` is the assertion that actually catches a broken sort control.

6. Read seed values from a clean context

`totalTransactions` was first recorded as 20, from a session that had already performed a transfer — each transfer adds a debit and a matching credit. The true figure is 18. Seed values read from an exploratory session encode whatever the explorer happened to do first.

7. Discover required fields from the app's own messages

The add-account dialog looked like it needed a name. Submitting it produced "Please select an account type", then "Please enter a valid starting balance" — and the balance field has no test id at all. Reading the validation messages was faster and more reliable than reading the markup.

Three defects the suite found

All three are in the application, verified by hand, and none of them fails the build. They are recorded with `test.fail()`, which is the right tool for a defect you do not control:

- The expectation stays in the suite, written as it **should** behave.
- The run stays green, so nobody learns to ignore a permanently red test.
- The moment somebody fixes it, the test **fails** — telling us to remove the annotation.

A skipped test would go quiet forever; a deleted one would lose the finding entirely.

FINDING	STANDARD	DETAIL
The skip link is unreachable by keyboard	WCAG 2.4.1	Correct markup, first in the DOM, works when focused — but the first Tab lands on the brand link, so a keyboard user can never reach it
The account dialog omits <code>aria-modal</code>	WAI-ARIA	<code>role="dialog"</code> and a label are present; without <code>aria-modal="true"</code> a screen reader is not told to confine the user to the dialog
Three transfer controls have no accessible name	WCAG 4.1.2	The memo field and both date-type radios announce as "blank". axe does not flag them — precisely the class an automated scan misses

NOTE Two further violations — `color-contrast` on four pages and `aria-prohibited-attr` on one — are **baselined** rather than annotated, because they recur across pages. They are excluded from the gating scans, so any **new** violation fails the build, and a never-failing test reports the backlog on every run.

And the planted one

The application ships a deliberate bug for `error_user`, and its own catalogue asks for it to be reproduced (TC-LOAN-004). This is the test that proves the framework can **find** something rather than confirm things work.

TESTS/UI/APPLY-LOAN.SPEC.TS

TypeScript

```
const displayed = await bank.applyLoan.displayedTotal();
const expected = await bank.applyLoan.expectedActiveTotal();

expect(displayed).toBeCloseTo(expected, 2);
```

PERSONA	DISPLAYED TOTAL	ACTIVE + PENDING LOANS	VERDICT
<code>standard_user</code>	\$39,300	\$39,300	Correct
<code>error_user</code>	\$24,300	\$39,300	Short by the \$15,000 newest pending loan

RULE A control test asserts the same comparison passes for the healthy account. Without it, a failing total could just as easily mean the assertion is wrong — the control is what makes the result trustworthy.

Getting that comparison right took two attempts. The total is labelled "Total (Active/Pending)" and the history paginates at five rows, so comparing it against the visible page is wrong twice over: it misses later pages and includes closed loans. `expectedActiveTotal()` walks the pages and filters by status.

What the suite covers

SPEC	TESTS	COVERS
<code>login.spec.ts</code>	11	Sign-in, refusal messages, password visibility, remember-me, the fixture-integrity check

SPEC	TESTS	COVERS
<code>dashboard.spec.ts</code>	8	TC-DASH-001..006, stat cards, every quick action
<code>navigation.spec.ts</code>	15	Ten destinations, brand link, unread badge, responsive drawer, the skip link
<code>accounts.spec.ts</code>	12	TC-ACC-001..006 across three personas, the dialog, the detail page
<code>transfer.spec.ts</code>	12	TC-XFER-001..006, cancellation, the frozen block
<code>transactions.spec.ts</code>	11	TC-TXN-001..006, the type filter, the CSV export
<code>money-movement.spec.ts</code>	11	TC-SEND-001..006 and TC-BILL-001..006
<code>apply-loan.spec.ts</code>	9	TC-LOAN-001..006 including the planted defect
<code>account-settings.spec.ts</code>	11	Notifications and profile
<code>bank.all.spec.ts</code>	15	axe across ten pages and both themes
<code>bank.visual.spec.ts</code>	7	Login, stat cards, sidebar in both themes, transfer form, empty state

110 tests, all passing. Three of them are documented defects recorded as expected failures.

Pointing this at your own application

- 1 Add your environment to `src/config/environments.ts` — URLs and behaviour flags only, no secrets.
- 2 **Explore before writing.** Open the app, dump its test ids, click the widgets. Every item in the section above came from doing that, and each would have cost more to discover through a failing test.
- 3 Model shared chrome as a component, not a page object, the way `BankShell` is. It has no URL and appears everywhere.
- 4 Write a page object per page. Declare components as readonly fields; write methods that read as business actions.
- 5 Add them to the fixture so every spec can reach them without constructing anything.
- 6 Put dataset facts in one file, read from a **clean** session, and add a test that cross-checks them against the application.

- 7 Record defects you do not control with `test.fail()`, and baseline recurring accessibility violations. Both keep the suite green and the findings visible.

Project structure

Every folder and file, with its purpose

Every file in the repository, grouped by role. Each entry links to its full reference — purpose, when and how to change it, and why the change belongs there.

playwright-ui-framework/	
├─ playwright.config.ts	Projects, reporters, timeouts, artifacts
├─ tsconfig.json	Strict TypeScript + path aliases
├─ eslint.config.mjs	Type-aware linting
├─ package.json	Dependencies and every command
├─ .env.example	Documented environment template
├─ Dockerfile / docker-compose.yml	CI-identical local runs
├─ Makefile	Discoverable task shortcuts
├─ .github/workflows/	Sharded CI with merged reports
├─ .vscode/	Shared editor behaviour
├─ docs/	This documentation (site + PDF + markdown)
├─ scripts/docs/	The documentation generator
├─ src/	
│ └─ config/	Validated environment resolution
│ └─ core/	BaseComponent, BasePage, locator resolution
│ └─ components/	The component library
│ └─ form/	17 form controls
│ └─ data/	Tables, grids, lists, trees, pagination, cards
│ └─ navigation/	Links, tabs, accordions, menus, breadcrumbs, steppers
│ └─ feedback/	Modals, alerts, tooltips, progress, loaders
│ └─ media/	Images, players, canvas, frames, carousels
│ └─ advanced/	Drag-drop, shadow DOM, editors, infinite scroll, charts
│ └─ fixtures/	The test and expect that specs import
│ └─ hooks/	Global setup/teardown, auth session capture
│ └─ pages/	Page objects
│ └─ api/	HTTP client for seeding data
│ └─ utils/	Logging, waits, network, files, a11y, visual
│ └─ reporters/	Custom run summary
│ └─ data/	Static test data
│ └─ types/	Shared type definitions
└─ tests/	ui/ · visual/ · a11y/

Root configuration

FILE	PURPOSE
<code>.editorconfig</code>	Charset, indentation, final newline and trailing-whitespace rules that apply in any editor, including ones without Prettier installed.
<code>.env</code>	Your machine-local configuration, created by copying <code>.env.example</code> .
<code>.env.example</code>	The documented list of every variable the framework understands, with safe defaults and no real values.

FILE	PURPOSE
<code>.gitignore</code>	Keeps dependencies, run artifacts, saved sessions and secrets out of the repository: <code>node_modules</code> , <code>test-results</code> , <code>reports</code> , <code>blob-report</code> , <code>playwright/.cache</code> , <code>storage/*.json</code> , and every <code>.env</code> file except <code>.env.example</code> .
<code>.nvmrc</code>	Pins Node 22 (LTS).
<code>.prettierignore</code>	Keeps Prettier away from generated and vendored output: <code>node_modules</code> , <code>dist</code> , <code>reports</code> , <code>test-results</code> , <code>playwright-report</code> , <code>blob-report</code> , <code>package-lock.json</code> and screenshot snapshot folders.
<code>.prettierrc.json</code>	Formatting rules: 100-character lines, single quotes, trailing commas, two-space indentation, LF endings.
<code>eslint.config.mjs</code>	Type-aware linting for the whole repository.
<code>package.json</code>	Declares dependencies, the Node engine range, the lint-staged rules and every command the team runs.
<code>playwright.config.ts</code>	The single entry point Playwright reads before anything else runs.
<code>tsconfig.json</code>	Compiler settings and path aliases for the whole repository.

CI, containers and editor tooling

FILE	PURPOSE
<code>.github/actions/setup/action.yml</code>	Node, dependencies and browsers, in the one place every job shares.
<code>.github/workflows/playwright.yml</code>	The GitHub Actions pipeline: a lint gate, a sharded three-browser test matrix, separate accessibility and visual jobs, and a final job that merges every shard into one HTML report.
<code>.vscode/extensions.json</code>	Prompts new contributors to install the Playwright test runner, ESLint, Prettier and EditorConfig extensions.
<code>.vscode/settings.json</code>	Format on save with Prettier, ESLint auto-fix on save, the workspace TypeScript version, non-relative import suggestions (so the aliases are offered), Playwright browser reuse, and noise folders hidden from search.
<code>docker-compose.yml</code>	Wraps the image with the environment variables, volume mounts and shared-memory settings a browser needs.

FILE	PURPOSE
<code>Dockerfile</code>	Builds a test image from the official Playwright base image pinned to the same version as <code>@playwright/test</code> , with browsers, system libraries and fonts already present.
<code>Makefile</code>	A discoverable front door for the npm scripts.

Documentation tooling

FILE	PURPOSE
<code>docs/README.md</code>	Explains what each documentation output is, how to regenerate them, and how to add an entry for a new file.
<code>scripts/docs/artifact.css</code>	The stylesheet for the hosted single-page manual, which has its own visual identity: IBM Plex Sans and Mono with Archivo for display, a cool slate ground and one ultramarine signal colour, plus full light, dark and system theme handling.
<code>scripts/docs/build-docs.mjs</code>	Merges the authored content modules with <code>docs/generated/api.json</code> and writes the HTML documentation site into <code>docs/site/</code> , including the single-page <code>print.html</code> used to produce the PDF and <code>artifact.html</code> used for the hosted version.
<code>scripts/docs/build-pdf.mjs</code>	Opens <code>docs/site/print.html</code> in the Chromium that ships with Playwright and prints it to <code>docs/playwright-ui-framework-documentation.pdf</code> with A4 pages, margins, and a header and footer carrying the document title and page numbers.
<code>scripts/docs/content/</code>	The prose source of this documentation, split into modules by area: <code>guides.mjs</code> for the long-form pages, and <code>files.*.mjs</code> for the per-file reference.
<code>scripts/docs/docs.css</code>	The single stylesheet for all three outputs: the screen layout (sidebar, tables, callouts, API blocks), the light and dark palettes, and the print sheet including the PDF cover page and page-break rules.
<code>scripts/docs/extract-api.mjs</code>	Walks the repository and parses every TypeScript file with the TypeScript compiler API, recording exported classes, functions, interfaces, types, constants and re-exports along with their JSDoc summaries and public member signatures.
<code>scripts/docs/highlight.mjs</code>	Colours every code snippet at build time.
<code>scripts/docs/render.mjs</code>	Turns the structured content blocks (paragraphs, lists, tables, code, callouts, step lists, trees) and the extracted API data into HTML.

Configuration (src/config)

FILE	PURPOSE
<code>src/config/env.config.ts</code>	The only file in the framework permitted to read <code>process.env</code> .
<code>src/config/environment.ts</code>	Static, non-secret settings for each environment — base URL, API URL, default retries and workers, and whether to ignore HTTPS errors.
<code>src/config/timeouts.ts</code>	Every wait in the framework references one of these named budgets — <code>INSTANT</code> , <code>SHORT</code> , <code>MEDIUM</code> , <code>LONG</code> , <code>EXTRA_LONG</code> , <code>TEST</code> , <code>HOOK</code> , <code>EXPECT</code> , <code>POLL_INTERVAL</code> , <code>DEBOUNCE</code> , <code>ANIMATION</code> — so tuning flakiness is a single-file change instead of a repo-wide hunt for magic numbers.

Core (src/core)

FILE	PURPOSE
<code>src/core/base.component.ts</code>	The most important file in the framework.
<code>src/core/base.page.ts</code>	Base class for page objects.
<code>src/core/locator.factory.ts</code>	Resolves any <code>SelectorLike</code> — a CSS or XPath string, an existing <code>Locator</code> , or a function that derives one from a scope — into a real <code>Locator</code> , applying the iframe and shadow-DOM options a component declares.

Types (src/types)

FILE	PURPOSE
<code>src/types/index.ts</code>	Every cross-cutting type in one place: environment names and shape, user roles, component options, <code>SelectorLike</code> , <code>ElementState</code> , click and fill options, table and sorting types, network mocking and capture shapes, accessibility and visual options, and the retry policy.

Component library entry points

FILE	PURPOSE
<code>src/components/component.factory.ts</code>	An ergonomic factory bound to a scope — a page, an iframe or a container locator.

Form controls

FILE	PURPOSE
<code>src/component s/form/autocomplete.ts</code>	Typeahead fields that fetch suggestions asynchronously.
<code>src/component s/form/button .ts</code>	Any clickable control: a real <code><button></code> , an <code><input type="submit"></code> , an anchor with <code>role="button"</code> , or a div a framework decided to treat as a button.
<code>src/component s/form/checkb ox.ts</code>	Native checkboxes and ARIA checkboxes, including the tri-state <code>indeterminate</code> case used by "select all" controls.
<code>src/component s/form/color- picker.ts</code>	Native <code><input type="color"></code> and custom swatch pickers: setting a hex value with the events frameworks expect, reading the value, picking a named swatch, and reading the rendered colour.
<code>src/component s/form/date- picker.ts</code>	Covers both interaction paths: typing into the input, and navigating the calendar popup month by month.
<code>src/component s/form/dropdo wn.ts</code>	Non-native dropdowns, comboboxes and listboxes from React-Select, MUI, Ant Design, Kendo and similar libraries.
<code>src/component s/form/file- upload.ts</code>	Uploads in all three flavours: a plain <code><input type="file"></code> , a hidden input behind a styled button (via the file chooser event), and a drag-and-drop zone (via a synthetic <code>DataTransfer</code> drop).
<code>src/component s/form/form.t s</code>	Treats a whole <code><form></code> as one component.
<code>src/component s/form/multi- select.ts</code>	Extends <code>Dropdown</code> for controls that hold several values at once, usually rendered as removable chips.
<code>src/component s/form/radio- group.ts</code>	A set of mutually exclusive options addressed by value, label or index.
<code>src/component s/form/rating .ts</code>	Star and heart rating controls: rating by position, reading the current and maximum rating, hover preview, read-only detection and clearing.
<code>src/component s/form/select .ts</code>	Native <code><select></code> elements.

FILE	PURPOSE
<code>src/components/form/slider.ts</code>	Range inputs and <code>role="slider"</code> controls, with three strategies because custom sliders rarely honour all of them: setting the value through the native setter (firing the events frameworks listen for), dragging the handle proportionally, and stepping with arrow keys.
<code>src/components/form/text-area.ts</code>	Multi-line fields.
<code>src/components/form/text-input.ts</code>	Single-line inputs of every flavour — text, email, password, number, tel, url, search — plus <code>contenteditable</code> fields.
<code>src/components/form/toggle.ts</code>	Switches: <code>role="switch"</code> , styled checkboxes, or a pair of divs.

Data display

FILE	PURPOSE
<code>src/components/data/cards.ts</code>	Cards and tiles — product cards, dashboard widgets, search results.
<code>src/components/data/data-grid.ts</code>	Extends <code>Table</code> for enterprise grids — AG Grid, MUI DataGrid, Kendo, PrimeNG — which differ in three ways that matter: rows are <code>role="row"</code> divs, data loads asynchronously, and only the visible window exists in the DOM.
<code>src/components/data/lists.ts</code>	Ordered and unordered lists, <code>role="list"</code> and card collections: reading items, clicking by text or index, membership checks, index lookup and order verification.
<code>src/components/data/pagination.ts</code>	Page controls: next, previous, first, last, a specific page, current page, total pages (from info text or the highest page link), page size, enabled state of the arrows, and <code>forEachPage()</code> for walking the whole set.
<code>src/components/data/table.ts</code>	HTML tables, with everything addressable by column name rather than index: reading all rows as objects, single cells, whole columns, finding rows by criteria, clicking a control inside a matching row, sorting, verifying sort order, and row selection.
<code>src/components/data/tree.ts</code>	Tree views and file explorers: expanding and collapsing, walking a path of ancestors to a leaf, selection state, nesting level from <code>aria-level</code> , listing visible nodes and children.

Navigation

FILE	PURPOSE
<code>src/components/navigation/accordion.ts</code>	Collapsible sections: expand, collapse, toggle, expansion state (via <code>aria-expanded</code> , the <code>open</code> attribute on <code>details</code> , or class names), section content, expand-all and collapse-all, and counting expanded sections to verify single-open behaviour.
<code>src/components/navigation/breadcrumb.ts</code>	Breadcrumb trails: reading the trail, the current page, navigating to an ancestor, going up a number of levels, and depth.
<code>src/components/navigation/link.ts</code>	Anchor, including the awkward parts: reading <code>href</code> and <code>target</code> , clicking and waiting for navigation, opening and returning a new tab, cheap reachability checking with a HEAD request, and internal-versus-external detection.
<code>src/components/navigation/menu.ts</code>	Navigation menus, menubars and context menus, including nested submenus.
<code>src/components/navigation/step-per.ts</code>	Multi-step wizards: next, back, finish, jumping to a step, current step number and name, all step names, completion and disabled state, and whether you are on the last step.
<code>src/components/navigation/tabs.ts</code>	Tab strips and their panels: selecting by name or index, reading the active tab and all tab names, disabled state, resolving the panel bound to a tab through <code>aria-controls</code> , and arrow-key navigation per the WAI-ARIA pattern.

Feedback

FILE	PURPOSE
<code>src/components/feedback/alert.ts</code>	Toasts, snackbars and inline alerts.
<code>src/components/feedback/modal.ts</code>	Dialogs, drawers and confirmations: waiting for open and close, title and body text, confirm, cancel, close, Escape, overlay click, arbitrary buttons by name — plus the two things modals routinely get wrong, focus trapping and ARIA semantics.
<code>src/components/feedback/progress.ts</code>	Two related components in one file.
<code>src/components/feedback/tooltip.ts</code>	Tooltips and popovers.

Media

FILE	PURPOSE
<code>src/components/media/canvas.ts</code>	Canvas elements — signature pads, drawing tools, charts and games.
<code>src/components/media/carousel.ts</code>	Image and content carousels: next, previous, going to a slide by indicator or by stepping, current slide index and text, slide count, verifying autoplay advances, and swiping for the mobile path.
<code>src/components/media/frame.ts</code>	An iframe wrapper.
<code>src/components/media/image.ts</code>	Images, including the cases that matter: whether the bitmap actually decoded (<code>complete</code> plus non-zero <code>naturalWidth</code> , not merely "element visible"), intrinsic dimensions, broken detection, lazy-loading, <code>srcset</code> entries and accessible alt-text rules.
<code>src/components/media/media-player.ts</code>	Video and audio driven through the media element's own DOM API rather than custom player chrome, so the same component works for any skin: play, pause, seek, volume, mute, playback rate, readiness, playback progress, ended state, current source and text tracks.

Advanced interactions

FILE	PURPOSE
<code>src/components/advanced/chart.ts</code>	SVG charts from D3, Highcharts, Chart.js in SVG mode or Recharts.
<code>src/components/advanced/drag-drop.ts</code>	Drag and drop with three strategies deliberately, because no single approach works across every library: Playwright's built-in <code>dragTo</code> for standard HTML5 drag, <code>dragManually()</code> with explicit mouse steps for react-dnd, dnd-kit and Sortable, and <code>dragWithDataTransfer()</code> dispatching synthetic events for custom handlers.
<code>src/components/advanced/infinite-scroll.ts</code>	Infinite-scroll and lazy-loaded lists.
<code>src/components/advanced/rich-text-editor.ts</code>	WYSIWYG editors — Quill, TinyMCE, CKEditor, ProseMirror, Slate.

FILE	PURPOSE
<code>src/component s/advanced/sha dow-dom.ts</code>	Web components with a shadow root.

Fixtures (src/fixtures)

FILE	PURPOSE
<code>src/fixtures/a uth.fixture.ts</code>	Extends the base fixtures with authenticated browsing: <code>authenticatedContext</code> and <code>authenticatedPage</code> for the standard user, and <code>pageAs(role)</code> for signing in as any role on demand.
<code>src/fixtures/b ank.fixture.ts</code>	Provides every page object ready-made, plus <code>signedIn</code> and <code>signInAs</code> for tests that need an authenticated session.
<code>src/fixtures/b ase.fixture.ts</code>	Composes the <code>test</code> object every spec uses.
<code>src/fixtures/c ustom- matchers.ts</code>	Domain-specific matchers that extend Playwright's <code>expect</code> : <code>toBeAccessible</code> , <code>toBeInteractive</code> , <code>toHaveNormalizedText</code> , <code>toHaveValidationError</code> , <code>toBeInViewport</code> and <code>toBeSorted</code> .
<code>src/fixtures/i ndex.ts</code>	Re-exports the composed <code>test</code> and the extended <code>expect</code> .

Hooks (src/hooks)

FILE	PURPOSE
<code>src/hooks/a uth.setup.t s</code>	Runs as the <code>setup</code> project before every other project.
<code>src/hooks/g lobal.setup. ts</code>	Runs once before the entire suite, in the parent process: creates the report, results, storage and download directories, removes the previous HTML report so a stale one cannot be mistaken for the current run, logs the resolved run configuration, and records the start time.
<code>src/hooks/g lobal.teardo wn.ts</code>	Runs once after the entire suite: removes the temporary scratch directory and logs total duration and where reports were written.

Page objects (src/pages)

FILE	PURPOSE
<code>src/pages/bank/account-detail.page.ts</code>	A single account: its balance and a filterable, sortable transaction table.
<code>src/pages/bank/accounts.page.ts</code>	The account list and the add/edit dialog it opens.
<code>src/pages/bank/apply-loan.page.ts</code>	Loan applications and their history — including the page that carries the application's planted defect.
<code>src/pages/bank/bank-shell.ts</code>	The application chrome — top bar and sidebar — modelled once as a component and composed into every page object.
<code>src/pages/bank/bill-pay.page.ts</code>	Bill payment: a biller search, an amount, a date and a memo.
<code>src/pages/bank/dashboard.page.ts</code>	The landing page: net-worth cards, quick-action tiles and the recent-activity widget.
<code>src/pages/bank/dropdown-panel.ts</code>	The <code>VISIBLE_LISTBOX</code> selector every custom dropdown in the application shares.
<code>src/pages/bank/index.ts</code>	Barrel for the SecureBank page objects.
<code>src/pages/bank/login.page.ts</code>	The sign-in page — the only page reachable without authenticating, and therefore the entry point for every other page object.
<code>src/pages/bank/notifications.page.ts</code>	The notification centre and the unread badges it drives.
<code>src/pages/bank/profile.page.ts</code>	Profile, password change and security settings.
<code>src/pages/bank/send-money.page.ts</code>	External payments to a saved payee, and the add-payee dialog.
<code>src/pages/bank/transactions.page.ts</code>	All activity across every account: search, account filter, type filter, three sortable columns, pagination and CSV export.
<code>src/pages/bank/transfer.page.ts</code>	Internal transfers.

API client (src/api)

FILE	PURPOSE
<code>src/api/api.client.ts</code>	A thin HTTP client for test setup, not for testing the API itself.

Reporters (src/reporters)

FILE	PURPOSE
<code>src/reporters/summary.reporter.ts</code>	A custom Playwright reporter that prints a compact end-of-run summary and writes <code>reports/summary.json</code> containing totals, every failure with its first error line, flaky tests, the five slowest tests and the full test list with tags and durations.

Utilities (src/utls)

FILE	PURPOSE
<code>src/utls/ally.utls.ts</code>	Runs axe-core against a page or a subtree, attaches both a JSON and an HTML report to the test result, formats violations into a readable failure message, and fails the test by default when violations are found.
<code>src/utls/browser.utls.ts</code>	Console capture: <code>captureConsole</code> records console errors, uncaught exceptions and failed requests into a container the <code>consoleErrors</code> fixture owns, so a test can assert the page logged nothing unexpected.
<code>src/utls/data.utls.ts</code>	Seeded, reproducible test data.
<code>src/utls/date.utls.ts</code>	<code>toIsoDate</code> and <code>format</code> for writing dates into inputs and comparing them, and <code>humanizeDuration</code> for readable log and report output.
<code>src/utls/file.utls.ts</code>	Creating a directory if it is missing, and resolving a path against the test-data directory so callers can pass a name relative to <code>src/data</code> instead of a relative path that depends on where the caller lives.
<code>src/utls/logger.ts</code>	Structured logging with console and file output, deliberately dependency-free.
<code>src/utls/network.utls.ts</code>	The network control surface for a page or context: <code>mock</code> a response, <code>modifyResponse</code> to rewrite a real one, <code>abort</code> to simulate failure, <code>delay</code> to exercise loading states, block heavy assets or third-party trackers, capture and search requests, wait for a specific response, and toggle offline.

FILE	PURPOSE
<code>src/utils/retry.util.ts</code>	<code>waitUntil</code> — polls a condition until it holds or the timeout elapses, and fails with a message naming the condition.
<code>src/utils/string.util.ts</code>	<code>normalizeText</code> — collapses the whitespace the DOM inserts, so an assertion compares the text a person sees rather than the text the markup happens to contain.

Test data (src/data)

FILE	PURPOSE
<code>src/data/personas.ts</code>	The seven SecureBank accounts and the facts about the seeded dataset that tests assert against.
<code>src/data/README.md</code>	Explains what belongs in the data directory: static fixtures only, with anything unique per run generated at runtime so parallel workers never collide.

Tests folder

FILE	PURPOSE
<code>tests/ally/.gitkeep</code>	Placeholder for the <code>accessibility</code> project's specs — axe scans across key pages and states.
<code>tests/ally/bank.ally.spec.ts</code>	Automated axe scans across all ten pages and both themes, with the application's existing violations baselined.
<code>tests/README.md</code>	The rules for writing specs: import from <code>@fixtures/index</code> , tag every test, one behaviour per test, no selectors in specs, seed state through the API, and never use <code>waitForTimeout</code> .
<code>tests/ui/.gitkeep</code>	Placeholder keeping <code>tests/ui/</code> in version control.
<code>tests/ui/account-settings.spec.ts</code>	Notifications and profile — the two pages the application's catalogue does not cover.
<code>tests/ui/account.spec.ts</code>	Covers TC-ACC-001..006 across three personas, plus the add dialog and the detail page.
<code>tests/ui/apply-loan.spec.ts</code>	Covers TC-LOAN-001..006 — including reproducing the application's planted defect.

FILE	PURPOSE
<code>tests/ui/dashboard.spec.ts</code>	Covers the application's own TC-DASH-001..006, plus the stat cards and every quick action.
<code>tests/ui/login.spec.ts</code>	Authentication: the happy path, a wrong password, a suspended account, password visibility, remember-me, and a fixture-integrity check.
<code>tests/ui/money-movement.spec.ts</code>	Covers TC-SEND-001..006 and TC-BILL-001..006 — the two external payment flows.
<code>tests/ui/navigation.spec.ts</code>	The application shell: every sidebar destination, the brand link, the unread badge, responsive behaviour and the skip link.
<code>tests/ui/transactions.spec.ts</code>	Covers TC-TXN-001..006: loading, search, empty state, account filter, sorting, pagination, the type filter and the CSV export.
<code>tests/ui/transfer.spec.ts</code>	Covers TC-XFER-001..006: form loading, validation, the confirmation step, the success page, and the resulting balances.
<code>tests/visual/.gitkeep</code>	Placeholder for the <code>visual</code> project's specs.
<code>tests/visual/bank.visual.spec.ts</code>	Visual regression on the login page, the dashboard cards, the sidebar in both themes, the transfer form and the empty transactions state.

Repository documentation

FILE	PURPOSE
<code>docs/ARCHITECTURE.md</code>	The concise markdown version of the design rationale, for reading on GitHub without building the documentation site.
<code>docs/COMPONENTS.md</code>	The markdown summary of every component and its key methods, for quick lookup in an editor or on GitHub.
<code>README.md</code>	The front door: quick start, the folder map, how a test looks, the command table, environment handling, reporting outputs and links to the deeper documentation.

Reference: configuration & tooling

Root config, CI, containers, editor and docs tooling

Root configuration

`.editorconfig`

Charset, indentation, final newline and trailing-whitespace rules that apply in any editor, including ones without Prettier installed.

WHEN YOU NEED TO CHANGE IT

- You change indentation or line endings in Prettier and want editors to agree before formatting runs.

HOW TO CHANGE IT

- 1 Keep this file and `.prettierrc.json` consistent; if they disagree, files churn between contributors.

WHY THE CHANGE BELONGS HERE

It is the lowest-common-denominator agreement that works for everyone on the team regardless of tooling.

Related: `.prettierrc.json`

`.env`

Your machine-local configuration, created by copying `.env.example`. It is git-ignored. `env.config.ts` loads it first, then `.env.<TEST_ENV>` on top, and real process variables win over both so CI secrets are never overwritten.

WHEN YOU NEED TO CHANGE IT

- You point your machine at a different environment, enable debugging, or set local credentials.

HOW TO CHANGE IT

- 1
- Copy the template and edit. For a one-off run, prefer an inline variable so your file stays clean.

```
cp .env.example .env
BASE_URL=https://pr-451.preview.example.com npm run test:smoke
```

WHY THE CHANGE BELONGS HERE

Keeping machine-specific values out of the repository is what lets ten engineers point the same suite at ten different environments without conflict.

Related: `.env.example` · `src/config/env.config.ts`

`.env.example`

The documented list of every variable the framework understands, with safe defaults and no real values. Copy it to `.env` to configure a local run.

VARIABLE	DEFAULT	EFFECT
<code>TEST_ENV</code>	<code>dev</code>	Selects an entry from <code>src/config/environments.ts</code> .
<code>BASE_URL</code>	environment default	Overrides the target URL — use for preview deployments.
<code>HEADLESS</code>	<code>true</code>	Forced to true in CI regardless of this value.
<code>SLOW_MO</code>	<code>0</code>	Milliseconds of delay per action, for demos and debugging.
<code>WORKERS</code> / <code>RETRIES</code>	environment default	Override parallelism and retry policy.
<code>TIMEOUT_ACTION/NAVIGATION/EXPECT/TEST</code>	see <code>timeouts.ts</code>	Timeout budgets in milliseconds.
<code>TRACE</code> / <code>VIDEO</code> / <code>SCREENSHOT</code>	<code>retain-on-failure</code>	Artifact capture policy.
<code>VIEWPORT_WIDTH</code> / <code>VIEWPORT_HEIGHT</code>	1920 x 1080	Default viewport.
<code>LOG_LEVEL</code>	<code>info</code>	<code>error</code> <code>warn</code> <code>info</code> <code>debug</code> <code>trace</code> .

VARIABLE	DEFAULT	EFFECT
<code>*_USER</code> / <code>*_PASSWORD</code>	empty	Credentials per role, read through <code>getUser(role)</code> .
<code>API_BASE_URL</code> / <code>API_TOKEN</code>	environment default	Used by the API client for seeding data.
<code>ALLURE_ENABLED</code>	<code>false</code>	Adds the Allure reporter to the run.

WHEN YOU NEED TO CHANGE IT

- You add a variable to the zod schema in `env.config.ts`. The two must always be updated together.

HOW TO CHANGE IT

- 1 Add the variable to the schema first, then document it here with a safe placeholder value and a comment describing its effect.

```
# Feature flag overrides applied before every test
FEATURE_FLAGS=
```

WHY THE CHANGE BELONGS HERE

This file is the only discoverable inventory of what the framework can be configured with. An undocumented variable is one that nobody outside its author can use.

WATCH OUT FOR

- Never put a real credential here — the file is committed.

Related: `src/config/env.config.ts` · `.env`

`.gitignore`

Keeps dependencies, run artifacts, saved sessions and secrets out of the repository:
`node_modules`, `test-results`, `reports`, `blob-report`, `playwright/.cache`, `storage/*.json`,
and every `.env` file except `.env.example`.

WHEN YOU NEED TO CHANGE IT

- You add a new artifact directory, or a new secret-bearing file.

HOW TO CHANGE IT

- 1 Add the pattern, and if the file was already committed, remove it from the index explicitly.

```
git rm --cached path/to/file
```

WHY THE CHANGE BELONGS HERE

Saved storage states in `storage/` are live session cookies. Committing one is equivalent to committing a password.

WATCH OUT FOR

- `.env.example` is deliberately un-ignored — it is the template that documents every supported variable.

Related: `.env.example` · `src/config/env.config.ts`

`.nvmrc`

Pins Node 22 (LTS). `nvm use` reads it locally and the CI workflow reads it through `node-version-file`, so local and CI runtimes match.

WHEN YOU NEED TO CHANGE IT

- The project moves to a new Node LTS release.

HOW TO CHANGE IT

- 1 Change the version, run a full suite locally, then let CI confirm. Update the `engines` range in `package.json` in the same commit.

WHY THE CHANGE BELONGS HERE

Playwright supports LTS releases. Odd-numbered Node releases regularly break tooling that depends on module-resolution behaviour, and a version pin turns that class of surprise into a deliberate upgrade.

Related: `package.json` · `.github/workflows/playwright.yml`

`.prettierignore`

Keeps Prettier away from generated and vendored output: `node_modules`, `dist`, `reports`, `test-results`, `playwright-report`, `blob-report`, `package-lock.json` and screenshot snapshot folders.

WHEN YOU NEED TO CHANGE IT

- You add a folder of generated artifacts.

HOW TO CHANGE IT

- 1 Append the folder. Mirror the entry in `.gitignore` if the output should also stay out of version control.

WHY THE CHANGE BELONGS HERE

Formatting generated files creates noisy diffs and can corrupt binary snapshots.

Related: `.gitignore`

`.prettierrc.json`

12 lines

Formatting rules: 100-character lines, single quotes, trailing commas, two-space indentation, LF endings. Applied by `npm run format`, by the editor on save, and by the pre-commit hook.

WHEN YOU NEED TO CHANGE IT

- The team agrees on a different house style. This is a taste decision, so change it once and reformat the whole repo in a single commit.

HOW TO CHANGE IT

- 1 Edit the option, then reformat everything so no future diff mixes style churn with real changes.

```
npm run format && git commit -am "chore: reformat with new Prettier settings"
```

WHY THE CHANGE BELONGS HERE

Formatting arguments waste review time. One config, applied mechanically, ends them.

Related: `.prettierrignore` · `eslint.config.mjs`

eslint.config.mjs

97 lines

Type-aware linting for the whole repository. It layers the JavaScript recommended rules, `typescript-eslint` type-checked rules, Playwright-specific rules for test files, and Prettier last so formatting never fights the linter.

- `no-floating-promises` and `no-misused-promises` are errors — an un-awaited Playwright call is the single most common source of phantom passes.
- `consistent-type-imports` keeps type-only imports out of the runtime bundle.
- `explicit-function-return-type` is a warning, and `--max-warnings=0` in the `lint` script promotes it to a build failure.
- Test files additionally get `eslint-plugin-playwright`; `expect-expect` is disabled because assertions often live inside page objects and components.
- `.mjs` and `.js` files opt out of type-aware rules, since they are not part of the TypeScript program.

WHEN YOU NEED TO CHANGE IT

- A rule produces repeated false positives across the codebase.
- You add a new file category that needs different rules (for example a scripts folder that may use `console`).
- You adopt a new plugin.

HOW TO CHANGE IT

- 1 Add a scoped block rather than weakening a rule globally. Order matters — later blocks win, and `prettier` must stay last.

```
{
  files: ['src/reporters/**/*.ts', 'scripts/**/*.ts'],
  rules: { 'no-console': 'off' },
},
```

- 2 For a one-off exception, disable on the line and say why. An undocumented disable comment will be copied by the next person.

```
// React tracks the native setter, so a direct assignment is ignored.  
// eslint-disable-next-line @typescript-eslint/unbound-method  
const setter = Object.getOwnPropertyDescriptor(window.HTMLInputElement.prototype, 'value
```

WHY THE CHANGE BELONGS HERE

Lint rules encode the review comments you would otherwise write by hand every sprint. Putting the exception next to the code — with a reason — keeps the global standard strict.

Related: `.prettierrc.json` · `package.json`

package.json

79 lines

Declares dependencies, the Node engine range, the lint-staged rules and every command the team runs. The scripts are the documented interface to the framework: if a task is worth doing twice, it belongs here rather than in someone's shell history.

SCRIPT GROUP	SCRIPTS	PURPOSE
Run	<code>test</code> , <code>test:headed</code> , <code>test:ui</code> , <code>test:debug</code>	Everyday execution and debugging.
Slice	<code>test:chromium</code> , <code>test:firefox</code> , <code>test:webkit</code> , <code>test:mobile</code> , <code>test:smoke</code> , <code>test:regression</code> , <code>test:failed</code>	Run a subset by browser or tag.
Specialised	<code>test:a11y</code> , <code>test:visual</code> , <code>test:visual:update</code>	Accessibility and screenshot suites.
Report	<code>report</code> , <code>report:allure</code>	Open generated reports.
Quality	<code>typecheck</code> , <code>lint</code> , <code>lint:fix</code> , <code>format</code> , <code>format:check</code> , <code>validate</code>	Everything CI enforces, runnable locally.
Housekeeping	<code>clean</code> , <code>install:browsers</code> , <code>codegen</code> , <code>prepare</code>	Environment maintenance.
Docs	<code>docs</code> , <code>docs:api</code> , <code>docs:html</code> , <code>docs:pdf</code>	Regenerate this documentation.

WHEN YOU NEED TO CHANGE IT

- You add a dependency, or remove one that is no longer imported anywhere.
- A team workflow needs a name (a new tag suite, a new environment shortcut).
- The supported Node range changes.

HOW TO CHANGE IT

- 1 Add a script named after the intent, not the flags, so it reads well in CI logs and in `make help`.

```
"test:checkout": "playwright test tests/ui/checkout --project=chromium",  
"test:staging": "cross-env TEST_ENV=staging playwright test"
```

- 2 Install dependencies as devDependencies. Nothing here ships to production, and a stray runtime dependency confuses vulnerability scanners.

```
npm install --save-dev <package>
```

WHY THE CHANGE BELONGS HERE

Scripts are the contract between humans, the Makefile and CI. The GitHub workflow calls the same commands you run locally, so a green local `npm run validate` means a green CI lint job.

WATCH OUT FOR

- `zod` is pinned to v3 on purpose: v4's CommonJS build fails to initialise under the transform Playwright applies to config files.
- There is no `"type": "module"` field. Playwright transpiles TypeScript to CommonJS, which keeps every CommonJS-only dependency interoperable.

Related: `Makefile` · `.github/workflows/playwright.yml`

playwright.config.ts

225 lines

The single entry point Playwright reads before anything else runs. It declares the browser projects, timeouts, artifact policy, reporters and global hooks. It contains no environment values of its own: everything variable is imported from `src/config/env.config.ts`, which has already validated it.

THE SHAPE OF THE FILE

TypeScript

```
const reporters: ReporterDescription[] = [ /* list, html, junit, json, summary, ctrf */ ];

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  workers: config.workers ?? (config.isCI ? '50%' : undefined),
  retries: config.retries,
  timeout: config.timeouts.test,
  forbidOnly: config.isCI,
  expect: { timeout: config.timeouts.expect, toHaveScreenshot: { /* ... */ } },
  globalSetup: './src/hooks/global.setup.ts',
  globalTeardown: './src/hooks/global.teardown.ts',
  use: { baseURL: config.baseURL, trace: config.artifacts.trace, testIdAttribute: 'data-testid' },
  projects: [ /* setup, chromium, firefox, webkit, edge, mobile, tablet, visual, accessibility */ ],
  reporter: reporters,
});
```

WHEN YOU NEED TO CHANGE IT

- You add a browser, device profile or a new specialised suite (for example a `performance` or `smoke-only` project).
- You need a different default timeout, retry count or worker count for the whole suite.
- You want to add or remove a reporter.
- The application under test must be started by the test run itself (uncomment the `webServer` block).
- You change where snapshots or artifacts are written.

HOW TO CHANGE IT

- 1 To add a browser or device project, copy an existing entry and give it a unique `name`. Keep `dependencies: ['setup']` so it inherits authenticated sessions, and keep the `testIgnore` for the specialised folders so visual and a11y specs do not run in every browser.

```
{
  name: 'galaxy-s23',
  use: { ...devices['Galaxy S9+'] },
  dependencies: ['setup'],
  testIgnore: ['**/visual/**', '**/a11y/**'],
},
```

- 2 To change a timeout, do NOT hard-code a number here. Change the named budget in `src/config/timeouts.ts` or the environment variable it reads, so every wait in the framework moves together.

- 3 To start the app under test with the suite, uncomment the `webServer` block at the bottom and point `command` at your dev server script.

```
webServer: {  
  command: 'npm run start',  
  url: config.baseURL,  
  reuseExistingServer: !config.isCI,  
  timeout: 120_000,  
},
```

- 4 To add a reporter, push it onto the `reporters` array before `defineConfig` runs. Guard CI-only or opt-in reporters with a condition so local runs stay fast.

```
if (config.allureEnabled) {  
  reporters.push(['allure-playwright', { resultsDir: 'reports/allure-results' }]);  
}
```

WHY THE CHANGE BELONGS HERE

Playwright resolves this file once, in the parent process, before workers exist. Anything decided here applies to the entire run and cannot be overridden per test, which is exactly why environment resolution happens in `env.config.ts` first: by the time this file executes, configuration is already validated and typed.

WATCH OUT FOR

- Adding a project without `dependencies: ['setup']` means its tests start signed out.
- The `visual` project is deliberately pinned to one browser and viewport; running visual specs elsewhere produces meaningless diffs.
- Passing `--reporter=...` on the command line replaces the whole reporter list, so `reports/summary.json` is not written on those runs.

Related: `src/config/env.config.ts` · `src/config/timeouts.ts` · `src/hooks/global.setup.ts`

tsconfig.json

52 lines

Compiler settings and path aliases for the whole repository. Strict mode is on in full, including `noUncheckedIndexedAccess`, so array and record access must be handled rather than assumed.

SETTING	VALUE	WHY
<code>strict</code> + <code>noUncheckedIndexedAccess</code>	<code>true</code>	A test framework that lies about types produces failures nobody can diagnose.
<code>module</code> / <code>moduleResolution</code>	<code>ESNext</code> / <code>Bundler</code>	Lets source use extensionless imports, which Playwright resolves at runtime.
<code>noEmit</code> (via script)	<code>typecheck only</code>	Playwright transpiles at runtime; the repo never ships compiled JS.
<code>paths</code>	<code>@core/*</code> , <code>@components/*</code> , ...	Import by role rather than by relative depth.

WHEN YOU NEED TO CHANGE IT

- You add a new top-level folder under `src/` that deserves its own alias.
- You need to loosen or tighten a compiler check for the whole repo.
- You upgrade TypeScript and a new flag becomes relevant.

HOW TO CHANGE IT

- 1 To add an alias, add it to `paths` and use it immediately; both `tsc` and Playwright read this file, so no second registration is needed.

```
"paths": {
  "@core/*": [".src/core/*"],
  "@flows/*": [".src/flows/*"]
}
```

- 2 Never disable `strict` to make an error go away. Fix the type, or narrow with a documented guard.

WHY THE CHANGE BELONGS HERE

Aliases and strictness are repo-wide contracts. Changing them in one place keeps editor, `npm run typecheck` and the Playwright runtime in agreement; a local override in one file would drift.

WATCH OUT FOR

- `baseUrl` is deliberately absent: TypeScript 6 deprecates it, and `paths` alone resolves relative to this file.

Related: `package.json` · `eslint.config.mjs`

CI, containers and editor tooling

`.github/actions/setup/action.yml`

88 lines

Node, dependencies and browsers, in the one place every job shares. A composite action so the setup block is written once rather than copied into all five jobs.

This exists because installing dominated the pipeline. On one measured run `npm ci` took 421 seconds per job while the tests took 8 — installation was 97% of the wall time, and the suite it was there to run was a rounding error.

The npm cache was already hitting, so the cost was not fetching tarballs. It was npm rebuilding `node_modules` from them, plus an audit round-trip that every parallel job made at once. The contention shows in the spread: one shard installed in 23 seconds and the others took seven minutes for identical work.

What it does

STEP	WHY
Restore <code>node_modules</code>	Keyed on <code>.nvmrc</code> + <code>package-lock.json</code> . A hit skips installation entirely and costs about as long as untarring the directory.
Install on a miss only	<code>--prefer-offline --no-audit --fund=false</code> removes two registry round-trips; <code>timeout 300</code> bounds it so a stall fails loudly rather than silently.
Restore browsers	Only when the <code>browsers</code> input is set. Keyed on the lockfile, because that is what pins the Playwright version the binaries belong to.
<code>install-deps</code> always	The apt packages are not cached — every job starts on a clean runner — so the system libraries are installed even on a cache hit.

NOTE The job graph is what makes the cache pay: the first job has no dependencies and every other job declares `needs` on it, so the cache is populated before anything else starts. Removing that ordering would put every job back to a cold install.

WHEN YOU NEED TO CHANGE IT

- A job needs a browser it currently does not.
- Installation gets slow again.

HOW TO CHANGE IT

- 1 Pass the browser through the `browsers` input rather than adding a `playwright install` step to the job. A step outside this action gets no caching.

```
- uses: ./github/actions/setup
  with:
    browsers: chromium
```

- 2 If a cache key ever needs to change, change it here once. The key deliberately hashes the lockfile and nothing else about the source, so editing a test never invalidates the dependency cache.

WHY THE CHANGE BELONGS HERE

Five copies of a setup block drift apart. One that every job calls means a caching change is made once and is either right everywhere or wrong everywhere — which is the easier of the two to notice.

Related: `.github/workflows/playwright.yml` · `package.json`

`.github/workflows/playwright.yml`

175 lines

The GitHub Actions pipeline: a lint gate, a sharded three-browser test matrix, separate accessibility and visual jobs, and a final job that merges every shard into one HTML report. It runs on pushes, pull requests, a nightly schedule, and on demand with an environment and tag filter.

JOB	DEPENDS ON	WHAT IT DOES
<code>lint</code>	—	<code>npm run typecheck</code> , <code>lint</code> , <code>format:check</code> . Fails fast before any browser starts.
<code>test</code>	<code>lint</code>	3 browsers x 3 shards = 9 parallel jobs, each uploading a blob report.
<code>accessibility</code>	<code>lint</code>	Runs the <code>accessibility</code> project and uploads the axe reports.
<code>visual</code>	<code>lint</code>	Runs the <code>visual</code> project on a fixed runner image so baselines stay valid.
<code>report</code>	all of the above	<code>playwright merge-reports</code> turns the blob shards into one browsable HTML report.

WHEN YOU NEED TO CHANGE IT

- You add a browser project or want a different shard count.
- A new secret or repository variable is needed.
- You want a different trigger — for example running only `@smoke` on pull requests and everything nightly.
- Artifact retention needs to change.

HOW TO CHANGE IT

- 1 To change sharding, edit the matrix and the `--shard` argument together. They must agree or tests will be silently skipped.

```
strategy:
  matrix:
    project: [chromium, firefox, webkit]
    shard: [1, 2, 3, 4]
# ...
run: npx playwright test --project=${{ matrix.project }} --shard=${{ matrix.shard }}/4
```

- 2 To run a reduced suite on pull requests and the full suite nightly, branch on the event name.

```
run: npx playwright test --project=${{ matrix.project }} ${{ github.event_name == 'pull_
```

- 3 To add a credential, define it in the repository settings and pass it through `env:` on the step. Never inline a secret value.

```
env:
  PARTNER_API_KEY: ${ secrets.PARTNER_API_KEY }
```

WHY THE CHANGE BELONGS HERE

Sharding is what keeps wall-clock time flat as the suite grows; blob reports plus a merge step are what make a sharded run still readable as one report. Splitting a11y and visual into their own jobs keeps their very different failure modes out of the functional matrix.

WATCH OUT FOR

- Visual baselines are rendered by a specific OS and browser build. Changing `runs-on` will invalidate every screenshot.
- The browser cache key is derived from `package-lock.json`; bumping Playwright automatically busts it.

Related: `playwright.config.ts` · `package.json` · `Dockerfile`

`.vscode/extensions.json`

9 lines

Prompts new contributors to install the Playwright test runner, ESLint, Prettier and EditorConfig extensions.

WHEN YOU NEED TO CHANGE IT

- A tool becomes part of the standard workflow.

HOW TO CHANGE IT

- 1 Add its marketplace identifier to `recommendations`.

WHY THE CHANGE BELONGS HERE

The Playwright extension provides in-editor run, debug and trace viewing; without it, newcomers fall back to raw CLI and miss the best debugging tools available.

Related: `.vscode/settings.json`

`.vscode/settings.json`

20 lines

Format on save with Prettier, ESLint auto-fix on save, the workspace TypeScript version, non-relative import suggestions (so the aliases are offered), Playwright browser reuse, and noise folders hidden from search.

WHEN YOU NEED TO CHANGE IT

- A setting helps everyone on the team. Personal preferences belong in your user settings, not here.

HOW TO CHANGE IT

- 1 Add the key. Keep the list short — a bloated workspace settings file quietly overrides personal setups.

WHY THE CHANGE BELONGS HERE

Shared editor behaviour means the code you commit is already formatted and linted, so the pre-commit hook rarely has to intervene.

Related: `.vscode/extensions.json`

`docker-compose.yml`

23 lines

Wraps the image with the environment variables, volume mounts and shared-memory settings a browser needs. Reports, results and the tests folder are mounted so output and updated baselines land back on your machine.

WHEN YOU NEED TO CHANGE IT

- A new environment variable must reach the container, or you need another directory mounted.

HOW TO CHANGE IT

- 1 Pass variables through with a default so the file works even when your shell has nothing set.

```
environment:  
  FEATURE_FLAGS: ${FEATURE_FLAGS:-}
```

WHY THE CHANGE BELONGS HERE

Chromium crashes on Docker's default 64 MB of shared memory; `shm_size: 2gb` and `ipc: host` are the fix, and they belong with the run configuration rather than in every engineer's notes.

Related: `Dockerfile`

`Dockerfile`

Builds a test image from the official Playwright base image pinned to the same version as `@playwright/test`, with browsers, system libraries and fonts already present. Running here reproduces CI rendering exactly.

USAGE

Shell

```
docker compose build
docker compose run --rm ui-tests --project=chromium --grep @smoke
```

WHEN YOU NEED TO CHANGE IT

- You upgrade Playwright — the image tag must move with it.
- The suite needs an extra system package (a font, a PDF tool, a database client).

HOW TO CHANGE IT

- 1 Keep the image tag and the installed `@playwright/test` version identical. A mismatch produces confusing driver errors.

```
FROM mcr.microsoft.com/playwright:v1.62.1-jammy
```

- 2 Add system packages before `npm ci` so the dependency layer stays cached.

```
RUN apt-get update && apt-get install -y --no-install-recommends fonts-noto-color-emoji
```

WHY THE CHANGE BELONGS HERE

Font and rendering differences between a developer laptop and CI are the classic cause of visual tests that pass locally and fail in the pipeline. The container removes that variable.

Related: `docker-compose.yml` · `.github/workflows/playwright.yml`

Makefile

A discoverable front door for the npm scripts. `make help` prints every target with its description, which is faster to scan than `package.json`.

WHEN YOU NEED TO CHANGE IT

- You add an npm script that people will run often, or a multi-step workflow worth naming.

HOW TO CHANGE IT

- 1 Add the target with a `##` comment — `make help` builds itself from those comments — and add the name to `.PHONY`.

```
smoke-staging: ## Run smoke tests against staging
TEST_ENV=staging npm run test:smoke
```

WHY THE CHANGE BELONGS HERE

Newcomers look for a Makefile. Giving them one that self-documents shortens the path from clone to first test run.

WATCH OUT FOR

- Make requires real tab characters for recipe indentation; spaces produce a confusing syntax error.

Related: `package.json`

Documentation tooling

`docs/README.md`

41 lines

Explains what each documentation output is, how to regenerate them, and how to add an entry for a new file. The first thing to read when the docs build fails.

WHEN YOU NEED TO CHANGE IT

- You add a documentation output or change the generation commands.

HOW TO CHANGE IT

- 1 Update the table and the commands; keep it consistent with the scripts in `package.json`.

WHY THE CHANGE BELONGS HERE

The build failing on an undocumented file is only a helpful signal if the fix is written down next to it.

Related: `scripts/docs/build-docs.mjs` · `scripts/docs/content/`

scripts/docs/artifact.css

The stylesheet for the hosted single-page manual, which has its own visual identity: IBM Plex Sans and Mono with Archivo for display, a cool slate ground and one ultramarine signal colour, plus full light, dark and system theme handling.

WHEN YOU NEED TO CHANGE IT

- You restyle the hosted manual, or add a block type that needs styling there.

HOW TO CHANGE IT

- Colours are defined as tokens in three places that must stay in step: the bare `:root` (light), the `prefers-color-scheme: dark` block guarded with `:root:not([data-theme='light'])`, and `:root[data-theme='dark']` for the explicit toggle. Style through the tokens, never with a literal inside a theme block.

WHY THE CHANGE BELONGS HERE

The hosted manual is read on someone else's screen in their theme, so every colour has to resolve in all three theme states — a colour defined only inside a dark-mode block renders one theme's text on the other theme's background.

Related: `scripts/docs/build-docs.mjs` · `scripts/docs/docs.css`

scripts/docs/build-docs.mjs

696 lines

Merges the authored content modules with `docs/generated/api.json` and writes the HTML documentation site into `docs/site/`, including the single-page `print.html` used to produce the PDF and `artifact.html` used for the hosted version. It fails the build if any file in the repository has no documentation entry.

RUN IT

Shell

```
npm run docs:html    # site only
npm run docs         # extract + site + PDF
```

NOTE The completeness check is the point: adding a source file without documenting it breaks `npm run docs`, so the documentation cannot silently fall behind the framework.

WHEN YOU NEED TO CHANGE IT

- You add a documentation page, reorder navigation, or change the visual design of the site.

HOW TO CHANGE IT

- 1 Add the page to the `PAGES` array with its id, title and content, and it appears in the navigation of every page automatically.
- 2 To document a new source file, add an entry keyed by its repository-relative path to the matching module in `scripts/docs/content/`.

WHY THE CHANGE BELONGS HERE

Generating the site from data rather than hand-maintaining HTML is what makes it feasible to keep 100+ file descriptions accurate over time.

Related: `scripts/docs/content/` · `scripts/docs/build-pdf.mjs`

`scripts/docs/build-pdf.mjs`

58 lines

Opens `docs/site/print.html` in the Chromium that ships with Playwright and prints it to `docs/playwright-ui-framework-documentation.pdf` with A4 pages, margins, and a header and footer carrying the document title and page numbers.

RUN IT

Shell

```
npm run docs:pdf
```

WHEN YOU NEED TO CHANGE IT

- You want different paper size, margins, or header and footer content.

HOW TO CHANGE IT

- 1 Adjust the options passed to `page.pdf()`. The header and footer are HTML fragments with Chromium's special classes for page numbers.

```
await page.pdf({
  path: outputPath,
  format: 'A4',
  printBackground: true,
  margin: { top: '18mm', bottom: '18mm', left: '14mm', right: '14mm' },
  footerTemplate: '<div style="font-size:8px"><span class="pageNumber"></span> / <span c
});
```

WHY THE CHANGE BELONGS HERE

The framework already depends on Playwright, so generating the PDF with it adds no new tooling and guarantees the PDF matches the HTML exactly.

Related: `scripts/docs/build-docs.mjs`

scripts/docs/content/

The prose source of this documentation, split into modules by area: `guides.mjs` for the long-form pages, and `files/*.mjs` for the per-file reference. Each file entry carries a purpose, when to change it, how to change it, why the change belongs there, gotchas and related files.

THE SHAPE OF A FILE ENTRY

`TypeScript`

```
'src/components/form/button.ts': {
  group: 'components-form',
  purpose: 'What the file is for.',
  changeWhen: ['Situations that require editing it'],
  changeHow: [{ text: 'Step', code: 'snippet' }],
  why: 'Why this is the right place for that change.',
  gotchas: ['Traps'],
  related: ['other/file.ts'],
}
```

WHEN YOU NEED TO CHANGE IT

- Any time you add, remove or meaningfully change a file in the repository.

HOW TO CHANGE IT

- 1 Add or edit the entry keyed by the repository-relative path, then run `npm run docs`. The build fails if a file has no entry, which is your reminder.

WHY THE CHANGE BELONGS HERE

Keeping prose beside a generated API surface gives the best of both: intent and rationale are written by a human, while signatures and members are always read from the code.

Related: `scripts/docs/build-docs.mjs`

`scripts/docs/docs.css`

The single stylesheet for all three outputs: the screen layout (sidebar, tables, callouts, API blocks), the light and dark palettes, and the print sheet including the PDF cover page and page-break rules.

WHEN YOU NEED TO CHANGE IT

- You change the visual design, add a block type that needs styling, or adjust the PDF page setup.

HOW TO CHANGE IT

- 1 Screen colours live in the `:root` custom properties and their dark-scheme overrides. Print rules live in the `body.print` and `@media print` sections at the bottom.

```
body.print .file { page-break-inside: avoid; break-inside: avoid; }
```

WHY THE CHANGE BELONGS HERE

One stylesheet for screen and print means a design change updates the site and the PDF together, rather than leaving the PDF looking a version behind.

Related: `scripts/docs/build-docs.mjs` · `scripts/docs/build-pdf.mjs`

`scripts/docs/extract-api.mjs`

177 lines

Walks the repository and parses every TypeScript file with the TypeScript compiler API, recording exported classes, functions, interfaces, types, constants and re-exports along with their JSDoc summaries and public member signatures. The result is written to `docs/generated/api.json`.

RUN IT

Shell

```
npm run docs:api # writes docs/generated/api.json
```

WHEN YOU NEED TO CHANGE IT

- You want an additional detail in the reference (decorators, parameter docs, private members).
- A new file type should be inventoried.

HOW TO CHANGE IT

- 1 Extend `memberSignature` or the statement loop in the extractor. Everything it records automatically appears in the generated pages — no page template needs editing.

WHY THE CHANGE BELONGS HERE

Hand-written API lists rot within one sprint. Reading the AST means the reference is always the code that actually shipped, and a forgotten method cannot silently go undocumented.

Related: `scripts/docs/build-docs.mjs`

`scripts/docs/highlight.mjs`

410 lines · 3 exports

Colours every code snippet at build time. It detects the language of a snippet (TypeScript, shell, YAML, JSON, env file or Dockerfile), tokenises it with a single-pass lexer, and emits `<span class="tok-*">` markup that the stylesheets theme.

NOTE Highlighting happens during the build, not in the browser. A runtime library could not colour the PDF at all, and would add a CDN dependency to pages that otherwise work offline.

TOKEN CLASSES EMITTED

TypeScript

tok-c	comment	tok-f	function
tok-k	keyword	tok-t	type
tok-s	string	tok-p	property
tok-n	number	tok-o	operator
tok-r	regex	tok-v	variable
tok-a	flag		

WHEN YOU NEED TO CHANGE IT

- A snippet is highlighted as the wrong language.
- You add snippets in a language the lexer does not know.
- A keyword or token type is coloured incorrectly.

HOW TO CHANGE IT

- 1 Language detection lives in `detectLanguage`. The subtle case it guards is that a TypeScript object property (`maxUploadMb: raw.MAX_UPLOAD_MB,`) looks exactly like a YAML key — the `LOOKS_LIKE_CODE` pattern is what separates them.

```
const LOOKS_LIKE_CODE =  
  /=>|\b(?:const|let|var|function|await|async|import|export|class|interface|extends|retu
```

- 2 To add a language, write a lexer built from one master pattern with named groups and register it in the `LEXERS` map. A single pass is what prevents the double-escaping bugs that come from highlighting already-escaped HTML.

```
const LEXERS = {  
  ts: highlightTs,  
  json: highlightJson,  
  sql: highlightSql, // new  
};
```

- 3 To force a language for one snippet, set `lang` on the content block instead of adjusting detection for everyone.

```
{ type: 'code', lang: 'shell', caption: 'Run it', text: 'npm run docs' }
```

- 4 Colours are not defined here. Add or change them in the `--sx-*` custom properties in `scripts/docs/docs.css` (site and print) and `scripts/docs/artifact.css` (hosted manual).

WHY THE CHANGE BELONGS HERE

Emitting classes rather than inline colours is what lets one highlighter serve three outputs with three different palettes — warm on the site, ink-safe in the PDF, cool in the hosted manual — and lets the hosted page recolour itself when the reader switches theme.

WATCH OUT FOR

- The lexer escapes HTML as it goes; never pass it text that is already escaped, or entities will be double-encoded.
- Template literals are highlighted recursively so `${...}` interpolations read as code rather than as string content.

Related: `scripts/docs/render.mjs` · `scripts/docs/docs.css` · `scripts/docs/artifact.css`

API SURFACE (3 EXPORTS)

FUNCTION `detectLanguage(code)`

CONST `LANGUAGE_LABEL`

FUNCTION `highlight(code, lang)`

Highlights a snippet. Pass `lang` to override the detected language. Returns HTML-escaped markup ready to place inside `<pre><code>`.

`scripts/docs/render.mjs`

197 lines · 6 exports

Turns the structured content blocks (paragraphs, lists, tables, code, callouts, step lists, trees) and the extracted API data into HTML. Shared by the multi-page site, the print sheet and the published artifact so all three cannot disagree.

WHEN YOU NEED TO CHANGE IT

- You need a new block type — a diagram, a definition list, a tabbed example.

HOW TO CHANGE IT

- 1 Add a case to `renderBlock` and a matching style rule in the stylesheet inside `build-docs.mjs`.

```
case 'diagram':  
  return `${block.svg}<figcaption>${inline(block.caption)}</figc
```

WHY THE CHANGE BELONGS HERE

One renderer means adding a block type improves every output format at once.

Related: `scripts/docs/build-docs.mjs`

API SURFACE (6 EXPORTS)

FUNCTION `esc(value = '')`

FUNCTION `inline(text = '')`

Inline markup: `code`, **bold**, text.

FUNCTION `renderBlocks(blocks = [])`

FUNCTION `slug(text = '')`

FUNCTION `fileAnchor(filePath)`

FUNCTION `renderFileEntry(filePath, doc, apiEntry)`

Renders one documented file: purpose, change guidance and API.

Reference: config, core & types

The foundation every other layer is built on

Configuration (src/config)

`src/config/env.config.ts`

195 lines · 3 exports

The only file in the framework permitted to read `process.env`. It loads `.env`, then `.env.` `<TEST_ENV>` on top, validates everything against a zod schema, and exports one frozen `config` object plus `getUser(role)` for credentials.

WHAT CONSUMERS SEE

TypeScript

```
import { config, getUser } from '@config/index';

config.baseURL;           // resolved URL for the selected environment
config.timeouts.action;   // validated number, never a string
config.artifacts.trace;   // 'on' | 'off' | 'retain-on-failure' | 'on-first-retry'
config.paths.downloads;   // absolute path, directories created by global setup

const admin = getUser('admin'); // throws a named error if the secret is missing
```

- `booleanish(fallback)` and `numeric(fallback)` pre-process raw strings into real booleans and numbers, falling back to a documented default instead of throwing on an empty variable.
- Real process variables always beat `.env` files, so CI secrets are never overwritten by a stray local file.
- `HEADLESS` is forced to true when `CI` is set — a headed browser on a CI runner hangs.
- Validation failure raises one error listing every offending variable, before a single browser starts.

WHEN YOU NEED TO CHANGE IT

- You introduce a new environment variable.
- You add a user role that needs credentials.
- A default value proves wrong for most runs.
- You need a new derived path under `config.paths`.

HOW TO CHANGE IT

- 1 Add the variable to the zod schema with a sensible fallback.

```
const schema = z.object({
  // ...
  FEATURE_FLAGS: z.string().optional(),
  MAX_UPLOAD_MB: numeric(25),
});
```

- 2 Expose it on the exported `config` object with the shape callers should see, converting or splitting it here rather than at every call site.

```
export const config = {
  // ...
  featureFlags: (raw.FEATURE_FLAGS ?? '').split(',').map((f) => f.trim()).filter(Boolean),
  maxUploadMb: raw.MAX_UPLOAD_MB,
} as const;
```

- 3 Document it in `.env.example` in the same commit, and add it to the CI workflow if it must be set there.
- 4 For a new credentialed role, add the variables to the schema and a row to the `CREDENTIALS` map; `getUser` then validates it automatically.

```
const CREDENTIALS: Record<UserRole, { username?: string; password?: string }> = {
  // ...
  auditor: { username: raw.AUDITOR_USER, password: raw.AUDITOR_PASSWORD },
};
```

WHY THE CHANGE BELONGS HERE

Centralising environment access converts a whole class of silent misconfiguration — a suite pointed at the wrong host, a timeout parsed as `NaN`, a login attempted with `undefined` — into one loud, immediate, specific error. Reading `process.env` anywhere else reintroduces exactly that risk.

WATCH OUT FOR

- The role name must also exist in the `UserRole` union in `src/types/index.ts`, or TypeScript will reject the `CREDENTIALS` entry.
- This module executes at import time. Keep it free of side effects beyond reading files and creating no I/O.

Related: `src/config/environments.ts` · `src/config/timeouts.ts` · `.env.example` · `src/types/index.ts`

API SURFACE (3 EXPORTS)

CONST `config`

The single, validated, strongly-typed view of runtime configuration. Nothing in the framework should read `process.env` directly.

FUNCTION `getUser(role: UserRole): TestUser`

Resolves credentials for a role, failing loudly at the point of use rather than silently logging in as `undefined`.

TYPE `AppConfig`

```
typeof config
```

`src/config/environments.ts`

94 lines · 1 exports

Static, non-secret settings for each environment — base URL, API URL, default retries and workers, and whether to ignore HTTPS errors. Safe to commit and the single source of truth for where an environment lives.

WHEN YOU NEED TO CHANGE IT

- A new environment appears (a preview tier, a regional deployment).
- An existing environment moves to a different URL.
- One environment needs different retry or parallelism defaults — a shared staging box may need fewer workers than a dedicated QA box.

HOW TO CHANGE IT

- 1 Add the name to the `EnvironmentName` union in `src/types/index.ts` first; TypeScript will then require the matching entry here.

```
export type EnvironmentName = 'local' | 'dev' | 'qa' | 'staging' | 'preprod' | 'prod';
```

- 2 Add the entry with all five fields.

```
preprod: {
  name: 'preprod',
  baseUrl: 'https://preprod.example.com',
  apiBaseUrl: 'https://preprod.example.com/api',
  retries: 2,
  workers: 4,
  ignoreHTTPSErrors: false,
},
```

- 3 Extend the `TEST_ENV` enum in the zod schema in `env.config.ts`, and add the value to the CI workflow's environment choice list.

WHY THE CHANGE BELONGS HERE

URLs are configuration, not secrets. Keeping them in version control means a reviewer can see exactly which host a change targets, and switching environments is one variable rather than a hunt through scripts.

WATCH OUT FOR

- Never add credentials here. This file is committed; secrets come from the process environment.

Related: `src/config/env.config.ts` · `src/types/index.ts`

API SURFACE (1 EXPORT)

FUNCTION `getEnvironment(name: EnvironmentName): EnvironmentConfig`

`src/config/timeouts.ts`

31 lines · 1 exports

Every wait in the framework references one of these named budgets — `INSTANT`, `SHORT`, `MEDIUM`, `LONG`, `EXTRA_LONG`, `TEST`, `HOOK`, `EXPECT`, `POLL_INTERVAL`, `DEBOUNCE`, `ANIMATION` — so tuning flakiness is a single-file change instead of a repo-wide hunt for magic numbers.

BUDGET	DEFAULT	USE FOR
<code>INSTANT</code>	1s	A class toggle, a fade, a ripple.
<code>SHORT</code>	5s	Tooltips, dropdown opening, toast appearance.
<code>MEDIUM</code>	15s	Default for any element interaction.
<code>LONG</code>	30s	Navigation, route change, heavy client render.
<code>EXTRA_LONG</code>	60s	Upload, download, report generation.
<code>DEBOUNCE</code>	500ms	Settle time after typing into a search field.
<code>ANIMATION</code>	300ms	Settle time before a screenshot comparison.

WHEN YOU NEED TO CHANGE IT

- A category of interaction is genuinely slower in your application.
- You need a new named category (for example `REPORT_GENERATION`).

HOW TO CHANGE IT

- 1 Add or adjust the named budget, then use the name — never the number — at the call site.

```
export const TIMEOUTS = {  
  // ...  
  REPORT_GENERATION: 180_000,  
} as const;  
  
await this.locator.waitFor({ state: 'visible', timeout: TIMEOUTS.REPORT_GENERATION });
```

WHY THE CHANGE BELONGS HERE

A raw `30000` scattered through a suite tells the next reader nothing about intent. A name says what class of wait it is, which makes it safe to tune every wait of that class at once when the application changes.

WATCH OUT FOR

- Raising a timeout hides slowness rather than fixing it. Confirm the wait is legitimately long before increasing a budget.

Related: `src/config/env.config.ts` · `src/core/base.component.ts`

API SURFACE (1 EXPORT)

`CONST` **TIMEOUTS**

Centralised timeout budget (milliseconds). Every wait in the framework must reference one of these named budgets so that tuning flakiness is a single-file change rather than a repo-wide grep.

Core (src/core)

`src/core/base.component.ts`

521 lines · 1 exports

The most important file in the framework. Every component in `src/components/` extends this class, so the behaviour that must be identical everywhere — step reporting, structured logging, scroll and stability handling, state inspection, named assertions and failure diagnostics — is written once here and inherited rather than copy-pasted.

- **Reported steps.** `this.step(title, fn)` wraps every public action in `test.step`, so a trace reads `Table "Results": sort by "Amount" desc` instead of an anonymous click.

- **Contextual failures.** A failure is re-thrown naming the component, its selector and the original cause, attached as `cause`.
- **Pre-action hygiene.** `prepare()` waits for visibility, scrolls into view, and optionally waits for the bounding box to stop moving — before every interaction.
- **State inspection.** `getState()` returns existence, visibility, enablement, editability, checked state, focus, text, value, box and every attribute in one call.
- **Named assertions.** `expectVisible`, `expectText`, `expectCount` and friends wrap web-first assertions with messages that identify the component.
- **Narrowing.** `nth(n)` (1-based, to read naturally), `first()` and `filterByText()` return a new instance of the same subclass.

THE PATTERN EVERY SUBCLASS FOLLOWS

TypeScript

```
export class Button extends BaseComponent {
  protected override get componentType(): string {
    return 'Button'; // shown in logs and step titles
  }

  async clickWhenReady(options: ClickOptions = {}): Promise<void> {
    await this.step('click when enabled', async () => { // reported step
      await this.waitForEnabled();
      await this.prepare(); // scroll + stability
      await this.locator.click({ timeout: this.timeout, ...options });
    });
  }
}
```

WHEN YOU NEED TO CHANGE IT

- A behaviour should apply to every element type — a new universal wait, a new diagnostic, a logging change.
- You want to change what happens before every interaction.
- You need a new shared assertion.

HOW TO CHANGE IT

- 1 Add the method here only if it is meaningful for every element. If it applies to one family, put it on that component instead.

```
/** Waits until the element stops reporting an ARIA busy state. */
async waitForNotBusy(timeout = this.timeout): Promise<void> {
  await this.step('wait for aria-busy to clear', async () => {
    await expect(this.locator).not.toHaveAttribute('aria-busy', 'true', { timeout });
  });
}
```

- 2 Wrap it in `this.step()` so it appears in traces, and use `this.timeout` so per-component overrides are honoured.
- 3 To change pre-action behaviour for everything, edit `prepare()`. Measure the cost first — it runs before every single interaction in the suite.

WHY THE CHANGE BELONGS HERE

This is the leverage point of the whole framework: one method here becomes available to all 37 component types with no duplication, and one improvement to error messages improves every failure in the suite.

WATCH OUT FOR

- `getValue()` is deliberately named `getInputValue()` on the base class so components whose value is numeric (`Slider`, `ProgressBar`) can define a correctly-typed `getValue()` of their own.
- Anything added here runs everywhere; an expensive check will slow the entire suite.
- `nth()` is 1-based here while Playwright's own `nth()` is 0-based — the class converts.

Related: `src/core/locator.factory.ts` · `src/core/base.page.ts` · `src/components/component.factory.ts`

API SURFACE (1 EXPORT)

CLASS **BaseComponent**

The root of the component model. Every UI element type in `src/components` extends this class, so the behaviour that must be identical everywhere — logging, step reporting, scrolling, stability waits, state inspection, diagnostics on failure — is written once here and inherited rather than copy-pasted. Design rules: - Wrap, never replace, Playwright's auto-waiting. No arbitrary sleeps. - Every public action reports itself as a `test.step` so traces read like a plain-English script. - Failures carry the component name and selector, not just "locator not found".

`locator: Locator`

`name: string`

`get page: Page`

`nth(position: number): this`

Narrows to the nth match (1-based) as a new component of the same type.

`first(): this`

`filterByText(text: string | RegExp): this`

Narrows this component to matches containing the given text.

`describe(): string`

`waitForStable(timeout = TIMEOUTS.SHORT): Promise<void>`

Waits until the bounding box stops moving — for animated or lazy content.

```
click(options: ClickOptions = {}):
```

```
Promise<void>
```

```
doubleClick(options: ClickOptions = {}):
```

```
Promise<void>
```

```
rightClick(options: ClickOptions = {}):
```

```
Promise<void>
```

```
forceClick(): Promise<void>
```

Dispatches a DOM click that bypasses actionability checks. Escape hatch for overlays you cannot dismiss — prefer `click()`.

```
hover(): Promise<void>
```

```
focus(): Promise<void>
```

```
blur(): Promise<void>
```

```
scrollIntoView(): Promise<void>
```

```
pressKey(key: string): Promise<void>
```

```
dragTo(target: BaseComponent | Locator):
```

```
Promise<void>
```

```
isVisible(): Promise<boolean>
```

```
isHidden(): Promise<boolean>
```

```
isEnabled(): Promise<boolean>
```

```
isDisabled(): Promise<boolean>
```

```
isEditable(): Promise<boolean>
```

```
isChecked(): Promise<boolean>
```

```
isFocused(): Promise<boolean>
```

```
exists(): Promise<boolean>
```

```
count(): Promise<number>
```

```
getText(): Promise<string>
```

```
getTextContent(): Promise<string>
```

```
getAllTexts(): Promise<string[]>
```

```
getInputValue(): Promise<string>
```

Raw value of a form control. Named `getInputValue` rather than `getValue` so components whose value is not a string (Slider, ProgressBar) can define a correctly-typed `getValue()` without fighting this signature.

```
getAttribute(attribute: string): Promise<string
```

```
| null>
```

```
hasAttribute(attribute: string):
```

```
Promise<boolean>
```

```
getCssValue(property: string): Promise<string>
```

```
hasClass(className: string): Promise<boolean>
```

```
getBoundingBox(): Promise<BoundingBox | null>
```

```
getAccessibleName(): Promise<string>
```

Accessible name as computed by the browser — what a screen reader announces.

```
getAriaRole(): Promise<string | null>
```

```
getState(): Promise<ElementState>
```

One-shot snapshot of everything observable — invaluable in failure output.

```
getAllAttributes(): Promise<Record<string, string>>
```

```
waitForVisible(timeout = this.timeout): Promise<void>
```

```
waitForHidden(timeout = this.timeout): Promise<void>
```

```
waitForAttached(timeout = this.timeout): Promise<void>
```

```
waitForDetached(timeout = this.timeout): Promise<void>
```

```
waitForEnabled(timeout = this.timeout): Promise<void>
```

```
waitForText(text: string | RegExp, timeout = this.timeout): Promise<void>
```

```
expectVisible(timeout = this.timeout): Promise<void>
```

```
expectHidden(timeout = this.timeout): Promise<void>
```

```
expectEnabled(timeout = this.timeout): Promise<void>
```

```
expectDisabled(timeout = this.timeout): Promise<void>
```

```
expectText(expected: string | RegExp, timeout = this.timeout): Promise<void>
```

```
expectContainsText(expected: string | RegExp, timeout = this.timeout): Promise<void>
```

```
expectValue(expected: string | RegExp, timeout = this.timeout): Promise<void>
```

```
expectAttribute(attribute: string, expected: string | RegExp): Promise<void>
```

```
expectCount(expected: number): Promise<void>
```

```
expectFocused(): Promise<void>
```

```
highlight(durationMs = 1000): Promise<void>
```

Draws a temporary outline — useful when debugging headed runs.

```
screenshot(name?: string): Promise<Buffer>
```

```
innerHTML(): Promise<string>
```

src/core/base.page.ts

185 lines · 1 exports

Base class for page objects. It owns navigation, load verification, page-level waits and page-level assertions, and requires every subclass to declare two things: the `path` it lives at and a `readyIndicator` that proves it finished rendering.

THE TWO REQUIRED DECLARATIONS

TypeScript

```
export class CheckoutPage extends BasePage {  
  protected readonly path = '/checkout';  
  protected readonly readyIndicator: SelectorLike = '[data-testid="checkout-root"]';  
}
```

- `goto()` navigates with query parameters and waits until the page is genuinely usable.
- `waitUntilLoaded()` blocks on the ready indicator; override it to add further checks.
- `waitForIdle()` waits for network quiet and an optional spinner to disappear.
- `actionWithResponse()` runs an action and returns the API response it triggered.
- `isCurrentPage()` supports conditional flows without throwing.
- `checkAccessibility()` runs an axe scan on the page.

WHEN YOU NEED TO CHANGE IT

- Every page in your application shares a behaviour — a cookie banner to dismiss, a global spinner, a tenant switch.
- You need a new page-level assertion or wait.

HOW TO CHANGE IT

- 1 Add the shared behaviour here so no page object repeats it.

```
/** Dismisses the consent banner if the application decided to show it. */  
protected async dismissConsentBanner(): Promise<void> {  
  const accept = this.page.getByRole('button', { name: /accept all/i });  
  if (await accept.isVisible().catch(() => false)) await accept.click();  
}
```


- 2 To make it automatic, call it from an overridden `waitUntilLoaded()` — and remember that subclasses overriding that method must call `super`.

```
override async waitUntilLoaded(timeout?: number): Promise<void> {
  await super.waitUntilLoaded(timeout);
  await this.dismissConsentBanner();
}
```

WHY THE CHANGE BELONGS HERE

Requiring a ready indicator is what makes navigation deterministic: without it, page objects fall back to arbitrary sleeps, which is the most common source of flakiness in UI suites.

WATCH OUT FOR

- Page objects must not contain business assertions. Verifying that the page loaded is in scope; verifying that the order total is \$80 is the test's job.

Related: `src/core/base.component.ts`

API SURFACE (1 EXPORT)

CLASS **BasePage**

Base for every page object. A page object owns: the URL it lives at, the components it exposes, and the high-level business actions a test would name out loud. It must NOT contain assertions about business outcomes (those belong in tests) beyond the self-verification of `waitUntilLoaded`.

`network: NetworkHelper`

`get url: string`

`goto(options: { query?: Record<string, string>; waitUntilLoaded?: boolean } = {}): Promise<void>` Navigates to this page and waits until it is genuinely usable.

`waitUntilLoaded(timeout = config.timeouts.navigation): Promise<void>` Blocks until the ready indicator is visible. Override to add more checks.

`reload(): Promise<void>`

`goBack(): Promise<void>`

`goForward(): Promise<void>`

`isCurrentPage(): Promise<boolean>` True when the browser is on this page — for conditional flows.

`title(): Promise<string>`

`currentUrl(): string`

`waitForIdle(options: { spinnerSelector?: string; timeout?: number } = {}): Promise<void>` Waits for the SPA to go quiet: no in-flight requests and no spinners.

```
waitForUrl(pattern: string | RegExp, timeout =
config.timeouts.navigation): Promise<void>
```

```
actionWithResponse(action: () => Promise<void>,
urlPattern: string | RegExp, timeout =
config.timeouts.navigation): Promise<Response>
```

Runs an action and returns the API response it triggered.

```
expectLoaded(): Promise<void>
```

```
expectUrl(pattern: string | RegExp):
Promise<void>
```

```
expectTitle(pattern: string | RegExp):
Promise<void>
```

```
checkAccessibility(options: A11yScanOptions =
{}): Promise<void>
```

Scans this page for WCAG violations.

```
captureScreenshot(name =
`${this.constructor.name}.png`): Promise<void>
```

Takes a full-page screenshot and attaches it to the report.

src/core/locator.factory.ts

38 lines · 2 exports

Resolves any `SelectorLike` — a CSS or XPath string, an existing `Locator`, or a function that derives one from a scope — into a real `Locator`, applying the `iframe` and `shadow-DOM` options a component declares. Also exports `by`, the catalogue of locator strategies in order of resilience.

WHY THIS MAKES IFRAMES ORDINARY

TypeScript

```
const frame = new Frame(page, '#payment-iframe');
const cardNumber = new TextInput(frame.locator, '#card-number'); // identical API
```

PREFERENCE	STRATEGY	WHEN
1	<code>by.testId</code>	Default. Stable against copy and styling changes.
2	<code>by.role</code>	Accessible and expresses intent.
3	<code>by.label</code> / <code>by.placeholder</code>	Form controls with visible labels.
4	<code>by.text</code>	Buttons and links whose text is the contract.
5	<code>by.css</code>	Legacy markup with nothing better.
6	<code>by.xpath</code>	Last resort — structural and brittle.

WHEN YOU NEED TO CHANGE IT

- Your application needs a locator strategy that is not covered (a framework-specific attribute, a data-cy convention).
- A new scoping mechanism appears — nested shadow roots, a custom portal container.

HOW TO CHANGE IT

- 1 Add the strategy to the `by` object with a short comment about when it is appropriate.

```
by.automationId = (scope: Page | Locator, id: string): Locator =>
  scope.locator(`[data-automation-id="${id}"]`);
```

- 2 If the whole application uses a different test-id attribute, change `testIdAttribute` in `playwright.config.ts` instead — then `getByTestId` and `by.testId` both follow it.

WHY THE CHANGE BELONGS HERE

Centralising resolution is what allows every component to accept three different selector forms without each one re-implementing the logic, and what keeps iframe and shadow-DOM handling out of the component code entirely.

Related: `src/core/base.component.ts` · `src/components/media/frame.ts` · `playwright.config.ts`

API SURFACE (2 EXPORTS)

TYPE Scope

`Page` | `Frame` | `Locator` | `FrameLocator`

FUNCTION

`resolveLocator`(`scope`: `Scope`, `selector`: `SelectorLike`, `options`: `Pick<ComponentOptions, 'frameSelector' | 'shadowHost'> = {}`): `Locator`

Resolves any `SelectorLike` into a `Locator` against a scope, applying the iframe and shadow-DOM options that components declare. Centralising this is what lets every component accept a string, a `Locator` or a builder function without each one re-implementing resolution.

Types (src/types)

`src/types/index.ts`

145 lines · 17 exports

Every cross-cutting type in one place: environment names and shape, user roles, component options, `SelectorLike`, `ElementState`, click and fill options, table and sorting types, network

mocking and capture shapes, accessibility and visual options, and the retry policy.

- `SelectorLike` is what lets components accept a string, a Locator or a builder function.
- `ComponentOptions` carries `name`, `timeout`, `autoScroll`, `waitForStable`, `frameSelector` and `shadowHost` — every component extends it for its own selectors.
- `ElementState` is the diagnostic snapshot returned by `getState()`.
- `UserRole` and `EnvironmentName` are unions, so adding a value makes TypeScript point at every place that must handle it.

WHEN YOU NEED TO CHANGE IT

- You add a role, an environment, or a concept shared by more than one module.
- A component option becomes generally useful.

HOW TO CHANGE IT

- 1 Add the union member or interface field here first, then follow the compiler errors — they are a complete to-do list of everywhere that must be updated.

```
export type UserRole = 'standard' | 'admin' | 'readonly' | 'guest' | 'auditor';
```

- 2 Keep types that belong to exactly one component in that component's file; only genuinely shared shapes belong here.

WHY THE CHANGE BELONGS HERE

A union defined once turns "add a role" from a search-and-hope exercise into a compiler-guided checklist. That property disappears the moment the same union is redeclared elsewhere.

Related: `src/config/environments.ts` · `src/config/env.config.ts` · `src/core/base.component.ts`

API SURFACE (17 EXPORTS)

TYPE `EnvironmentName`

```
'demo' | 'local' | 'dev' | 'qa' | 'staging' | 'prod'
```

INTERFACE `EnvironmentConfig`

```
name: EnvironmentName;
baseUrl: string;
apiBaseUrl: string;
retries: number;
workers?: number | undefined;
ignoreHTTPSErrors: boolean;
```

TYPE LogLevel

```
'error' | 'warn' | 'info' | 'debug' | 'trace'
```

TYPE UserRole

```
'standard' | 'admin' | 'readonly' | 'guest'
```

INTERFACE TestUser

```
role: UserRole;
username: string;
password: string;
storageStatePath?: string;
displayName?: string;
```

TYPE SelectorLike

Anything a component can be built from: a raw CSS/XPath selector, an already-resolved Locator, or a factory that derives one from a scope.

```
string | Locator | ((scope: Page | Locator) => Locator)
```

INTERFACE ComponentOptions

```
name?: string;
timeout?: number;
autoScroll?: boolean;
waitForStable?: boolean;
frameSelector?: string;
shadowHost?: string;
```

INTERFACE ElementState

The observable state of any UI element, used by assertions and diagnostics.

```
exists: boolean;
visible: boolean;
enabled: boolean;
editable: boolean;
checked: boolean | null;
focused: boolean;
text: string;
value: string | null;
boundingBox: BoundingBox | null;
attributes: Record<string, string>;
```

INTERFACE BoundingBox

```
x: number;
y: number;
width: number;
height: number;
```

TYPE ClickOptions

```
{ button?: 'left' | 'right' | 'middle'; clickCount?: number; delay?: number; force?: boolean;
```

TYPE FillOptions

```
{ clearFirst?: boolean; pressSequentially?: boolean; delay?: number; force?: boolean; timeo
```

TYPE TableRow

```
Record<string, string>
```

TYPE SortDirection

```
'asc' | 'desc' | 'none'
```

INTERFACE MockResponse

```
status?: number;
headers?: Record<string, string>;
body?: T;
contentType?: string;
delayMs?: number;
```

INTERFACE CapturedRequest

```
url: string;  
method: string;  
postData: string | null;  
headers: Record<string, string>;  
timestamp: number;
```

TYPE WcagTag

```
'wcag2a' | 'wcag2aa' | 'wcag21a' | 'wcag21aa' | 'wcag22aa' | 'best-practice'
```

INTERFACE AllyScanOptions

```
include?: string[];  
exclude?: string[];  
tags?: WcagTag[];  
disableRules?: string[];  
failOnViolation?: boolean;  
attachReport?: boolean;
```

Reference: component library

All 37 element types, grouped by family

Component library entry points

`src/components/component.factory.ts`

174 lines · 2 exports

An ergonomic factory bound to a scope — a page, an iframe or a container locator. Page objects call `ui(page).table(...)` instead of constructors, and `within()` re-scopes to a container so selectors stay short and strict-mode collisions are avoided.

SCOPING

TypeScript

```
const factory = ui(page); // whole page
const dialog = ui(page.locator('#confirm-dialog')); // inside one container
const payment = ui(new Frame(page, '#pay').locator); // inside an iframe

readonly submit = factory.button('[data-testid="submit"]', { name: 'Submit order' });
```

WHEN YOU NEED TO CHANGE IT

- You add a component class and want it available through the factory.

HOW TO CHANGE IT

- 1 Import the class, then add a method named after how a tester would say it. Keep the options type specific so editor autocomplete lists the component's own selectors.

```
timeline = (selector: SelectorLike, options?: TimelineOptions): Timeline =>
  new Timeline(this.scope, selector, options);
```

WHY THE CHANGE BELONGS HERE

Constructors need a scope argument every time; the factory carries it. That is what makes container-scoped and frame-scoped components a one-line change rather than a refactor.

WATCH OUT FOR

- `frame()` returns a `Frame`, which is a scope rather than a component — pass `frame.locator` to other factory calls.

Related: `src/core/locator.factory.ts`

API SURFACE (2 EXPORTS)

CLASS **UiFactory**

Ergonomic factory bound to a scope (a page, a frame or a container locator). Page objects use this instead of calling constructors directly: `private readonly ui = ui(this.page);` `readonly submit = this.ui.button('#submit', { name: 'Submit order' });` Scoping to a container keeps selectors short and prevents cross-widget strict-mode collisions: `const dialog = ui(page.locator('#confirm-dialog'));`

`within(selector: string): UiFactory`

Re-scopes the factory to a container within the current scope.

`button``input``textarea``checkbox``radioGroup``select``dropdown``multiSelect``autocomplete``fileUpload``datePicker``toggle``slider``rating``colorPicker``form``table``dataGrid``list``tree``pagination``card``link``tabs``accordion``menu``breadcrumb`

`stepper``modal``alert``tooltip``progressBar``loader``image``media``canvas``carousel``frame`

Frames are scopes, not components — pass `frame.locator` to other builders.

`draggable``dropZone``shadowHost``richText``infiniteScroll``chart`

FUNCTION `ui(scope: Scope): UiFactory`

Creates a UI factory bound to a page, frame or container locator.

Form controls

`src/components/form/autocomplete.ts`

129 lines · 2 exports

Typeahead fields that fetch suggestions asynchronously. It models the debounce and loading phases explicitly — type, wait out the debounce, wait for the loading indicator to clear, then wait for the list — which is what turns the flakiest widget in most suites into a reliable one.

WHEN YOU NEED TO CHANGE IT

- Your debounce is longer than the default 500 ms.
- The suggestion list, loading indicator or empty state uses different markup.

HOW TO CHANGE IT

1 Configure the timings and selectors at construction.

```
ui.autocomplete('#city', {
  debounceMs: 800,
  suggestionListSelector: '[data-testid="city-results"]',
  loadingSelector: '[data-testid="city-loading"]',
  noResultsSelector: '[data-testid="city-empty"]',
});
```

2 For a deterministic test, stub the suggestions endpoint with the `network` fixture instead of increasing timeouts.

```
await network.mock('*/api/cities*', { body: [{ id: 1, name: 'Lagos' }] });
```

WHY THE CHANGE BELONGS HERE

Waiting on the widget's own signals rather than a fixed sleep is what makes the test both fast and stable; stubbing the endpoint removes the remaining dependency on a third-party search service.

Related: `src/utils/network.utils.ts`

API SURFACE (2 EXPORTS)

INTERFACE AutocompleteOptions

```
suggestionListSelector?: string;
suggestionItemSelector?: string;
loadingSelector?: string;
noResultsSelector?: string;
debounceMs?: number;
```

CLASS Autocomplete extends BaseComponent

Typeahead / autocomplete: type a fragment, wait for asynchronous suggestions, pick one. Explicitly models the debounce and loading phases that make these widgets the single most flaky control in most suites.

`get suggestionList: Locator`

`get suggestions: Locator`

`search(term: string): Promise<void>`

Types the term and waits for suggestions to finish loading.

`waitForSuggestions(timeout = this.timeout): Promise<void>`

`selectSuggestion(text: string | RegExp): Promise<void>`

`selectFirstSuggestion(): Promise<string>`

```
searchAndSelect(term: string, suggestion?:
string): Promise<void>
```

```
getSuggestions(): Promise<string[]>
```

```
suggestionCount(): Promise<number>
```

```
hasNoResults(): Promise<boolean>
```

```
selectWithKeyboard(position: number):
Promise<void>
```

Keyboard flow: ArrowDown to the nth suggestion, then Enter.

```
getHighlightedSuggestion(): Promise<string>
```

The suggestion currently highlighted via aria-activedescendant.

src/components/form/button.ts

64 lines · 1 exports

Any clickable control: a real `<button>`, an `<input type="submit">`, an anchor with `role="button"`, or a div a framework decided to treat as a button. Adds the click variants that matter in practice — clicking only once enabled, clicking and waiting for navigation, and clicking then waiting for the busy state to clear.

WHEN YOU NEED TO CHANGE IT

- Your application signals loading differently from the defaults (`aria-busy`, `.spinner`, `.loading`).
- A new click-and-wait pattern recurs across tests.

HOW TO CHANGE IT

- 1 Pass your own busy selector at the call site when it is page-specific.

```
await this.submitButton.clickAndWaitForCompletion('[data-testid="save-spinner"]');
```

- 2 If the same selector is used everywhere, change the default in the method signature so no call site has to repeat it.

WHY THE CHANGE BELONGS HERE

Click-then-wait belongs on the button rather than in every test, because the waiting rule is a property of the control, not of the scenario.

Related: `src/components/feedback/progress.ts`

API SURFACE (1 EXPORT)

CLASS **Button** extends `BaseComponent`

Any clickable control: `<button>`, `<input type="submit">`, `<a role="button">` or a div that a framework pretends is a button.

`clickWhenReady(options: ClickOptions = {}): Promise<void>` Clicks only once the control is genuinely interactive.

`clickAndNavigate(urlPattern?: string | RegExp): Promise<void>` Clicks and waits for the resulting navigation to settle.

`clickAndWaitForCompletion(busySelector = '[aria-busy="true"]', .spinner, .loading): Promise<void>` Submit buttons commonly show a spinner and re-enable when done.

`isLoading(): Promise<boolean>`

`getLabel(): Promise<string>`

src/components/form/checkbox.ts

82 lines · 1 exports

Native checkboxes and ARIA checkboxes, including the tri-state `indeterminate` case used by "select all" controls. `check()` and `unchecked()` are idempotent and fall back to a click when Playwright's strict checkable definition does not apply.

WHEN YOU NEED TO CHANGE IT

- Your checkbox is a styled div that ignores clicks on the input itself, or the label carries the click target.

HOW TO CHANGE IT

- 1 Point the component at the element that actually receives the click, and let the component read state from the input.

```
ui.checkbox('label[for="terms"]', { name: 'Accept terms' });
```

WHY THE CHANGE BELONGS HERE

Reading state from the native property or `aria-checked` while clicking the visible control is what makes one class work for both native and design-system checkboxes.

Related: `src/components/form/toggle.ts` · `src/components/form/radio-group.ts`

API SURFACE (1 EXPORT)**CLASS** **Checkbox** extends `BaseComponent`

Checkbox — native `<input type="checkbox">` or an ARIA checkbox. Handles the tri-state (`indeterminate`) case that most helpers forget.

`check(): Promise<void>`

`uncheck(): Promise<void>`

`toggle(): Promise<void>`

`set(checked: boolean): Promise<void>`

`isCheckedState(): Promise<boolean>`

Reads checked state from the native property or `aria-checked`.

`isIndeterminate(): Promise<boolean>`

Partially-selected parent checkbox, e.g. "select all" over a mixed list.

`getLabelText(): Promise<string>`

`expectChecked(): Promise<void>`

src/components/form/color-picker.ts

48 lines · 1 exports

Native `<input type="color">` and custom swatch pickers: setting a hex value with the events frameworks expect, reading the value, picking a named swatch, and reading the rendered colour.

WHEN YOU NEED TO CHANGE IT

- Your palette markup differs, or the picker is a canvas-based gradient.

HOW TO CHANGE IT

- 1 Pass a swatch selector to `pickSwatch()`; for canvas-based pickers use the `Canvas` component and click a coordinate.

WHY THE CHANGE BELONGS HERE

Setting a native colour input directly avoids driving an OS-level colour dialog, which no browser automation can control.

Related: `src/components/media/canvas.ts`

API SURFACE (1 EXPORT)

CLASS `ColorPicker` extends `BaseComponent`

`<input type="color">` and custom colour swatch pickers.

`setColor(hex: string): Promise<void>`

Sets a hex colour on a native input, firing the events frameworks listen for.

`getColor(): Promise<string>``pickSwatch(colorName: string, swatchSelector =
'swatch, [data-color]'): Promise<void>`

Picks a named swatch from a custom palette.

`getRenderedColor(): Promise<string>`Computed RGB of the rendered swatch —
what the user actually sees.

src/components/form/date-picker.ts

151 lines · 2 exports

Covers both interaction paths: typing into the input, and navigating the calendar popup month by month. Also supports ranges, reading the displayed month, and checking whether a date is disabled.

WHEN YOU NEED TO CHANGE IT

- Your calendar markup, month navigation controls or input format differ from the defaults.

HOW TO CHANGE IT

- 1 Configure the format and selectors once at construction.

```
ui.datePicker('#depart', {  
  inputFormat: 'DD/MM/YYYY',  
  calendarSelector: '.rdp',  
  daySelector: '.rdp-day:not(.rdp-day_disabled)',  
  nextMonthSelector: '.rdp-nav_button_next',  
});
```

- 2 Prefer `typeDate()` when the field accepts typing — it is faster and far less brittle than paging a calendar.

WHY THE CHANGE BELONGS HERE

Month navigation is bounded to 24 hops so a mis-parsed month fails with a clear message instead of looping forever.

WATCH OUT FOR

- `inputFormat` must match what the application expects, or the field will silently reject the typed value.

Related: `src/utils/date.utils.ts`

API SURFACE (2 EXPORTS)

INTERFACE **DatePickerOptions**

```

calendarSelector?: string;
daySelector?: string;
monthLabelSelector?: string;
nextMonthSelector?: string;
previousMonthSelector?: string;
inputFormat?: string;

```

CLASS **DatePicker** extends `BaseComponent`

Date picker, covering both interaction paths: typing into the input, and navigating the calendar popup month by month.

```
get calendar: Locator
```

```
open(): Promise<void>
```

```
close(): Promise<void>
```

```
typeDate(date: Date): Promise<void>
```

Fastest and least brittle path: type the value straight into the field.

```
pickDate(date: Date): Promise<void>
```

Opens the calendar and clicks the day, paging months as needed.

```
navigateToMonth(target: Date): Promise<void>
```

Pages the calendar until the target month is shown (bounded to 24 hops).

```
getDisplayedMonth(): Promise<string>
```

```
getSelectedDate(): Promise<string>
```

```
clear(): Promise<void>
```

```
getDisabledDays(): Promise<string[]>
```

Days the picker refuses — min/max range and blackout dates.

```
isDateDisabled(date: Date): Promise<boolean>
```

```
pickRange(start: Date, end: Date):
Promise<void>
```

Selects a start/end pair in a range picker.

src/components/form/dropdown.ts

134 lines · 2 exports

Non-native dropdowns, comboboxes and listboxes from React-Select, MUI, Ant Design, Kendo and similar libraries. The component wraps the visible trigger; the panel is looked up on the page rather than under the trigger, because most libraries render it in a portal at body level.

ADAPTING IT TO A LIBRARY — CONFIGURATION, NOT A NEW CLASS

TypeScript


```
ui.dropdown('#country', {
  panelSelector: '.MuiAutocomplete-popover',
  optionSelector: '.MuiAutocomplete-option',
  searchSelector: '#country',
  valueSelector: '.MuiAutocomplete-input',
});
```

WHEN YOU NEED TO CHANGE IT

- You adopt a component library whose panel, option or search markup differs from the defaults.
- The dropdown needs a different open or close gesture.

HOW TO CHANGE IT

- 1 First try the options above — most libraries need nothing more.
- 2 If the interaction itself differs, subclass and override the specific method, keeping everything else inherited.

```
class HoverDropdown extends Dropdown {
  override async open(): Promise<void> {
    await this.step('open on hover', async () => {
      await this.locator.hover();
      await this.panel.waitFor({ state: 'visible' });
    });
  }
}
```

WHY THE CHANGE BELONGS HERE

Portalled panels are why a naive `trigger.locator(".option")` fails on most modern UI kits; looking the panel up on the page is the difference between a component that works with one library and one that works with all of them.

WATCH OUT FOR

- `isOpen()` prefers `aria-expanded` and falls back to panel visibility, so accessible markup gives the most reliable behaviour.

Related: `src/components/form/multi-select.ts` · `src/components/form/autocomplete.ts`

API SURFACE (2 EXPORTS)

INTERFACE `DropdownOptions`

```
panelSelector?: string;
optionSelector?: string;
valueSelector?: string;
searchSelector?: string;
```

CLASS **Dropdown** extends `BaseComponent`

Custom (non-native) dropdown / combobox / listbox — React-Select, MUI, Ant Design, Kendo and friends. The trigger locator is the visible control; the panel is looked up on the page (not under the trigger) because most libraries portal it to `<body>`.

`get panel: Locator`

`get optionsLocator: Locator`

`open(): Promise<void>`

`close(): Promise<void>`

`isOpen(): Promise<boolean>`

`selectOption(label: string | RegExp): Promise<void>` Opens, optionally filters, then picks the option with this exact text.

`selectByIndex(index: number): Promise<void>`

`search(term: string): Promise<void>` Types into the dropdown's own search field and waits for the list to settle.

`getSelectedText(): Promise<string>`

`getOptions(): Promise<string[]>`

`optionCount(): Promise<number>`

`hasOption(label: string): Promise<boolean>`

`selectWithKeyboard(steps: number): Promise<void>` Keyboard selection — the path real keyboard users take.

src/components/form/file-upload.ts

154 lines · 2 exports

Uploads in all three flavours: a plain `<input type="file">`, a hidden input behind a styled button (via the file chooser event), and a drag-and-drop zone (via a synthetic `DataTransfer` drop). Also covers in-memory buffers, upload progress, error messages, accepted types and removal.

THREE PATHS, ONE COMPONENT

TypeScript

```
await upload.upload('files/sample.txt'); // set files directly
await upload.uploadViaFileChooser('files/sample.txt'); // real user path through the dialog
await upload.dropFile('report.csv', 'text/csv', 'a,b\n1,2'); // drag and drop
await upload.uploadBuffer('big.bin', 'application/octet-stream', Buffer.alloc(5_000_000));
```

WHEN YOU NEED TO CHANGE IT

- Your file list, progress bar or error container uses different markup, or a new upload gesture appears.

HOW TO CHANGE IT

- 1 Pass the selectors at construction. For size-boundary tests, generate the file at run time rather than committing a large binary.

```
const tooBig = path.join(os.tmpdir(), 'over-limit.pdf');
fs.writeFileSync(tooBig, Buffer.alloc(11 * 1024 * 1024));
await upload.upload(tooBig);
expect(await upload.getErrorMessage()).toContain('10 MB');
```

WHY THE CHANGE BELONGS HERE

Setting files directly works even when the input is hidden, but only the file-chooser path exercises the button a user actually clicks. Both are provided because they test different things.

Related: `src/utils/file.utils.ts`

API SURFACE (2 EXPORTS)

INTERFACE **FileUploadOptions**

```
triggerSelector?: string;
fileListSelector?: string;
progressSelector?: string;
errorSelector?: string;
removeSelector?: string;
```

CLASS **FileUpload** extends `BaseComponent`

File upload in all three flavours: a plain `<input type="file">`, a hidden input behind a styled button, and a drag-and-drop zone.

`upload(files: string | string[]): Promise<void>`

Sets files directly on the input — works even when the input is hidden.

`uploadViaFileChooser(files: string | string[]): Promise<void>`

Uploads through the file chooser dialog — exercises the real user path when clicking a styled button opens the OS picker.

`uploadBuffer(fileName: string, mimeType: string, buffer: Buffer): Promise<void>`

Uploads an in-memory buffer without touching disk.

`dropFile(fileName: string, mimeType: string, content: string): Promise<void>`

Simulates a drag-and-drop onto the zone by dispatching a `DataTransfer` drop.

`clearFiles(): Promise<void>``getSelectedFileNames(): Promise<string[]>``getUploadedFileNames(): Promise<string[]>`

File names as rendered by the application's own file list.

`waitForUploadComplete(timeout =
TIMEOUTS.EXTRA_LONG): Promise<void>``getUploadProgress(): Promise<number>``getErrorMessage(): Promise<string>``removeFile(fileName: string): Promise<void>``getAcceptedTypes(): Promise<string[]>`

The `accept` attribute, split into a list of allowed types.

`allowsMultiple(): Promise<boolean>`

src/components/form/form.ts

189 lines · 1 exports

Treats a whole `<form>` as one component. `fill()` takes a plain object keyed by field label and drives each control with the right interaction, inferring the control type from the DOM — which is what stops page objects accumulating twenty near-identical setter methods. Also exposes submit, reset, current values, error messages, browser validity, invalid field names and tab order.

A TWENTY-FIELD FORM IS ONE OBJECT LITERAL

TypeScript

```
await signupForm.fill({
  'Full name': 'Ada Lovelace',
  Country: 'Nigeria',      // native select or custom dropdown — both work
  'Accept terms': true,    // checkbox or switch
  Avatar: 'files/photo.png' // file input
});
await signupForm.submit();
expect(await signupForm.getErrors()).toHaveLength(0);
```

WHEN YOU NEED TO CHANGE IT

- A field cannot be found by label, placeholder, name or test id.
- Your application introduces a control type the inference does not recognise.

HOW TO CHANGE IT

- 1 For one awkward field, pass a spec with an explicit selector and kind.

```
await form.fill(
  { Country: 'Nigeria' },
  { Country: { kind: 'dropdown', selector: '[data-testid="country-combobox"]' } },
);
```

- 2 To support a new control type everywhere, add it to the `FieldKind` union, to `inferKind()` and to the `switch` in `setField()`.

```
case 'rating':
  await new Rating(this.page, field, { name: label }).rate(Number(value));
  break;
```

WHY THE CHANGE BELONGS HERE

Filling forms is the single most repeated action in UI testing. Making it data-driven collapses the largest source of page-object boilerplate and makes data-driven test cases trivial.

WATCH OUT FOR

- `fieldByLabel()` tries label, then placeholder, then `name / data-testid`. If a form has two fields with the same label, pass an explicit selector.

Related: `src/components/form/dropdown.ts` · `src/components/form/checkbox.ts`

API SURFACE (1 EXPORT)

CLASS `Form` extends `BaseComponent`

Composite component for a whole `<form>`. `fill()` takes a plain data object keyed by field label and drives each control with the right interaction — which is what stops page objects accumulating twenty near-identical `setX()` methods.

`fill(data: FormData, specs: Record<string, FieldSpec> = {}): Promise<void>`

Fills every field in the data object, inferring each control's type.

`setField(label: string, value: string | number | boolean | string[], spec: FieldSpec = {}): Promise<void>`

Sets one field, resolving it by label unless a selector is supplied.

`fieldByLabel(label: string): Locator`

Locates a control by its visible label, with sensible fallbacks.

`submit(submitSelector = 'button[type="submit"]', input[type="submit"]): Promise<void>`

`reset(resetSelector = 'button[type="reset"]'): Promise<void>`

`getValues(): Promise<Record<string, string>>`

Current values of every named control — for round-trip assertions.

```
getErrors(errorSelector = '[role="alert"],
.error, .invalid-feedback'): Promise<string[]>
```

Every validation message the form is currently showing.

```
hasErrors(): Promise<boolean>
```

```
isValid(): Promise<boolean>
```

Browser-level constraint validation for the whole form.

```
getInvalidFields(): Promise<string[]>
```

Names of every control the browser considers invalid.

```
fieldCount(): Promise<number>
```

```
getTabOrder(): Promise<string[]>
```

Tab order as the keyboard sees it — catches broken focus management.

src/components/form/multi-select.ts

96 lines · 2 exports

Extends `Dropdown` for controls that hold several values at once, usually rendered as removable chips. Adds selecting many, reading selected items, removing one, clearing all, and the Backspace gesture that removes the last chip.

WHEN YOU NEED TO CHANGE IT

- Your chips, their remove buttons, or the clear-all control use different markup.

HOW TO CHANGE IT

- 1 Pass `chipSelector`, `chipRemoveSelector` or `clearAllSelector` when constructing.

```
ui.multiSelect('#tags', { chipSelector: '.ant-select-selection-item', chipRemoveSelector
```

WHY THE CHANGE BELONGS HERE

Extending `Dropdown` rather than duplicating it means an improvement to panel handling benefits both classes.

Related: `src/components/form/dropdown.ts`

API SURFACE (2 EXPORTS)

INTERFACE `MultiSelectOptions`

```
chipSelector?: string;
chipRemoveSelector?: string;
clearAllSelector?: string;
```

CLASS **MultiSelect** extends `Dropdown`

Multi-select / tag input: several values selected at once, usually rendered as removable chips.

`get chips: Locator`

`selectMany(labels: string[]): Promise<void>`

`getSelectedItems(): Promise<string[]>`

`removeItem(label: string): Promise<void>`

`clearAll(): Promise<void>`

`selectedCount(): Promise<number>`

`isSelected(label: string): Promise<boolean>`

`removeLastWithKeyboard(): Promise<void>`

Backspace in an empty tag input removes the last chip.

src/components/form/radio-group.ts

129 lines · 1 exports

A set of mutually exclusive options addressed by value, label or index. The component wraps the container — a `role="radiogroup"`, a fieldset, or any wrapper — and discovers options beneath it.

WHEN YOU NEED TO CHANGE IT

- Your radio options are not `input[type="radio"]` or `[role="radio"]`.

HOW TO CHANGE IT

- 1 Pass `optionSelector` when constructing.

```
ui.radioGroup('[data-testid="plan"]', { optionSelector: '.plan-card' });
```

WHY THE CHANGE BELONGS HERE

Addressing options by label rather than index keeps tests readable and stable when the option order changes.

Related: `src/components/form/checkbox.ts`

API SURFACE (1 EXPORT)

CLASS **RadioGroup** extends `BaseComponent`

A set of mutually-exclusive radio options, addressed by value, label or index. The group locator should point at the container (`[role="radiogroup"]`, a fieldset, or any wrapper); options are

discovered beneath it.

```
selectByValue(value: string): Promise<void>
```

```
selectByLabel(label: string | RegExp):  
Promise<void>
```

```
selectByIndex(index: number): Promise<void>
```

```
getSelectedValue(): Promise<string | null>
```

```
getSelectedLabel(): Promise<string>
```

```
getOptionValues(): Promise<string[]>
```

```
getOptionLabels(): Promise<string[]>
```

```
optionCount(): Promise<number>
```

```
selectWithKeyboard(direction: 'next' |  
'previous'): Promise<void>
```

Arrow-key navigation is the accessible way to change a radio group.

src/components/form/rating.ts

77 lines · 2 exports

Star and heart rating controls: rating by position, reading the current and maximum rating, hover preview, read-only detection and clearing.

WHEN YOU NEED TO CHANGE IT

- Your rating items are not `[role="radio"]`, `.star`, `li` or `svg`, or selection is signalled by an unusual class.

HOW TO CHANGE IT

- 1 Pass `itemSelector` and `selectedClassPattern` at construction.

```
ui.rating('.product-rating', { itemSelector: '.rating-icon', selectedClassPattern: /is-f
```

WHY THE CHANGE BELONGS HERE

Reading `aria-valuenow` first means a properly labelled widget needs no configuration at all.

API SURFACE (2 EXPORTS)

INTERFACE `RatingOptions`


```
itemSelector?: string;
selectedClassPattern?: RegExp;
```

CLASS **Rating** extends `BaseComponent`

Star / heart rating control.

`get items: Locator`

`rate(value: number): Promise<void>`

Rates by clicking the nth star (1-based).

`getRating(): Promise<number>`

`getMaxRating(): Promise<number>`

`hoverRating(value: number): Promise<void>`

Hover preview — many rating widgets show a different value on hover.

`isReadOnly(): Promise<boolean>`

`clear(): Promise<void>`

src/components/form/select.ts

93 lines · 1 exports

Native `<select>` elements. Selection by value, label or index; reading selected value and label; enumerating options with their disabled state; multiple-selection support.

WHEN YOU NEED TO CHANGE IT

- Rarely. Native select behaviour is standardised.

HOW TO CHANGE IT

- 1 If the control is not a real `<select>`, use `Dropdown` instead — the two are separate classes precisely because their mechanics differ.

WHY THE CHANGE BELONGS HERE

Native selects are driven by Playwright's `selectOption`, which is faster and far more reliable than clicking a rendered list.

Related: `src/components/form/dropdown.ts`

API SURFACE (1 EXPORT)

CLASS **Select** extends `BaseComponent`

Native `<select>`. For framework dropdowns that render a listbox in a portal, use `Dropdown` instead.

```

selectByValue(value: string | string[]):
Promise<void>

selectByLabel(label: string | string[]):
Promise<void>

selectByIndex(index: number): Promise<void>

getSelectedValue(): Promise<string>

getSelectedLabel(): Promise<string>

getSelectedValues(): Promise<string[]>

getOptions(): Promise<Array<{ value: string;
label: string; disabled: boolean }>>

getOptionLabels(): Promise<string[]>

optionCount(): Promise<number>

isMultiple(): Promise<boolean>

hasOption(label: string): Promise<boolean>

clearSelection(): Promise<void>

```

src/components/form/slider.ts

119 lines · 1 exports

Range inputs and `role="slider"` controls, with three strategies because custom sliders rarely honour all of them: setting the value through the native setter (firing the events frameworks listen for), dragging the handle proportionally, and stepping with arrow keys.

WHEN YOU NEED TO CHANGE IT

- A custom slider ignores `setValue()` — that method requires a native range input.

HOW TO CHANGE IT

- 1 Fall back to `dragToValue()` or `stepBy()`, which work for any implementation.

```

await volume.dragToValue(75);
await volume.stepBy(3); // three ArrowRight presses

```

WHY THE CHANGE BELONGS HERE

React and similar frameworks track the native value setter, so assigning `element.value` directly is ignored. Calling the prototype setter is what makes the framework see the change.

Related: `src/components/feedback/progress.ts`

API SURFACE (1 EXPORT)

CLASS **Slider** extends `BaseComponent`

Range slider — native `<input type="range">` or `role="slider"`. Supports keyboard, mouse-drag and direct-value strategies, because custom sliders rarely honour all three.

`getValue(): Promise<number>`

`getMin(): Promise<number>`

`getMax(): Promise<number>`

`getStep(): Promise<number>`

`setValue(value: number): Promise<void>`

Sets the value on a native range input and fires input/change events.

`dragToValue(value: number): Promise<void>`

Drags the handle to a proportional position — works for any slider.

`stepBy(steps: number): Promise<void>`

Steps with arrow keys — the accessible path and the most reliable one.

`setToMin(): Promise<void>`

`setToMax(): Promise<void>`

`getPercentage(): Promise<number>`

Current value as a 0–100 percentage of the range.

src/components/form/text-area.ts

35 lines · 1 exports

Multi-line fields. Inherits all validation handling from `TextInput` and adds row count, character count and `typeLines()`, which enters multi-line content using either Enter or Shift+Enter for editors where Enter submits.

WHEN YOU NEED TO CHANGE IT

- A character counter or auto-grow behaviour needs asserting.

HOW TO CHANGE IT

- 1 Add the assertion here rather than in tests, so every textarea in the application is verified the same way.

WHY THE CHANGE BELONGS HERE

The difference between Enter and Shift+Enter is a property of the control; encoding it once avoids a class of confusing test failures in chat and comment fields.

Related: `src/components/form/text-input.ts`

API SURFACE (1 EXPORT)

CLASS `TextArea` extends `TextInput`Multi-line text field. Inherits validation handling from `TextInput`.`getRowCount(): Promise<number>``getCharacterCount(): Promise<number>`

Number of characters currently entered — for counter/limit assertions.

`typeLines(lines: string[], newlineKey: 'Enter' | 'Shift+Enter' = 'Enter'): Promise<void>`

Enters multi-line content, using Shift+Enter where Enter would submit.

`isResizable(): Promise<boolean>`

`src/components/form/text-input.ts`

167 lines · 1 exports

Single-line inputs of every flavour — text, email, password, number, tel, url, search — plus `contenteditable` fields. Handles the two ways of entering text (`fill` and `key-by-key` `pressSequentially`), clearing, pasting, and the whole validation surface: native constraint messages, `aria-invalid`, and error text resolved through `aria-describedby`.

TYPING MODES MATTER

TypeScript

```
await input.type('SAVE20'); // fast fill
await input.type('4242 4242', { pressSequentially: true }); // masked / per-key validation
await input.typeAndSettle('lap'); // types, then waits out the debounce
```

WHEN YOU NEED TO CHANGE IT

- Your application renders validation errors somewhere `aria-describedby` does not reach.
- A field type needs special handling (a currency mask, a phone formatter).

HOW TO CHANGE IT

- 1 Extend `getErrorMessage()` to look in your application's error container as a fallback, keeping the accessible lookup first.

```
async getErrorMessage(): Promise<string> {
  const accessible = await super.getErrorMessage();
  if (accessible) return accessible;
  const sibling = this.locator.locator('xpath=following-sibling::*[contains(@class,"field-error")]');
  return (await sibling.count()) > 0 ? (await sibling.first().innerText()).trim() : '';
}
```

- 2 For a specialised field, subclass rather than adding flags: `class CurrencyInput extends TextInput` keeps the general case simple.

WHY THE CHANGE BELONGS HERE

Reading validation state through ARIA first means the component tests the same signal a screen reader uses, so accessible markup and reliable tests reinforce each other.

WATCH OUT FOR

- `fill()` replaces the value in one operation and does not fire per-character events; masked inputs need `pressSequentially: true`.

Related: `src/components/form/text-area.ts` · `src/fixtures/custom-matchers.ts`

API SURFACE (1 EXPORT)

CLASS `TextInput` extends `BaseComponent`

Single-line inputs of every flavour (text, email, password, number, tel, url, search) plus `contenteditable` fields.

`type(value: string, options: FillOptions = {}): Promise<void>` Sets the value. `pressSequentially` types key by key — needed for inputs with masks, debounced search or per-keystroke validation.

`append(value: string): Promise<void>` Appends to the existing value instead of replacing it.

`clear(): Promise<void>`

`clearWithKeyboard(): Promise<void>` Clears using keyboard only — for fields that ignore programmatic fill.

`typeAndSettle(value: string, debounceMs = TIMEOUTS.DEBOUNCE): Promise<void>` Types then waits out the debounce — search boxes and typeaheads.

`submit(): Promise<void>`

`getValue(): Promise<string>` Current text value of the field.

`getPlaceholder(): Promise<string | null>`

`getInputType(): Promise<string>`

`getMaxLength(): Promise<number | null>`

`isRequired(): Promise<boolean>`

`isReadOnly(): Promise<boolean>`

`isValid(): Promise<boolean>` True when the browser's constraint validation marks the field invalid.

`getValidationMessage(): Promise<string>` The native browser validation message, e.g. "Please fill out this field".

`getErrorMessage(): Promise<string>`

The application's own error text, resolved via `aria-describedby` / `aria-errormessage`.

`hasError(): Promise<boolean>``paste(value: string): Promise<void>`

Pastes text via the clipboard API — some fields behave differently on paste.

src/components/form/toggle.ts

60 lines · 1 exports

Switches: `role="switch"`, styled checkboxes, or a pair of divs. State is read from the native `checked` property, then `aria-checked`, then `aria-pressed`, then class names — so it works across design systems. `turnOn` and `turnOff` are idempotent.

WHEN YOU NEED TO CHANGE IT

- Your switch signals state only through a class name the default pattern does not match.

HOW TO CHANGE IT

- 1 Extend the class-name pattern in `isOn()`, or better, ask for `aria-checked` to be added — it fixes the test and the accessibility of the control at once.

WHY THE CHANGE BELONGS HERE

Checking several state signals in priority order is what lets one class serve MUI, Ant, Bootstrap and hand-rolled switches.

Related: `src/components/form/checkbox.ts`

API SURFACE (1 EXPORT)

CLASS `Toggle` extends `BaseComponent`

Toggle / switch control — `role="switch"`, a styled checkbox, or a div pair.

`isOn(): Promise<boolean>``turnOn(): Promise<void>``turnOff(): Promise<void>``set(state: boolean): Promise<void>``toggle(): Promise<boolean>`

`toggleWithKeyboard(): Promise<void>`

Space is the accessible activation key for a switch.

Data display

`src/components/data/card.ts`

99 lines · 2 exports

Cards and tiles — product cards, dashboard widgets, search results. A small composite of title, body, image, price and actions, with `withTitle()` to narrow a collection to one card.

WHEN YOU NEED TO CHANGE IT

- Your card anatomy differs, or a card gains a new region worth asserting.

HOW TO CHANGE IT

- 1 Pass the region selectors at construction, then address individual cards by title.

```
const card = ui.card('.product-card').withTitle('Wireless Mouse');
await card.clickAction('Add to cart');
```

WHY THE CHANGE BELONGS HERE

Selecting a card by its title rather than its index keeps tests readable and immune to sort-order changes.

Related: `src/components/data/list.ts`

API SURFACE (2 EXPORTS)

INTERFACE `CardOptions`

```
titleSelector?: string;
bodySelector?: string;
imageSelector?: string;
actionSelector?: string;
priceSelector?: string;
```

CLASS `Card` extends `BaseComponent`

Card / tile — product cards, dashboard widgets, result items. A card is a small composite: title, body, image and actions.

```
withTitle(title: string | RegExp): Card
```

Narrows a collection of cards to the one with this title.

```
getTitle(): Promise<string>
```

```
getBody(): Promise<string>
```

```
getPrice(): Promise<string>
```

```
get image: Locator
```

```
getActionLabels(): Promise<string[]>
```

```
clickAction(name: string | RegExp):  
Promise<void>
```

```
isSelected(): Promise<boolean>
```

```
count(): Promise<number>
```

All cards in the collection this locator matches.

```
getAllTitles(): Promise<string[]>
```

src/components/data/data-grid.ts

165 lines · 2 exports

Extends `Table` for enterprise grids — AG Grid, MUI DataGrid, Kendo, PrimeNG — which differ in three ways that matter: rows are `role="row"` divs, data loads asynchronously, and only the visible window exists in the DOM. Adds waiting for data, scrolling to a row, reading all rows through virtualisation, column filtering, resizing, reordering, row expansion and inline cell editing.

WHEN YOU NEED TO CHANGE IT

- You adopt a different grid library, or need a grid feature not yet covered (grouping, pinned columns, aggregation rows).

HOW TO CHANGE IT

- Most libraries need only selector configuration.

```
ui.dataGrid('.ag-root-wrapper', {  
  rowSelector: '.ag-row',  
  cellSelector: '.ag-cell',  
  headerCellSelector: '.ag-header-cell',  
  virtualScrollContainerSelector: '.ag-body-viewport',  
  loadingSelector: '.ag-overlay-loading-center',  
});
```

- Add new grid features as methods here, bounded by a maximum scroll count so a missing row fails with a clear message instead of hanging.

WHY THE CHANGE BELONGS HERE

"Not in the DOM" does not mean "not in the data" for a virtualised grid. Scrolling until the row renders is the only correct way to assert on it, and doing that once here keeps the complexity out of tests.

WATCH OUT FOR

- `getAllRowsVirtualized()` deduplicates by row content; genuinely identical rows collapse into one.

Related: `src/components/data/table.ts`

API SURFACE (2 EXPORTS)

INTERFACE

DataGridOptions

```
loadingSelector?: string;
emptyStateSelector?: string;
virtualScrollContainerSelector?: string;
filterInputSelector?: string;
columnMenuSelector?: string;
```

CLASS

DataGrid

extends Table

Enterprise data grid (AG Grid, MUI DataGrid, Kendo, PrimeNG). Differs from `Table` in three ways that matter: rows are `role="row"` divs, content loads asynchronously, and only the visible window exists in the DOM.

<code>waitForData(timeout = TIMEOUTS.LONG): Promise<void></code>	Waits for the grid's own loading overlay to clear.
<code>isEmptyState(): Promise<boolean></code>	
<code>scrollToRow(criteria: Partial<TableRow>, maxScrolls = 40): Promise<number></code>	Scrolls a virtualised grid until the row is rendered, then returns it. Virtualisation means "not in the DOM" does not mean "not in the data".
<code>getAllRowsVirtualized(maxScrolls = 100): Promise<TableRow[]></code>	Loads every row of a virtualised grid by scrolling to the bottom.
<code>filterColumn(columnName: string, value: string): Promise<void></code>	Types into a column's filter box and waits for the grid to refresh.
<code>resizeColumn(columnName: string, deltaX: number): Promise<number></code>	Resizes a column by dragging its header handle; returns the new width.
<code>reorderColumn(fromColumn: string, toColumn: string): Promise<void></code>	Reorders columns by dragging one header onto another.
<code>expandRow(rowIndex: number, expanderSelector = '[aria-expanded], .expander'): Promise<void></code>	Expands a master/detail or tree row.

```
editCell(rowIndex: number, columnName: string,  
value: string): Promise<void>
```

Double-click a cell to enter inline edit, type, and commit with Enter.

src/components/data/list.ts

70 lines · 2 exports

Ordered and unordered lists, `role="list"` and card collections: reading items, clicking by text or index, membership checks, index lookup and order verification.

WHEN YOU NEED TO CHANGE IT

- Your list items are not `li` or `[role="listitem"]`.

HOW TO CHANGE IT

- 1 Pass `itemSelector`.

```
ui.list('[data-testid="results"]', { itemSelector: '.result-card' });
```

WHY THE CHANGE BELONGS HERE

Order assertions belong on the list, because "is it sorted" is a property of the collection rather than of any one test.

Related: `src/components/data/card.ts`

API SURFACE (2 EXPORTS)

INTERFACE `ListOptions`

```
itemSelector?: string;
```

CLASS `ListView` extends `BaseComponent`

Ordered/unordered lists, `role="list"`, and card collections.

```
get items: Locator
```

```
getItems(): Promise<string[]>
```

```
itemCount(): Promise<number>
```

```
isEmpty(): Promise<boolean>
```

```
itemByText(text: string | RegExp): Locator
```

```
clickItem(text: string | RegExp): Promise<void>
```

```
clickItemAt(index: number): Promise<void>
```

```
hasItem(text: string): Promise<boolean>
```

```
getItemIndex(text: string): Promise<number>
```

```
isOrderedAs(expected: string[]):  
Promise<boolean>
```

Order matters for sorted/prioritised lists.

src/components/data/pagination.ts

141 lines · 2 exports

Page controls: next, previous, first, last, a specific page, current page, total pages (from info text or the highest page link), page size, enabled state of the arrows, and `forEachPage()` for walking the whole set.

WHEN YOU NEED TO CHANGE IT

- Your pagination markup or info text differs from the defaults.

HOW TO CHANGE IT

- 1 Configure the selectors, then use `forEachPage` to run an assertion across every page.

```
await pagination.forEachPage(async (page) => {  
  expect(await table.rowCount()).toBeLessThanOrEqual(25);  
});
```

WHY THE CHANGE BELONGS HERE

Deriving total pages from the info text first, then from the page links, means the component works whether or not the application renders a summary.

Related: `src/components/data/table.ts`

API SURFACE (2 EXPORTS)

INTERFACE `PaginationOptions`

```

nextSelector?: string;
previousSelector?: string;
firstSelector?: string;
lastSelector?: string;
pageSelector?: string;
currentPageSelector?: string;
pageSizeSelector?: string;
infoSelector?: string;

```

CLASS **Pagination** extends `BaseComponent`

Pagination control: page numbers, next/previous, and page-size selection.

`goToNext(): Promise<void>`

`goToPrevious(): Promise<void>`

`goToFirst(): Promise<void>`

`goToLast(): Promise<void>`

`goToPage(pageNumber: number): Promise<void>`

`getCurrentPage(): Promise<number>`

`getTotalPages(): Promise<number>`

Total pages, from the info text ("Page 2 of 7") or the highest page link.

`setPageSize(size: number | string): Promise<void>`

`isNextEnabled(): Promise<boolean>`

`isPreviousEnabled(): Promise<boolean>`

`forEachPage(onPage: (pageNumber: number) => Promise<void>): Promise<void>`

Walks every page, running `onPage` for each.

src/components/data/table.ts

227 lines · 2 exports

HTML tables, with everything addressable by column **name** rather than index: reading all rows as objects, single cells, whole columns, finding rows by criteria, clicking a control inside a matching row, sorting, verifying sort order, and row selection.

NAME-BASED ADDRESSING IS THE POINT

TypeScript

```

expect(await results.getAllRows()).toEqual([{ Name: 'Ada', Amount: '10' }]);
expect(await results.getColumnValues('Amount')).toBeSorted('asc');
await results.clickRowAction({ Name: 'Ada' }, 'button[aria-label="Edit"]');

```

WHEN YOU NEED TO CHANGE IT

- Your table markup is not `thead` / `tbody` / `th` / `td`.

- Sorting is signalled by something other than `aria-sort` or an `asc/desc` class.
- You need a new table-wide operation (export, column totals, multi-column sort).

HOW TO CHANGE IT

- 1 Point the component at your markup with the options.

```
ui.table('#report', {
  headerSelector: '.thead',
  rowSelector: '.tr>tr',
  cellSelector: '.td',
  headerCellSelector: '.th',
});
```

- 2 Add table-wide behaviour as a method here so every table gets it, and use `getColumnIndex()` so it keeps working when columns move.

```
async getColumnTotal(columnName: string): Promise<number> {
  const values = await this.getColumnValues(columnName);
  return values.reduce((sum, value) => sum + Number(normalizeText(value).replace(/^[^d.-
```

WHY THE CHANGE BELONGS HERE

Index-based table tests break the moment a column is inserted — the most common cause of large-scale suite breakage. Resolving the index from the header text once, at call time, removes that entire failure mode.

WATCH OUT FOR

- `getColumnIndex()` throws with the list of available headers, which is usually enough to diagnose the failure without opening the trace.

Related: `src/components/data/data-grid.ts` · `src/fixtures/custom-matchers.ts`

API SURFACE (2 EXPORTS)

INTERFACE `TableOptions`

```
headerSelector?: string;
rowSelector?: string;
cellSelector?: string;
headerCellSelector?: string;
```

CLASS `Table` extends `BaseComponent`

HTML table. Everything is addressable by column **name** rather than index, so tests keep working when a column is inserted — the single most common cause of large-scale table-test breakage.

<code>get rows: Locator</code>	
<code>get headerCells: Locator</code>	
<code>getHeaders(): Promise<string[]></code>	
<code>rowCount(): Promise<number></code>	
<code>columnCount(): Promise<number></code>	
<code>isEmpty(): Promise<boolean></code>	
<code>getColumnIndex(columnName: string): Promise<number></code>	Index of a column by header text. Throws with the available headers listed.
<code>getAllRows(): Promise<TableRow[]></code>	The whole table as objects keyed by header text.
<code>getRow(rowIndex: number): Promise<TableRow></code>	
<code>getCellText(rowIndex: number, column: number string): Promise<string></code>	
<code>getCell(rowIndex: number, column: number string): Promise<Locator></code>	Cell by row index and column — the column may be an index or a header name.
<code>getColumnValues(columnName: string): Promise<string[]></code>	
<code>findRowsByColumnValue(columnName: string, value: string RegExp): Promise<Locator></code>	Rows whose named column matches <code>value</code> .
<code>findRowsContaining(value: string RegExp): Locator</code>	Rows containing the given text in any cell.
<code>findRowIndex(criteria: Partial<TableRow>): Promise<number></code>	Row index matching a full or partial record; -1 when nothing matches.
<code>containsRow(criteria: Partial<TableRow>): Promise<boolean></code>	
<code>clickRowAction(criteria: Partial<TableRow>, actionSelector: string): Promise<void></code>	Clicks a control (button/link) inside the row matching the criteria.
<code>sortBy(columnName: string, direction: SortDirection = 'asc'): Promise<void></code>	
<code>getSortDirection(columnName: string): Promise<SortDirection></code>	
<code>isColumnSorted(columnName: string, direction: SortDirection = 'asc'): Promise<boolean></code>	Verifies the column is genuinely ordered — string or numeric aware.
<code>selectRow(rowIndex: number, checkboxSelector = 'input[type="checkbox"]'): Promise<void></code>	
<code>selectAllRows(headerCheckboxSelector = 'thead input[type="checkbox"]'): Promise<void></code>	

```
getSelectedRowCount(checkboxSelector =
☐):
Promise<number>
```

src/components/data/tree.ts

112 lines · 2 exports

Tree views and file explorers: expanding and collapsing, walking a path of ancestors to a leaf, selection state, nesting level from `aria-level`, listing visible nodes and children.

WHEN YOU NEED TO CHANGE IT

- Your tree nodes are not `[role="treeitem"]`, or expansion state is signalled only by a class.

HOW TO CHANGE IT

- 1 Pass `nodeSelector`, `labelSelector` and `toggleSelector`; `expandPath()` then works unchanged.

```
await tree.expandPath(['src', 'components', 'form', 'button.ts']);
```

WHY THE CHANGE BELONGS HERE

Expanding ancestors in order is the only reliable way to reach a leaf in a lazily rendered tree, and it is worth writing once.

API SURFACE (2 EXPORTS)

INTERFACE **TreeOptions**

```
nodeSelector?: string;
labelSelector?: string;
toggleSelector?: string;
```

CLASS **TreeView** extends `BaseComponent`

Tree view / file explorer — `role="tree"` with expandable `treeitem` nodes.

```
get nodes: Locator
```

```
node(label: string): Locator
```

```
expand(label: string): Promise<void>
```

```
collapse(label: string): Promise<void>
```

```
expandPath(path: string[]): Promise<void>
```

Expands each ancestor in turn, then selects the leaf.

```
isExpanded(label: string): Promise<boolean>
```

```
select(label: string): Promise<void>
```

```
isSelected(label: string): Promise<boolean>
```

```
getLevel(label: string): Promise<number>
```

Nesting depth via aria-level, or -1 when the tree does not expose it.

```
getVisibleNodes(): Promise<string[]>
```

```
getChildren(parentLabel: string):  
Promise<string[]>
```

```
nodeCount(): Promise<number>
```

Navigation

src/components/navigation/accordion.ts

108 lines · 2 exports

Collapsible sections: expand, collapse, toggle, expansion state (via `aria-expanded`, the `open` attribute on `details`, or class names), section content, expand-all and collapse-all, and counting expanded sections to verify single-open behaviour.

WHEN YOU NEED TO CHANGE IT

- Your accordion uses different header or panel markup.

HOW TO CHANGE IT

- 1 Pass `headerSelector`, `panelSelector` and `itemSelector` at construction.

WHY THE CHANGE BELONGS HERE

Checking three different expansion signals in order is what makes one class work for `<details>`, ARIA accordions and CSS-class implementations.

Related: `src/components/navigation/tabs.ts`

API SURFACE (2 EXPORTS)

INTERFACE `AccordionOptions`


```
headerSelector?: string;
panelSelector?: string;
itemSelector?: string;
```

CLASS **Accordion** extends `BaseComponent`

Accordion / collapsible sections.

`get headers: Locator`

`header(title: string | RegExp): Locator`

`expand(title: string | RegExp): Promise<void>`

`collapse(title: string | RegExp): Promise<void>`

`toggle(title: string | RegExp): Promise<void>`

`isExpanded(title: string | RegExp):
Promise<boolean>`

`getContent(title: string | RegExp):
Promise<string>`

`getSectionTitles(): Promise<string[]>`

`sectionCount(): Promise<number>`

`expandAll(): Promise<void>`

`collapseAll(): Promise<void>`

`getExpandedCount(): Promise<number>`

True when opening one section closes the others.

src/components/navigation/breadcrumb.ts

69 lines · 2 exports

Breadcrumb trails: reading the trail, the current page, navigating to an ancestor, going up a number of levels, and depth.

WHEN YOU NEED TO CHANGE IT

- Your separator or item markup differs.

HOW TO CHANGE IT

- 1 Pass `itemSelector` and `separator` so trail text is cleaned correctly.

WHY THE CHANGE BELONGS HERE

Stripping the separator from item text is what makes trail assertions readable rather than full of slashes and chevrons.

API SURFACE (2 EXPORTS)

INTERFACE **BreadcrumbOptions**

```
itemSelector?: string;
separator?: string;
```

CLASS **Breadcrumb** extends `BaseComponent`

Breadcrumb trail — position within the hierarchy, and navigation up it.

```
get items(): Locator
```

```
getTrail(): Promise<string[]>
```

```
getCurrent(): Promise<string>
```

```
navigateTo(name: string | RegExp):
Promise<void>
```

```
goUp(levels = 1): Promise<void>
```

```
depth(): Promise<number>
```

src/components/navigation/link.ts

67 lines · 1 exports

Anchors, including the awkward parts: reading `href` and `target`, clicking and waiting for navigation, opening and returning a new tab, cheap reachability checking with a HEAD request, and internal-versus-external detection.

WHEN YOU NEED TO CHANGE IT

- You need link-checking behaviour beyond a HEAD request, or your application intercepts anchor clicks.

HOW TO CHANGE IT

- 1 Use `clickAndGetNewTab()` for `target="_blank"` links; it returns the new page for assertions.

```
const docs = await helpLink.clickAndGetNewTab();
await expect(docs).toHaveTitle(/documentation/i);
```

WHY THE CHANGE BELONGS HERE

Capturing the new page at click time is the only reliable way to test a link that opens a tab; polling the context afterwards races the browser.

Related: `src/utils/browser.utils.ts`

API SURFACE (1 EXPORT)

CLASS **Link** extends `BaseComponent`

Anchor element, including download and new-tab behaviours.

`getHref(): Promise<string | null>`

`getTarget(): Promise<string | null>`

`opensInNewTab(): Promise<boolean>`

`clickAndWait(expectedUrl?: string | RegExp): Promise<void>` Clicks and waits for the URL to change.

`clickAndGetNewTab(): Promise<Page>` Follows a `target="_blank"` link and returns the new tab.

`isReachable(): Promise<boolean>` Verifies the target resolves without leaving the page — cheap link checking.

`isExternal(): Promise<boolean>`

`src/components/navigation/menu.ts`

104 lines · 2 exports

Navigation menus, menubars and context menus, including nested submenus. `navigateTo(path)` walks a nested path with the gesture the menu expects (click or hover), and `openContextMenu(target)` right-clicks an element to open it.

WHEN YOU NEED TO CHANGE IT

- Your menu opens on hover, or submenu markup differs.

HOW TO CHANGE IT

- 1 Set `openOn: 'hover'` and the submenu selector at construction.

```
const nav = ui.menu('nav.main', { openOn: 'hover', submenuSelector: '.dropdown-panel' })
await nav.navigateTo(['Reports', 'Sales', 'By region']);
```

WHY THE CHANGE BELONGS HERE

Encoding the open gesture as configuration keeps one class usable for both click-driven and hover-driven navigation.

API SURFACE (2 EXPORTS)

INTERFACE **MenuOptions**

```
itemSelector?: string;
submenuSelector?: string;
openOn?: 'click' | 'hover';
```

CLASS **Menu** extends `BaseComponent`

Navigation menu / menubar / context menu, including nested submenus.

`get items: Locator`

`item(name: string | RegExp): Locator`

`clickItem(name: string | RegExp): Promise<void>`

Clicks a top-level entry by name. Use `navigateTo` for nested paths.

`navigateTo(path: string[]): Promise<void>`

Walks a nested path, opening each level the way the menu expects. `['Reports', 'Sales', 'By region']`

`getItemNames(): Promise<string[]>`

`hasItem(name: string): Promise<boolean>`

`isItemDisabled(name: string): Promise<boolean>`

`getActiveItem(): Promise<string>`

`openContextMenu(target: Locator): Promise<void>`

Opens a context menu at the given element and returns this menu.

`src/components/navigation/stepper.ts`

106 lines · 2 exports

Multi-step wizards: next, back, finish, jumping to a step, current step number and name, all step names, completion and disabled state, and whether you are on the last step.

WHEN YOU NEED TO CHANGE IT

- Your wizard buttons are labelled differently, or steps are marked complete by another class.

HOW TO CHANGE IT

1 Pass the button selectors at construction.

```
ui.stepper('.wizard', {
  nextSelector: '[data-testid="continue"]',
  finishSelector: '[data-testid="place-order"]',
});
```

WHY THE CHANGE BELONGS HERE

Keeping step navigation in the component lets a checkout test read as business steps rather than as button clicks.

API SURFACE (2 EXPORTS)

INTERFACE **StepperOptions**

```
stepSelector?: string;
nextSelector?: string;
backSelector?: string;
finishSelector?: string;
```

CLASS **Stepper** extends `BaseComponent`

Multi-step wizard / stepper: current step, completion state, and traversal.

`get steps: Locator`

`next(): Promise<void>`

`back(): Promise<void>`

`finish(): Promise<void>`

`getCurrentStep(): Promise<number>` Current step number, 1-based.

`getCurrentStepName(): Promise<string>`

`getStepNames(): Promise<string[]>`

`stepCount(): Promise<number>`

`isStepCompleted(stepNumber: number): Promise<boolean>`

`isStepDisabled(stepNumber: number): Promise<boolean>`

`goToStep(stepNumber: number): Promise<void>` Jumps directly to a step when the stepper allows non-linear navigation.

`isLastStep(): Promise<boolean>`

src/components/navigation/tabs.ts

111 lines · 2 exports

Tab strips and their panels: selecting by name or index, reading the active tab and all tab names, disabled state, resolving the panel bound to a tab through `aria-controls`, and arrow-key navigation per the WAI-ARIA pattern.

WHEN YOU NEED TO CHANGE IT

- Your tabs are not `[role="tab"]`, or panels are not linked with `aria-controls`.

HOW TO CHANGE IT

- 1 Pass `tabSelector` and `panelSelector`; assert against `activePanel` so the assertion follows the selection.

```
await tabs.select('Billing');
await expect(tabs.activePanel).toContainText('Payment method');
```

WHY THE CHANGE BELONGS HERE

Resolving the panel from `aria-controls` means the component follows the application's own wiring instead of guessing which panel is visible.

Related: `src/components/navigation/accordion.ts`

API SURFACE (2 EXPORTS)**INTERFACE** **TabsOptions**

```
tabSelector?: string;
panelSelector?: string;
```

CLASS **Tabs** extends `BaseComponent`

Tab strip with associated panels (`role="tablist"` / `tab` / `tabpanel`).

```
get tabs: Locator
```

```
tab(name: string | RegExp): Locator
```

```
select(name: string | RegExp): Promise<void>
```

```
selectByIndex(index: number): Promise<void>
```

```
getActiveTab(): Promise<string>
```

```
getTabNames(): Promise<string[]>
```

```
tabCount(): Promise<number>
```

```
isTabActive(name: string): Promise<boolean>
```

```
isTabDisabled(name: string): Promise<boolean>
```

```
get activePanel: Locator
```

The panel currently shown — assert content against this.

```
panelFor(name: string | RegExp):  
Promise<Locator>
```

Panel bound to a tab via aria-controls, falling back to the visible one.

```
navigateWithKeyboard(direction: 'next' |  
'previous'): Promise<void>
```

Arrow-key navigation between tabs, per the WAI-ARIA tabs pattern.

Feedback

`src/components/feedback/alert.ts`

109 lines · 2 exports

Toasts, snackbars and inline alerts. `waitForMessage()` races the auto-dismiss timer instead of assuming the toast is still on screen; severity is classified from role and class names; dismissal, bulk dismissal, auto-dismiss verification and screen-reader announcement checks are included.

WHEN YOU NEED TO CHANGE IT

- Your toast markup or severity classes differ, or a new severity appears.

HOW TO CHANGE IT

- 1 Extend the severity patterns in `getSeverity()`, or pass a message selector when the text sits in a child element.

```
ui.alert('.Toastify__toast', { messageSelector: '.Toastify__toast-body' });
```

WHY THE CHANGE BELONGS HERE

Transient toasts are a classic flaky assertion: the test reads the element after it has gone. Capturing the text during the visibility window removes the race entirely.

WATCH OUT FOR

- If a toast is genuinely gone before the assertion, slow the application rather than the test — or assert on the resulting state instead.

Related: `src/components/feedback/tooltip.ts`

API SURFACE (2 EXPORTS)

INTERFACE **AlertOptions**

```
messageSelector?: string;  
closeSelector?: string;
```

CLASS **Alert** extends `BaseComponent`

Toast / snackbar / inline alert. Toasts are transient by design, so `waitForMessage` races the auto-dismiss timer rather than assuming the element is still there.

`waitForMessage(timeout = TIMEOUTS.SHORT): Promise<string>` Waits for the alert to appear and returns its text before it disappears.

`getMessage(): Promise<string>`

`getAllMessages(): Promise<string[]>`

`getSeverity(): Promise<AlertSeverity>` Classifies the alert from its role/class — success vs error assertions.

`dismiss(): Promise<void>`

`dismissAll(): Promise<void>`

`waitForAutoDismiss(timeout = TIMEOUTS.MEDIUM): Promise<void>` Confirms the toast disappears on its own within the expected window.

`count(): Promise<number>`

`isAnnouncedToScreenReaders(): Promise<boolean>` Screen readers only announce alerts with a live region.

src/components/feedback/modal.ts

133 lines · 2 exports

Dialogs, drawers and confirmations: waiting for open and close, title and body text, confirm, cancel, close, Escape, overlay click, arbitrary buttons by name — plus the two things modals routinely get wrong, focus trapping and ARIA semantics.

ACCESSIBILITY CHECKS THAT BELONG ON THE COMPONENT

TypeScript

```
expect(await confirmDialog.hasCorrectAriaSemantics()).toBe(true); // role + aria-modal  
expect(await confirmDialog.isFocusTrapped()).toBe(true);
```

WHEN YOU NEED TO CHANGE IT

- Your dialog markup or button labels differ, or your design system needs a different close gesture.

HOW TO CHANGE IT

- 1 Pass the selectors at construction; use `clickButton(name)` for buttons that are not confirm or cancel.

WHY THE CHANGE BELONGS HERE

Focus trapping and `aria-modal` are correctness properties of every dialog, so checking them belongs with the component rather than in a separate accessibility test.

Related: `src/utils/allutils.ts`

API SURFACE (2 EXPORTS)

INTERFACE `ModalOptions`

```
titleSelector?: string;
bodySelector?: string;
closeSelector?: string;
confirmSelector?: string;
cancelSelector?: string;
overlaySelector?: string;
```

CLASS `Modal` extends `BaseComponent`

Modal dialog / drawer / confirmation popup. Also verifies the two things modals routinely get wrong: focus trapping and Escape-to-close.

```
waitForOpen(timeout = this.timeout):
Promise<void>
```

```
waitForClose(timeout = this.timeout):
Promise<void>
```

```
isOpen(): Promise<boolean>
```

```
getTitle(): Promise<string>
```

```
getBodyText(): Promise<string>
```

```
confirm(): Promise<void>
```

```
cancel(): Promise<void>
```

```
close(): Promise<void>
```

```
closeWithEscape(): Promise<void>
```

```
clickOverlay(): Promise<void>
```

```
button(name: string | RegExp): Locator
```

```
clickButton(name: string | RegExp):
Promise<void>
```

`isFocusTrapped(): Promise<boolean>`

True when Tab cycles focus inside the dialog, as a modal must.

`hasCorrectAriaSemantics(): Promise<boolean>`

A correct dialog exposes `role=dialog` and `aria-modal=true`.

src/components/feedback/progress.ts

80 lines · 2 exports

Two related components in one file. `ProgressBar` reads value, maximum, percentage and indeterminate state, and can wait for completion or a threshold. `Loader` models a spinner or skeleton — an element whose whole job is to disappear — with `waitForFinish()` and `around(action)`.

WRAPPING AN ACTION IN ITS SPINNER

TypeScript

```
await loader.around(() => saveButton.click()); // click, then wait for the spinner to clear
```

WHEN YOU NEED TO CHANGE IT

- Your progress indicator does not expose `aria-valuenow`, or a new loading pattern appears.

HOW TO CHANGE IT

- 1 Add the reading logic to `getValue()`; keep `waitForComplete` expressed in terms of percentage so callers do not care about the source.

WHY THE CHANGE BELONGS HERE

A spinner that never appears is as common as one that never leaves; `waitForFinish` tolerates both by treating the appearance as optional and the disappearance as required.

Related: `src/components/form/button.ts`

API SURFACE (2 EXPORTS)

CLASS `ProgressBar` extends `BaseComponent`

Determinate/indeterminate progress bar.

`getValue(): Promise<number>``getMax(): Promise<number>``getPercentage(): Promise<number>``isIndeterminate(): Promise<boolean>`

```
waitForComplete(timeout = TIMEOUTS.EXTRA_LONG):
Promise<void>
```

```
waitForProgress(minimumPercent: number, timeout
= TIMEOUTS.LONG): Promise<void>
```

CLASS **Loader** extends `BaseComponent`

Spinner / skeleton loader — an element whose whole job is to disappear.

```
waitForFinish(timeout = TIMEOUTS.LONG):
Promise<void>
```

```
isLoading(): Promise<boolean>
```

```
around(action: () => Promise<void>, timeout =
TIMEOUTS.LONG): Promise<void>
```

Runs an action and waits for the spinner it triggers to clear.

src/components/feedback/tooltip.ts

95 lines · 2 exports

Tooltips and popovers. The component wraps the **trigger**; the bubble is found separately because it is nearly always portalled. Supports hover, click and focus triggers, reading text, native `title` attributes, computed placement, and whether trigger and bubble are linked by `aria-describedby`.

WHEN YOU NEED TO CHANGE IT

- Your tooltip is triggered differently or rendered in a specific container.

HOW TO CHANGE IT

- 1 Pass `trigger` and `tooltipSelector` at construction.

```
ui.tooltip('#help-icon', { trigger: 'click', tooltipSelector: '.popover-body' });
```

WHY THE CHANGE BELONGS HERE

Native `title` tooltips never appear in the DOM, so the component reads the attribute directly — otherwise those tests would be impossible to write.

API SURFACE (2 EXPORTS)

INTERFACE **TooltipOptions**

```
tooltipSelector?: string;
trigger?: 'hover' | 'click' | 'focus';
```

CLASS **Tooltip** extends `BaseComponent`

Tooltip / popover. The component wraps the **trigger**; the bubble is found separately because it is nearly always portalled elsewhere in the DOM.

`get bubble: Locator`

`show(): Promise<string>`

Triggers the tooltip and returns its text.

`hide(): Promise<void>`

`isVisible(): Promise<boolean>`

`getText(): Promise<string>`

`getNativeTitle(): Promise<string | null>`

Native `title` attribute tooltips never render as DOM — read them directly.

`getPlacement(): Promise<'top' | 'bottom' | 'left' | 'right' | 'unknown'>`

Where the bubble landed relative to the trigger — for placement assertions.

`isAccessiblyLinked(): Promise<boolean>`

Accessible tooltips link the trigger to the bubble via `aria-describedby`.

Media

`src/components/media/canvas.ts`

113 lines · 1 exports

Canvas elements — signature pads, drawing tools, charts and games. Since canvas has no DOM to query, everything is coordinate- or pixel-based: drawing strokes, clicking and hovering at points, reading a pixel colour, checking whether the canvas is blank, exporting a data URL and reading dimensions.

WHEN YOU NEED TO CHANGE IT

- You need a new gesture (pinch, multi-touch) or a different verification method.

HOW TO CHANGE IT

- 1 Compose new gestures from mouse moves relative to the canvas origin, as `draw()` does.

```
await signature.scribble();
expect(await signature.isBlank()).toBe(false);
```

WHY THE CHANGE BELONGS HERE

Pixel and blank checks are the only assertions available for canvas content, so providing them here prevents every test from re-implementing `getImageData` logic.

WATCH OUT FOR

- `getPixelColor` and `isBlank` require a same-origin, non-tainted canvas.

API SURFACE (1 EXPORT)

CLASS `Canvas` extends `BaseComponent`

`<canvas>` — signature pads, drawing tools, charts and games. Canvas has no DOM to query, so everything here is coordinate- or pixel-based. Coordinates are relative to the canvas's top-left corner.

`draw(points: Point[]): Promise<void>` Draws a continuous stroke through the given points.

`clickAt(point: Point): Promise<void>`

`hoverAt(point: Point): Promise<void>`

`drawLine(from: Point, to: Point): Promise<void>` Draws a straight line — the simplest signature-pad interaction.

`scribble(): Promise<void>` Draws a scribble that reliably produces non-blank canvas content.

`getPixelColor(point: Point): Promise<[number, number, number, number]>` RGBA of one pixel — verifies something was actually drawn.

`isBlank(): Promise<boolean>` True when every pixel is fully transparent.

`toDataURL(): Promise<string>` Canvas contents as a data URL — usable as a visual baseline.

`getDimensions(): Promise<{ width: number; height: number }>`

`src/components/media/carousel.ts`

123 lines · 2 exports

Image and content carousels: next, previous, going to a slide by indicator or by stepping, current slide index and text, slide count, verifying autoplay advances, and swiping for the mobile path.

WHEN YOU NEED TO CHANGE IT

- Your carousel markup or active-slide signal differs.

HOW TO CHANGE IT

1 Pass `slideSelector`, `indicatorSelector` and `activeSlidePattern` at construction.

WHY THE CHANGE BELONGS HERE

Autoplay is a behaviour worth asserting and easy to get wrong; `waitForAutoAdvance()` gives it a bounded, explicit check.

API SURFACE (2 EXPORTS)

INTERFACE `CarouselOptions`

```
slideSelector?: string;
nextSelector?: string;
previousSelector?: string;
indicatorSelector?: string;
activeSlidePattern?: RegExp;
```

CLASS `Carousel` extends `BaseComponent`

Image/content carousel with arrows, indicators and (often) autoplay.

`get slides(): Locator`

`get indicators(): Locator`

`next(): Promise<void>`

`previous(): Promise<void>`

`goToSlide(index: number): Promise<void>`

`getCurrentSlideIndex(): Promise<number>` Current slide number, 1-based.

`slideCount(): Promise<number>`

`getCurrentSlideText(): Promise<string>`

`waitForAutoAdvance(timeout = TIMEOUTS.MEDIUM): Promise<boolean>` Confirms autoplay advances the carousel on its own.

`swipe(direction: 'left' | 'right'): Promise<void>` Swipes horizontally — the mobile interaction path.

`src/components/media/frame.ts`

84 lines · 1 exports

An iframe wrapper. Frames are not components — they are **scopes**. This class hands you a `FrameLocator` that any other component can be built against, so cross-frame testing needs no special-case code anywhere else. Also exposes the iframe element itself, load waiting, nesting, the frame URL and a static listing of all frames.

ANY COMPONENT, INSIDE A FRAME

TypeScript

```
const payment = new Frame(page, '#payment-iframe');
await payment.waitForLoaded();

const card = new TextInput(payment.locator, '#card-number');
const pay = new Button(payment.locator, '#submit');
// or: const inFrame = ui(payment.locator);
```

WHEN YOU NEED TO CHANGE IT

- You need nested frame handling or frame-specific diagnostics.

HOW TO CHANGE IT

- 1 Use `child()` for nested frames; use the static `Frame.listFrames(page)` when the selector is unknown.

WHY THE CHANGE BELONGS HERE

Treating a frame as a scope rather than a component is what keeps iframe support from leaking into all 37 component classes.

WATCH OUT FOR

- Cross-origin frames cannot be inspected from the parent document, so `waitForLoaded()` degrades gracefully instead of failing.

Related: `src/core/locator.factory.ts`

API SURFACE (1 EXPORT)

CLASS **Frame**

iframe wrapper. Frames are not components — they are **scopes**. This class hands you a `FrameLocator` that every other component can be constructed against, so cross-frame testing needs no special-case code anywhere else: `const frame = new Frame(page, '#payment-iframe');`
`const cardNumber = new TextInput(frame.locator, '#card-number');`

`get locator: FrameLocator`

Scope to pass into component constructors.

`get element: Locator`

The iframe element itself — for size, visibility and attribute checks.

`waitForLoaded(timeout = TIMEOUTS.LONG):
Promise<void>`

`child(selector: string): Frame`

`get url(): Promise<string | null>`

`count(): Promise<number>`

```
evaluate(fn: () => T): Promise<T>
```

Runs JS inside the frame's own document (same-origin only).

```
listFrames(page: Page): Array<{ name: string; url: string }>
```

Lists every frame on the page — useful when the selector is unknown.

src/components/media/image.ts

75 lines · 1 exports

Images, including the cases that matter: whether the bitmap actually decoded (`complete` plus non-zero `naturalWidth`, not merely "element visible"), intrinsic dimensions, broken detection, lazy-loading, `srcset` entries and accessible alt-text rules.

WHEN YOU NEED TO CHANGE IT

- You need image-specific assertions such as aspect ratio or format checking.

HOW TO CHANGE IT

- 1 Add the method here and express it in terms of `getNaturalSize()` so it works for any image.

```
async getAspectRatio(): Promise<number> {
  const { width, height } = await this.getNaturalSize();
  return height === 0 ? 0 : width / height;
}
```

WHY THE CHANGE BELONGS HERE

A broken image is still a visible element. Checking the decoded bitmap is the only assertion that catches a 404 asset.

API SURFACE (1 EXPORT)

CLASS `Image` extends `BaseComponent`

`<img>` — including the load-failure and lazy-loading cases.

```
getSrc(): Promise<string | null>
```

```
getAlt(): Promise<string | null>
```

```
isLoading(): Promise<boolean>
```

True once the browser has decoded the bitmap — not merely "element visible".

```
waitForLoaded(timeout = this.timeout):
Promise<void>
```



```
getNaturalSize(): Promise<{ width: number; height: number }>
```

Intrinsic pixel dimensions — catches wrong-asset and stretched-image bugs.

```
isBroken(): Promise<boolean>
```

```
isLazyLoaded(): Promise<boolean>
```

```
getSrcSet(): Promise<string[]>
```

srcset entries, for responsive-image coverage.

```
hasAccessibleAlt(): Promise<boolean>
```

Decorative images must have empty alt; meaningful ones must have text.

src/components/media/media-player.ts

149 lines · 1 exports

Video and audio driven through the media element's own DOM API rather than custom player chrome, so the same component works for any skin: play, pause, seek, volume, mute, playback rate, readiness, playback progress, ended state, current source and text tracks.

WHEN YOU NEED TO CHANGE IT

- You need to test the player's visible controls rather than the media element itself.

HOW TO CHANGE IT

- 1 Use `Button` components for the chrome and this component for verification — click the custom play button, then assert `isPlaying()` here.

WHY THE CHANGE BELONGS HERE

Driving the element API is stable across player redesigns; asserting through it while clicking the real controls tests both layers honestly.

WATCH OUT FOR

- Autoplay policies block programmatic `play()` with sound in some browsers; mute first when that applies.

API SURFACE (1 EXPORT)

CLASS `MediaPlayer` extends `BaseComponent`

`<video>` and `<audio>`. Drives the media element through its DOM API rather than clicking custom player chrome, so the same component works for any skin.

```
play(): Promise<void>
```

```

pause(): Promise<void>

isPlaying(): Promise<boolean>

getCurrentTime(): Promise<number>

getDuration(): Promise<number>

seekTo(seconds: number): Promise<void>

setVolume(level: number): Promise<void>

getVolume(): Promise<number>

mute(): Promise<void>

unmute(): Promise<void>

isMuted(): Promise<boolean>

setPlaybackRate(rate: number): Promise<void>

waitForReady(timeout = TIMEOUTS.LONG): Promise<void>
readyState >= 3 means enough data is
buffered to play through.

waitForPlaybackProgress(seconds: number, timeout
= TIMEOUTS.LONG): Promise<void>

hasEnded(): Promise<boolean>

getSource(): Promise<string>

getTextTracks(): Promise<Array<{ kind: string;
label: string; language: string }>>
Captions/subtitles tracks — required for
accessible video.

```

Advanced interactions

src/components/advanced/chart.ts

141 lines · 2 exports

SVG charts from D3, Highcharts, Chart.js in SVG mode or Recharts. Charts have no text to assert on, so the useful checks are the number of marks, the legend, axis labels, tooltips revealed on hover, series toggling, values in data attributes, bar heights as a proxy for relative values, and the accessible description.

WHEN YOU NEED TO CHANGE IT

- You use a canvas-based charting library, or need to assert on a different visual property.

HOW TO CHANGE IT

- 1 For SVG charts, configure the selectors. For canvas-based charts, use the `Canvas` component plus a visual baseline instead.

```
ui.chart('.highcharts-container', {
  dataPointSelector: '.highcharts-point',
  legendSelector: '.highcharts-legend',
  tooltipSelector: '.highcharts-tooltip',
});
```

WHY THE CHANGE BELONGS HERE

Hovering a mark and reading its tooltip is usually the only way to verify a data value, because the rendered chart itself carries no accessible text.

Related: `src/components/media/canvas.ts`

API SURFACE (2 EXPORTS)

INTERFACE `ChartOptions`

```
seriesSelector?: string;
dataPointSelector?: string;
legendSelector?: string;
tooltipSelector?: string;
axisLabelSelector?: string;
```

CLASS `Chart` extends `BaseComponent`

SVG chart (D3, Highcharts, Chart.js in SVG mode, Recharts). Charts have no text to assert on, so the useful checks are: the right number of marks, the legend, the axis labels, and the tooltip a hover reveals.

`get dataPoints: Locator`

`dataPointCount(): Promise<number>`

`seriesCount(): Promise<number>`

`hoverDataPoint(index: number): Promise<string>`

Hovers a mark and returns the tooltip text it reveals.

`clickDataPoint(index: number): Promise<void>`

`getLegendItems(): Promise<string[]>`

`toggleSeries(name: string): Promise<void>`

Clicking a legend entry usually toggles that series' visibility.

`getAxisLabels(): Promise<string[]>`

`getDataValues(attribute = 'data-value'): Promise<string[]>`

Values encoded in data attributes, when the library provides them.

`getBarHeights(): Promise<number[]>`

Bar heights as a proxy for relative values — for ordering assertions.

```
waitForRender(timeout = TIMEOUTS.LONG):
```

```
Promise<void>
```

```
getAccessibleDescription(): Promise<string>
```

Charts should expose a title/description for non-sighted users.

src/components/advanced/drag-drop.ts

128 lines · 2 exports

Drag and drop with three strategies deliberately, because no single approach works across every library: Playwright's built-in `dragTo` for standard HTML5 drag, `dragManually()` with explicit mouse steps for react-dnd, dnd-kit and Sortable, and `dragWithDataTransfer()` dispatching synthetic events for custom handlers. Also `dragBy()` for reordering and `dragWithKeyboard()` for the accessible path. `DropZone` covers the target: active state, dropped items and membership.

ESCALATION ORDER WHEN A DRAG DOES NOT WORK

TypeScript

```
await item.dragTo(zone); // 1. built-in
await item.dragManually(zone, { steps: 25, holdMs: 150 }); // 2. explicit mouse
await item.dragWithDataTransfer(zone); // 3. synthetic events
```

WHEN YOU NEED TO CHANGE IT

- A library needs a different gesture — a longer hold, more intermediate moves, a specific drop position.

HOW TO CHANGE IT

- 1 Tune `dragManually` options first; add a new strategy method only if a library genuinely needs one.

WHY THE CHANGE BELONGS HERE

Discovering mid-sprint that your DnD library ignores synthetic events is expensive. Shipping all three strategies turns that discovery into a one-line change.

WATCH OUT FOR

- `dragManually` deliberately makes a small initial move — that is how most libraries detect that a drag has started.

Related: `src/components/form/file-upload.ts`

API SURFACE (2 EXPORTS)

CLASS Draggable extends `BaseComponent`

Draggable element. Provides three strategies because no single one works everywhere: - `dragTo` — Playwright's built-in; works for most HTML5 DnD. - `dragManually` — explicit mouse steps; needed by react-dnd, dnd-kit, Sortable. - `dragWithDataTransfer` — synthetic events; last resort for custom handlers.

`isDraggable(): Promise<boolean>`

`dragManually(target: BaseComponent | Locator, options: DragOptions = {}): Promise<void>`

Mouse-driven drag with explicit steps — the most broadly compatible path.

`dragWithDataTransfer(target: BaseComponent | Locator): Promise<void>`

Dispatches a full HTML5 drag sequence with a shared DataTransfer.

`dragBy(deltaX: number, deltaY: number, steps = 20): Promise<void>`

Moves the element by a pixel offset — for reordering and canvas layouts.

`dragWithKeyboard(direction: 'up' | 'down' | 'left' | 'right', times = 1): Promise<void>`

Keyboard-accessible drag: Space to lift, arrows to move, Space to drop.

CLASS DropZone extends `BaseComponent`

A drop zone: where draggables land.

`isActive(): Promise<boolean>`

True while the zone is highlighted as a valid target.

`getDroppedItems(itemSelector = '.item, li, [draggable]): Promise<string[]>`

`itemCount(itemSelector = '.item, li, [draggable]): Promise<number>`

`containsItem(text: string, itemSelector = '.item, li, [draggable]): Promise<boolean>`

src/components/advanced/infinite-scroll.ts

143 lines · 2 exports

Infinite-scroll and lazy-loaded lists. The hard part is knowing when loading has stopped, so this waits for the item count to stabilise rather than trusting a spinner that may never render. Supports loading one more page, loading everything up to a cap, scrolling until a specific item appears, and end-of-list detection.

WHEN YOU NEED TO CHANGE IT

- Your feed scrolls the window rather than a container, or uses a different end marker.

HOW TO CHANGE IT

- 1 Set `scrollWindow: true` and the marker selectors at construction; always keep a scroll cap.

```
const feed = ui.infiniteScroll('#feed', { scrollWindow: true, endMarkerSelector: '.no-more' });
const total = await feed.loadAll(30);
```

WHY THE CHANGE BELONGS HERE

The scroll cap is a safety net: an unbounded feed would otherwise scroll forever and fail as a timeout with no useful message.

Related: `src/components/data/data-grid.ts`

API SURFACE (2 EXPORTS)

INTERFACE `InfiniteScrollOptions`

```
itemSelector?: string;
loadingSelector?: string;
endMarkerSelector?: string;
scrollWindow?: boolean;
```

CLASS `InfiniteScroll` extends `BaseComponent`

Infinite-scroll / lazy-loaded list. The hard part is knowing when loading has stopped: this waits for the item count to stabilise rather than trusting a spinner that may never render.

`get items: Locator`

`itemCount(): Promise<number>`

`loadMore(): Promise<number>`

Scrolls once and waits for whatever it loaded. Returns the new count.

`loadAll(maxScrolls = 25): Promise<number>`

Scrolls until no new items arrive, the end marker appears, or the cap is reached. The cap is a safety net: never scroll an unbounded feed forever.

`scrollToItem(text: string, maxScrolls = 25): Promise<Locator>`

Scrolls until an item with the given text appears.

`hasReachedEnd(): Promise<boolean>`

`isLoading(): Promise<boolean>`

`scrollToBottom(): Promise<void>`

`scrollToTop(): Promise<void>`

src/components/advanced/rich-text-editor.ts

157 lines · 2 exports

WYSIWYG editors — Quill, TinyMCE, CKEditor, ProseMirror, Slate. Typing, replacing content, reading text **and** generated HTML, selecting a substring through the Range API, toolbar actions, keyboard formatting shortcuts, formatting assertions, word count and empty checks. Editors rendered inside an iframe are supported through the `iframeSelector` option.

WHEN YOU NEED TO CHANGE IT

- You switch editors, or need a formatting operation not covered.

HOW TO CHANGE IT

- 1 Configure the editable body, toolbar and iframe at construction.

```
ui.richText('.tox-tinymce', {
  iframeSelector: 'iframe.tox-edit-area__iframe',
  editableSelector: 'body#tinymce',
  toolbarSelector: '.tox-toolbar__primary',
});
```

WHY THE CHANGE BELONGS HERE

Asserting on rendered HTML rather than plain text is what makes these tests meaningful — plain text hides every formatting bug the editor could have.

Related: `src/components/media/frame.ts`

API SURFACE (2 EXPORTS)**INTERFACE RichTextEditorOptions**

```
editableSelector?: string;
toolbarSelector?: string;
iframeSelector?: string;
```

CLASS RichTextEditor extends BaseComponent

WYSIWYG editor (Quill, TinyMCE, CKEditor, ProseMirror, Slate). Asserting on rendered HTML is what makes these tests meaningful — plain text hides every formatting bug.

```
typeText(text: string): Promise<void>
```

```
setContent(text: string): Promise<void>
```

```
clear(): Promise<void>
```

```
getText(): Promise<string>
```

`getHtml(): Promise<string>`

The generated markup — where formatting assertions actually live.

`selectAll(): Promise<void>`

Selects all text so a toolbar action applies to the whole document.

`selectText(text: string): Promise<void>`

Selects a specific substring using the browser's Range API.

`clickToolbarButton(name: string): Promise<void>`

Applies a toolbar action to the current selection.

`bold(): Promise<void>``italic(): Promise<void>``underline(): Promise<void>``hasFormatting(tag: 'b' | 'strong' | 'i' | 'em' | 'u'): Promise<boolean>`

True when the selection is wrapped in the given tag.

`getWordCount(): Promise<number>``isEmpty(): Promise<boolean>`

`src/components/advanced/shadow-dom.ts`

74 lines · 1 exports

Web components with a shadow root. Playwright pierces **open** shadow roots automatically, so plain locators usually work; this class exists for the parts that do not — closed roots, slotted content, and inspecting the boundary itself. Also a static helper that lists every custom element on the page.

WHEN YOU NEED TO CHANGE IT

- You meet a closed shadow root, or need to assert on slotted light-DOM content.

HOW TO CHANGE IT

- Use `inShadow()` for ordinary cases and the evaluate-based helpers when the locator engine cannot reach inside.

```
const picker = ui.shadowHost('my-date-picker');
expect(await picker.getShadowMode()).toBe('open');
await picker.clickInShadow('.today');
```

WHY THE CHANGE BELONGS HERE

Most shadow-DOM advice on the internet is obsolete because Playwright handles the common case; documenting that here stops teams writing unnecessary workarounds.

Related: `src/core/locator.factory.ts`

API SURFACE (1 EXPORT)

CLASS **ShadowHost** extends `BaseComponent`

Web component with a shadow root. Playwright's engine pierces *open* shadow roots automatically, so plain locators usually work. This class exists for the parts that do not: closed roots, slotted content, and inspecting the boundary itself.

`inShadow(selector: string): Locator`

Element inside the shadow tree. Open roots are pierced automatically.

`hasShadowRoot(): Promise<boolean>``getShadowMode(): Promise<'open' | 'closed' | 'none'>``getShadowText(selector: string): Promise<string>`

Text of an element inside the shadow root, read through the DOM API.

`clickInShadow(selector: string): Promise<void>`

Clicks inside the shadow root via the DOM when the locator engine cannot.

`getSlottedContent(slotName?: string): Promise<string[]>`

Light-DOM nodes projected into a `<slot>`.

`listCustomElements(page: Page): Promise<string[]>`

Every custom element on the page — useful for discovery and audits.

Reference: fixtures, hooks, pages & API

How the framework reaches your tests

Fixtures (src/fixtures)

src/fixtures/auth.fixture.ts

71 lines · 1 exports

Extends the base fixtures with authenticated browsing: `authenticatedContext` and `authenticatedPage` for the standard user, and `pageAs(role)` for signing in as any role on demand. Sessions are replayed from the storage state files written by `auth.setup.ts`.

TWO USERS IN ONE TEST

TypeScript

```
test('admin sees the audit trail a viewer cannot', async ({ pageAs }) => {
  const adminPage = await pageAs('admin');
  const viewerPage = await pageAs('readonly');
  // both contexts are independent and already signed in
});
```

WHEN YOU NEED TO CHANGE IT

- You add a role that tests need to browse as.
- Your application needs more than cookies to restore a session (a tenant header, a local-storage token).

HOW TO CHANGE IT

- 1 Add the role to `UserRole`, add its credentials, add it to the `ROLES` array in `auth.setup.ts` — then `pageAs('auditor')` works with no change here.
- 2 If the session needs extra state, set it on the context after creation.

```
await context.addInitScript(() => window.localStorage.setItem('tenant', 'acme'));
```

WHY THE CHANGE BELONGS HERE

Replaying a saved session removes a login from every test. On a suite of 300 tests at 8 seconds per login, that is roughly 40 minutes of wall-clock time per run.

WATCH OUT FOR

- `storageStateFor()` returns `undefined` when the file does not exist, so a suite that does not use auth still runs — the tests simply start signed out.

Related: `src/hooks/auth.setup.ts` · `src/config/env.config.ts`

API SURFACE (1 EXPORT)

CONST `test`

Authentication via saved storage state. Logging in through the UI once per role (in `auth.setup.ts`) and replaying the cookies afterwards removes a login from every single test — usually the largest single saving available in a UI suite.

`src/fixtures/bank.fixture.ts`

97 lines · 1 exports

Provides every page object ready-made, plus `signedIn` and `signInAs` for tests that need an authenticated session.

Why there is no cleanup

RULE SecureBank keeps all its state in `localStorage` under `bank-app-v4`, and Playwright gives every test a **fresh browser context**. Every test therefore starts from the seeded dataset automatically: no cleanup step, no shared account to fight over, no ordering constraints. That is what lets this suite run fully parallel against a single shared demo application — normally the hardest problem in UI automation.

Why sign-in is a fixture rather than a `beforeEach`

A fixture is constructed only when a test names it, so the login specs — which must start signed *out* — simply do not ask for `signedIn` and pay nothing. A `beforeEach` would run for them too and would have to be skipped around.

THREE LEVELS OF ACCESS

TypeScript

```
test('...', async ({ bank }) => { ... }); // page objects, signed out
test('...', async ({ signedIn }) => { ... }); // signed in as the standard persona
test('...', async ({ signInAs }) => { // signed in as anyone
  const app = await signInAs('frozen');
});
```

The layering is `base → auth → bank`, each adding fixtures without the layer below knowing about it — so a project that deletes the bank suite loses nothing else.

WHEN YOU NEED TO CHANGE IT

- You add a page object.
- A test needs a differently-prepared session.

HOW TO CHANGE IT

- 1
- Add the page to `BankPages` and construct it in the `bank` fixture — two lines, and every test can reach it.

WHY THE CHANGE BELONGS HERE

Constructing eleven page objects in every spec would be eleven lines of noise per file. Doing it once, lazily, is what keeps a spec about the behaviour it is testing.

WATCH OUT FOR

- Page objects only build locators, so constructing all eleven is essentially free — no page is loaded until a test navigates.

Related: `src/pages/bank/index.ts` · `src/fixtures/index.ts` · `src/data/personas.ts`

API SURFACE (1 EXPORT)

|

`CONST` `test`

`src/fixtures/base.fixture.ts`

153 lines · 1 exports

Composes the `test` object every spec uses. It adds the framework's fixtures — component factory, logger, network control, console capture, generated data, credentials, API client — and overrides the built-in `page` fixture to attach diagnostics on failure and enforce the console-error policy.

FIXTURE	SCOPE	PROVIDES
<code>ui</code>	test	Component factory bound to the page.
<code>log</code>	test	Logger scoped to the test title; logs start and finish with status and duration.
<code>network</code>	test	Mocking, latency, abort, offline and request capture. Routes are reset after the test.

FIXTURE	SCOPE	PROVIDES
<code>consoleErrors</code>	test	Console errors, page errors and failed requests recorded during the test.
<code>testData</code>	test	A generated user, seeded from the test title so retries reproduce the same data.
<code>userFor</code>	test	<code>getUser(role)</code> for credentials.
<code>appConfig</code>	test	The validated configuration object.
<code>failOnConsoleErrors</code>	test (option)	Opt-in policy that fails a test if the page logged console errors.
<code>api</code>	worker	One HTTP client per worker, disposed at the end.

ENABLING THE CONSOLE-ERROR POLICY FOR A FILE OR A TEST

TypeScript

```
test.use({ failOnConsoleErrors: true });

test('checkout is clean @smoke', async ({ page, consoleErrors }) => {
  // ... the test fails at the end if the page logged an error
});
```

WHEN YOU NEED TO CHANGE IT

- Every test needs something new — a seeded tenant, a feature-flag override, a page object bundle.
- You want different diagnostics attached on failure.
- A resource is expensive and should be created once per worker instead of per test.

HOW TO CHANGE IT

- 1 Add the fixture's type to `TestFixtures` (or `WorkerFixtures`), then implement it. Code before `use()` is setup; code after is teardown.

```
export interface TestFixtures {  
  // ...  
  tenant: Tenant;  
}  
  
export const test = base.extend<TestFixtures, WorkerFixtures>({  
  tenant: async ({ api }, use) => {  
    const tenant = await api.post<Tenant>('/tenants', { name: generateUser(faker).username });  
    await use(tenant);  
    await api.delete(`/tenants/${tenant.id}`); // always clean up  
  },  
});
```

- 2 To change failure diagnostics, edit `attachDiagnostics`. Keep each attachment guarded with `.catch()` so a diagnostic failure never masks the real one.
- 3 Use worker scope for anything expensive and stateless; use test scope for anything a test could mutate.

```
api: [async ({}, use) => { /* ... */ }, { scope: 'worker' }],
```

WHY THE CHANGE BELONGS HERE

Fixtures are how the framework reaches tests without imports or inheritance. Adding a capability here makes it available everywhere, with automatic teardown and no boilerplate in specs.

WATCH OUT FOR

- `consoleErrors` deliberately does not depend on `page`; the overridden `page` attaches the listeners. A mutual dependency would be a fixture cycle and Playwright would refuse to run.
- `testData` is seeded from the test title, so renaming a test changes its generated data.

Related: `src/fixtures/auth.fixture.ts` · `src/fixtures/index.ts` · `src/utils/browser.utils.ts`

API SURFACE (1 EXPORT)

`CONST test`

The framework's test object. Tests import `test` and `expect` from `@fixtures/index` — never from `@playwright/test` directly — so every test automatically gets logging, console-error capture, network control and the component factory.

src/fixtures/custom-matchers.ts

139 lines · 1 exports

Domain-specific matchers that extend Playwright's `expect`: `toBeAccessible`, `toBeInteractive`, `toHaveNormalizedText`, `toHaveValidationError`, `toBeInViewport` and `toBeSorted`. Each exists because the equivalent inline check appeared often enough to deserve a name — and because a named matcher produces a far better failure message.

USAGE

TypeScript

```
await expect(page).toBeAccessible({ tags: ['wcag2aa'] });
await expect(saveButton.locator).toBeInteractive();
await expect(total).toHaveNormalizedText('Total due'); // whitespace-insensitive
await expect(emailField.locator).toHaveValidationError(/valid email/i);
expect(await table.getColumnValues('Amount')).toBeSorted('desc');
```

WHEN YOU NEED TO CHANGE IT

- The same multi-step assertion appears in three or more tests.

HOW TO CHANGE IT

- 1 Add the matcher to the `expect.extend` object. Return `pass` plus a `message` function that explains both directions — the negated form is what makes a matcher pleasant to debug.

```
async toHaveRowCount(table: Table, expected: number) {
  const actual = await table.rowCount();
  return {
    pass: actual === expected,
    message: () =>
      actual === expected
        ? `Expected the table not to have ${expected} rows`
        : `Expected ${expected} rows but found ${actual}`,
    name: 'toHaveRowCount',
  };
}
```

- 2 TypeScript picks up the new matcher automatically because `expect` is exported from here; no declaration merging is needed.

WHY THE CHANGE BELONGS HERE

A failure that says "expected ascending order: actual 30, 10 / expected 10, 30" is diagnosable from the CI log alone. `expect(true).toBe(false)` is not.

Related: `src/fixtures/index.ts` · `src/utils/ally.utils.ts`

API SURFACE (1 EXPORT)

CONST **expect**

Domain-specific assertions. Each one exists because the equivalent inline check appeared in enough tests to be worth naming — and because a named matcher produces a far better failure message than `expect(await x.y()).toBe(true)`.

src/fixtures/index.ts

17 lines · 2 exports

Re-exports the composed `test` and the extended `expect`. This is the single import surface for specs, and importing from here rather than from `@playwright/test` is what guarantees a test gets the framework's fixtures and matchers.

RULE Tests import `test` and `expect` from `@fixtures/index` — never from `@playwright/test` directly.

EVERY SPEC STARTS THIS WAY

TypeScript

```
import { test, expect } from '@fixtures/index';
```

WHEN YOU NEED TO CHANGE IT

- You add a fixture module that tests should reach.

HOW TO CHANGE IT

- 1 Extend the chain in `auth.fixture.ts` (which already extends `base.fixture.ts`) and re-export here, so tests keep one import line.

WHY THE CHANGE BELONGS HERE

One import path means new capabilities reach every existing test with no edits to those tests.

Related: `src/fixtures/base.fixture.ts` · `src/fixtures/auth.fixture.ts`

API SURFACE (2 EXPORTS)

RE-EXPORT **test** from `./bank.fixture'`

RE-EXPORT **expect** from `./custom-matchers'`

Hooks (src/hooks)

src/hooks/auth.setup.ts

69 lines

Runs as the `setup` project before every other project. For each role it signs in through the UI once and saves the session to `storage/<role>.json`. Its `signIn` function contains the only application-specific selectors in the framework.

CAREFUL This is the first file to edit when you point the framework at a real application. Its placeholder selectors will not match your login page.

WHAT TO REPLACE

TypeScript

```
async function signIn(page: Page, username: string, password: string): Promise<void> {
  await page.goto('/login', { waitUntil: 'domcontentloaded' });
  await page.getByLabel(/username|email/i).fill(username);
  await page.getByLabel(/password/i).fill(password);
  await page.getByRole('button', { name: /sign in|log in/i }).click();

  // Prove the session is real before saving it.
  await page.waitForURL((url) => !url.pathname.includes('/login'), {
    timeout: config.timeouts.navigation,
  });
}
```

WHEN YOU NEED TO CHANGE IT

- Your login page differs from the placeholder (it will).
- You add a role that needs a saved session.
- Login involves an extra step — SSO, MFA, a tenant chooser, a consent screen.

HOW TO CHANGE IT

- 1 Replace the selectors in `signIn`, or delegate to a `LoginPage` page object once you have one.

```
const login = new LoginPage(page);
await login.goto();
await login.signIn(username, password);
```

- 2 Add the role to the `ROLES` array; a storage state is then produced for it automatically.

```
const ROLES: UserRole[] = ['standard', 'admin', 'auditor'];
```

- 3 For MFA, prefer a test account with MFA disabled or a deterministic TOTP seed generated in code — never a manual step.

WHY THE CHANGE BELONGS HERE

Verifying the redirect before saving the state is what turns one login failure into one clear setup failure, instead of an entire suite failing later with unexplained authorisation errors.

WATCH OUT FOR

- Storage states expire when your session tokens do. If tests start failing as unauthenticated, delete `storage/*.json` and re-run.
- Never commit `storage/*.json` — those files are live sessions.

Related: `src/fixtures/auth.fixture.ts` · `src/config/env.config.ts` · `.gitignore`

`src/hooks/global.setup.ts`

42 lines

Runs once before the entire suite, in the parent process: creates the report, results, storage and download directories, removes the previous HTML report so a stale one cannot be mistaken for the current run, logs the resolved run configuration, and records the start time.

WHEN YOU NEED TO CHANGE IT

- The suite needs one-time preparation — a database reset, a feature-flag snapshot, a health check on the target environment.

HOW TO CHANGE IT

- 1 Keep it cheap and idempotent; it blocks every worker. Anything per-role belongs in `auth.setup.ts`, anything per-test in a fixture.

```
const health = await fetch(`${config.baseURL}/health`);
if (!health.ok) {
  throw new Error(`${config.env} is not healthy (${health.status}) - aborting before was
}
```

WHY THE CHANGE BELONGS HERE

Failing here costs seconds and produces one clear message. Discovering the same problem through 300 timing-out tests costs an hour and produces noise.

WATCH OUT FOR

- This runs in the parent process, so variables set here are not shared with workers except through `process.env`.

Related: `src/hooks/global.teardown.ts` · `playwright.config.ts`

`src/hooks/global.teardown.ts`

24 lines

Runs once after the entire suite: removes the temporary scratch directory and logs total duration and where reports were written. Artifacts — traces, videos, reports — are deliberately left in place.

WHEN YOU NEED TO CHANGE IT

- The suite creates shared external state that must be cleaned up, or you want to publish results somewhere.

HOW TO CHANGE IT

- 1 Add the cleanup, and guard it so a teardown failure does not mask the real test results.

```
try {
  await archiveReports();
} catch (error) {
  logger.warn('Report archiving failed', { error: String(error) });
}
```

WHY THE CHANGE BELONGS HERE

Per-test data should be cleaned by the fixture that created it; teardown is only for suite-wide state, so it stays small and reliable.

Related: `src/hooks/global.setup.ts`

Page objects (src/pages)

`src/pages/bank/account-detail.page.ts`

80 lines · 1 exports

A single account: its balance and a filterable, sortable transaction table.

NOTE Its URL carries an account id, so `goto()` is not used — the page is reached by clicking through from the accounts list, which is how a user reaches it and therefore what should be tested.

WHEN YOU NEED TO CHANGE IT

- The detail page gains a filter.

HOW TO CHANGE IT

- 1
- Add the component and a method that waits for the list to settle.

WHY THE CHANGE BELONGS HERE

The detail balance and the list balance are two renderings of one number — the place a stale cache shows up, and the reason this page object exists rather than being folded into the list.

Related: `src/pages/bank/accounts.page.ts` · `tests/ui/accounts.spec.ts`

API SURFACE (1 EXPORT)

CLASS `AccountDetailPage` extends `BasePage`

A single account's page: balance plus a filterable, sortable transaction table. Its `path` carries an account id, so `goto()` is not used — the page is reached by clicking through from the accounts list, which is how a user reaches it and therefore what should be tested.

`shell: BankShell`

`backLink`

`accountName`

`accountTypeBadge`

`accountBalance`

`searchInput`

`dateFromInput`

`dateToInput`

`typeFilter`

`transactionsTable`

`balance(): Promise<number>`

`search(term: string): Promise<void>`

Types into the search field and waits for the debounce to settle.

```
descriptions(): Promise<string[]>
```

Descriptions of every visible transaction row.

src/pages/bank/accounts.page.ts

173 lines · 1 exports

The account list and the add/edit dialog it opens.

One component models **both** the add and edit dialogs, because the application reuses the same form and changes only the container test id and the title. Two nearly identical members would have to be kept in step for no benefit.

Every field in the dialog is required

Established by submitting an incomplete form and reading what came back — first "Please select an account type", then "Please enter a valid starting balance". A page object that filled only the name failed with a dialog that never closed, and the cause was not obvious.

CAREFUL The starting-balance field carries **no** `data-testid` — only a generated id (`base-ui-r_a_`) that changes between renders and must never be selected on. It is located by placeholder, scoped to the dialog: a placeholder is user-visible text, which makes it a more meaningful anchor than a generated id.

The terms checkbox is a `span[role="checkbox"]`, not a native input. The Checkbox component already handles that — `check()` falls back to a click when Playwright's strict `check()` refuses a non-input element.

WHEN YOU NEED TO CHANGE IT

- The account form gains a field.

HOW TO CHANGE IT

- 1 Add the component and fill it in `addAccount()`. Check whether it is required by submitting without it.

WHY THE CHANGE BELONGS HERE

Discovering required fields by reading the application's own validation messages is faster and more reliable than reading its markup.

Related: `tests/ui/accounts.spec.ts` · `src/pages/bank/account-detail.page.ts`

API SURFACE (1 EXPORT)

CLASS **AccountsPage** extends `BasePage`

The account list, plus the add/edit dialog it opens.

`shell: BankShell``pageTitle``addAccountButton``accountsTable``accountDialog`

The add/edit dialog. One component for both, because the application reuses the same form and only changes the container's test id and the title. Two nearly identical page-object members would have to be kept in step for no benefit.

`accountNameInput``accountTypeDropdown``startingBalanceInput``acceptTermsCheckbox``saveAccountButton``cancelAccountButton``accountNames(): Promise<string[]>`

Account names in display order.

`accounts(): Promise<{ name: string; balance: number }[]>`

Every account's name and balance, parsed.

`totalBalance(): Promise<number>`

Sum of every account balance — the figure the dashboard should agree with.

`openAccount(name: string): Promise<void>`

Opens the detail page for an account by its displayed name.

`editAccount(name: string): Promise<void>`

Opens the edit dialog for an account by its displayed name.

`addAccount(input: { name: string; type?: string; startingBalance?: string; }): Promise<void>`

Creates an account through the dialog. Every field here is **required**, which was established by submitting an incomplete form and reading the messages the application returned — first "Please select an account type", then "Please enter a valid starting balance". A page object that only filled the name would fail with a dialog that never closes, and the cause would not be obvious. The terms checkbox exists only on **create**, not on edit, so it is checked conditionally — an unconditional `check()` would break the edit flow.

`src/pages/bank/apply-loan.page.ts`

234 lines · 1 exports

Loan applications and their history — including the page that carries the application's planted defect.

Why `allHistoryRows()` walks the pages

The history paginates at five rows, and the total beneath it is labelled **"Total (Active/Pending)"** — it covers all loans but counts only the ones that are not closed. Comparing the total against the visible page is therefore wrong twice over: it misses later pages and it includes closed loans.

NOTE An earlier version of the loan test did exactly that and reported a mismatch for a perfectly healthy account. Walking the pages and filtering by status is what makes the comparison mean anything — and it is what allows the planted defect to be demonstrated credibly.

TypeScript

```
async expectedActiveTotal(): Promise<number> {  
  const rows = await this.allHistoryRows();  
  return rows.filter((row) => !/closed/i.test(row.status))  
    .reduce((sum, row) => sum + row.amount, 0);  
}
```

WHEN YOU NEED TO CHANGE IT

- The loan form changes.
- The history gains a column.

HOW TO CHANGE IT

- 1 Update the components. If the total's definition changes, `expectedActiveTotal()` is the one place to change.

WHY THE CHANGE BELONGS HERE

The defect this page carries is the suite's most valuable result, and it is only credible because the comparison it rests on is provably correct for a healthy account.

WATCH OUT FOR

- `resetData()` is for a human debugging in headed mode. Tests never need it — a fresh context already reseeds.

Related: `tests/ui/apply-loan.spec.ts`

API SURFACE (1 EXPORT)

CLASS `ApplyLoanPage` extends `BasePage`

Loan applications and their history. This page carries the application's **planted defect**: for `error_user`, the history total omits the most recently added loan. That is deliberate on the application's part, and the suite reproduces it rather than working around it — a test framework that cannot demonstrate finding a real bug has not been demonstrated at all.

`shell`: `BankShell`

`openApplyLoanButton`

`resetDataButton`

`loanDialog`

`loanTypeDropdown`

`loanAmountInput`

`loanTermDropdown`

`interestRateInput`

`loanAccountDropdown`

`reviewLoanButton`

`cancelLoanButton`

`historyTable`

`historySearch`

`historyTypeFilter`

`historyDateFrom`

`historyDateTo`

`historyTotal`

`historyPagination`

`historyPaginationInfo`

`historyNextButton`

`historyPreviousButton`

<code>openApplication(): Promise<void></code>	Opens the application dialog.
<code>fillApplication(input: { type?: string; amount?: string; term?: string; account?: string; }): Promise<void></code>	Fills the dialog without submitting, so validation can be asserted.
<code>historyAmounts(): Promise<number[]></code>	The amounts on the CURRENT page of history, parsed.


```
allHistoryRows(): Promise<{ amount: number;
status: string }[]>
```

Every loan across every page, with its status. The history paginates at five rows, and the total beneath it is labelled "**Total (Active/Pending)**" — it covers all loans but counts only the ones that are not closed. So comparing the total against the visible page is wrong twice over: it misses later pages and it includes closed loans. An earlier version of the loan test did exactly that and reported a mismatch for a perfectly healthy account. Walking the pages and filtering by status is what makes the comparison mean something.

```
expectedActiveTotal(): Promise<number>
```

The figure the displayed total is supposed to equal: every loan that is not closed, summed across all pages.

```
displayedTotal(): Promise<number>
```

The total the page displays beneath the history table.

```
historyRowCount(): Promise<number>
```

How many loan rows are on the current page.

```
resetData(): Promise<void>
```

Restores the seeded dataset. Not needed for isolation — every test gets a fresh browser context and the application stores its state in `localStorage`, so state never leaks between tests. It is here for the human who has been clicking around in a headed debugging session and wants a clean slate.

`src/pages/bank/bank-shell.ts`

189 lines · 2 exports

The application chrome — top bar and sidebar — modelled once as a component and composed into every page object.

A **component, not a page object**: it has no URL of its own and appears on ten pages. Modelling it once means a navigation change is one edit rather than ten, and keeps each page object about that page.

The theme toggle has no test id

It is located by accessible name instead. That is not a workaround — an accessible-name locator asserts something a test id cannot: that the control is announced correctly to a screen reader. If the label were removed, this locator would fail, and it should.

Two bugs this file records

THE STATE MUST BE READ BEFORE THE CLICK

TypeScript

```
const wasDark = await this.isDarkMode();
await this.themeToggle.click();
await this.page.waitForFunction(
  (previous) => document.documentElement.classList.contains('dark') !== previous,
  wasDark,
);
```

An earlier version read the mode *after* clicking and passed the already-changed value into the wait, so the condition could never become true. Cheap mistake, expensive symptom: it looked like the application failed to switch theme.

CAREFUL Opening the mobile drawer renders a **second** `[data-testid="bank-sidebar"]`, so an unscoped lookup fails strict mode and `.first()` silently picks the hidden desktop copy — an assertion that can never pass. `mobileNavLink()` scopes to the overlay, which is the only correct answer.

WHEN YOU NEED TO CHANGE IT

- The navigation gains a destination.
- The chrome changes.

HOW TO CHANGE IT

- 1 Add the slug to `NavDestination`. The compiler then lists every place that must handle it, and `navLink()` works without further change.

WHY THE CHANGE BELONGS HERE

Anything on every page belongs in one component. Re-asserting the sidebar in ten specs would be ten places to update and ten chances to disagree.

WATCH OUT FOR

- The unread badge is **removed** from the DOM at zero, so `unreadCount()` reads absence as zero rather than waiting for the text "0".

Related: `tests/ui/navigation.spec.ts` · `src/pages/bank/index.ts`

API SURFACE (2 EXPORTS)

CLASS `BankShell`

The application chrome that surrounds every signed-in page: the top bar and the left sidebar. It is a **component, not a page object**, because it has no URL of its own and appears on ten different

pages. Modelling it once and composing it into each page means a change to the navigation is one edit here, rather than ten — and it keeps each page object about that page. Every page object exposes it as `shell`, so a test navigates the same way regardless of where it happens to be.

`brandLink`

`logoutButton`

`notificationsLink`

`topbarNotificationBadge`

`mobileMenuButton`

`sidebar`

`sidebarNotificationBadge`

`userInfo`

`navLink(destination: NavDestination): Link`

The sidebar destinations, keyed by the slug the application uses. A record rather than a method per link: the pattern is completely regular, and eleven near-identical methods would be eleven places to update.

`navigateTo(destination: NavDestination): Promise<void>`

Clicks a sidebar destination and waits for the URL to settle.

`get themeToggle: Button`

The theme toggle. This is the one control in the application with **no `data-testid`**, so it is located by its accessible name instead. That is not a workaround: an accessible-name locator asserts something a test id cannot — that the control is announced correctly to a screen reader. If the label were ever removed, this locator would fail, and it should.

`isDarkMode(): Promise<boolean>`

True when the application is currently rendering its dark theme.

`toggleTheme(): Promise<boolean>`

Flips the theme and returns the mode that is now active.

`unreadCount(): Promise<number>`

The unread-notification count, or 0 when the badge is absent entirely.

`logout(): Promise<void>`

Signs out and waits for the login page.

`get mobileDrawer: Locator`

The navigation drawer shown on narrow viewports. Opening it renders a **second `[data-testid="bank-sidebar"]`** inside an overlay, so the desktop sidebar and the drawer both match that selector and Playwright's strict mode rejects an unscoped lookup. Scoping to the overlay is the fix; `.first()` would silently pick the hidden desktop copy and produce a "visible" assertion that can never pass.

`mobileNavLink(destination: NavDestination): Locator`

A navigation link inside the mobile drawer specifically.

`openMobileMenu(): Promise<void>`

Opens the mobile drawer and waits for it to be usable.

`get skipLink: Locator`

The "skip to content" link. Visually hidden until focused — the standard pattern. It is exposed here because its behaviour is asserted by the accessibility suite.

`signedInAs(): Promise<string>`

The username shown in the sidebar footer.

TYPE `NavDestination`

Every sidebar destination, as the application slugs them.

```
| 'dashboard' | 'accounts' | 'transfer' | 'send-money' | 'bill-pay' | 'transactions' | 'app
```

`src/pages/bank/bill-pay.page.ts`

134 lines · 1 exports

Bill payment: a biller search, an amount, a date and a memo.

Two component choices worth explaining

The biller field is an **Autocomplete**, not a Dropdown: typing filters a list. The payment date is modelled as a **TextInput**, not a DatePicker — the DatePicker component drives a *custom* calendar by clicking through months, which this control does not have. A native date input is set by writing its `yyyy-mm-dd` value, and TextInput is the component that does that.

RULE Using the fancier component because of its name would be reaching for the wrong tool. Match the component to the markup, not to the label on the field.

CAREFUL The add-biller dialog has a second required field — the account/reference number — with **no test id and no associated `<label>`**, verified against the running application. It is located by placeholder, which is the only user-visible anchor available. The accessibility suite records the underlying labelling gap.

WHEN YOU NEED TO CHANGE IT

- The biller dialog changes.

HOW TO CHANGE IT

- 1 Update the components. Confirm which fields are required by submitting without them.

WHY THE CHANGE BELONGS HERE

A dialog that never closes is the symptom of an unfilled required field, and it is worth writing down which they are so the next person does not rediscover it.

Related: `tests/ui/money-movement.spec.ts`

API SURFACE (1 EXPORT)

CLASS `BillPayPage` extends `BasePage`

Bill payment: pick a biller, an amount and a date.

`shell`: `BankShell`

`fromAccount`

`addBillerButton`

`billerSearch`

The biller field is a **search box**, not a dropdown — typing filters a list of billers. Modelled as an Autocomplete so the type-filter-select sequence comes from the component rather than being rebuilt per test.

`amountInput`

`paymentDateInput`

`memoInput`

`reviewButton`

`cancelButton`

`addBillerDialog`

`billerNameInput`

`billerReferenceInput`

The account/reference number. Carries **no data-testid** and — verified against the running application — **no associated <label>** either, so `getByLabel` cannot find it. That is an accessibility gap in the application: a screen reader announces an unlabelled text box. Located by placeholder, which is the only user-visible anchor available. Placeholder text is a weaker locator than a label (it is a hint, not a name, and it disappears on input) — using it here is a compromise forced by the markup, and the accessibility suite records the underlying gap.

`saveBillerButton`

`cancelBillerButton`

```
addBiller(name: string, reference =
`REF-${Date.now().toString().slice(-8)}`):
Promise<void>
```

Adds a biller through the dialog. Both fields are required. An earlier version filled only the name and the dialog simply never closed — the application was rejecting the submission and the test was waiting for a close that would never come.

```
futureDate(days: number): string
```

A date `days` in the future, in the `yyyy-mm-dd` a date input requires.

src/pages/bank/dashboard.page.ts

111 lines · 2 exports

The landing page: net-worth cards, quick-action tiles and the recent-activity widget. Also exports `parseCurrency`, which six page objects share.

CURRENCY PARSING LIVES IN ONE PLACE

TypeScript

```
export function parseCurrency(text: string): number { ... }
```

`$17,050.00` has a symbol, a thousands separator and two decimals, and the application also renders signed amounts (`+$3,200.00`, `-$87.43`). Re-deriving that in nine specs is nine chances to get it subtly wrong, and it throws on unparseable text rather than silently returning `NaN`.

The recent-activity widget is modelled as a **Table**, so row access, column lookup and sorting come from the component rather than from ad-hoc locators in each test.

WHEN YOU NEED TO CHANGE IT

- A stat card is added.
- A quick action is added.

HOW TO CHANGE IT

- 1 Add the slug to `QuickAction`; `quickAction()` and `useQuickAction()` then work unchanged.

WHY THE CHANGE BELONGS HERE

The dashboard is where two independently rendered views of the same data meet, which is why its most valuable test compares its total against the accounts page rather than against a constant.

Related: `tests/ui/dashboard.spec.ts` · `src/pages/bank/accounts.page.ts`

API SURFACE (2 EXPORTS)

CLASS **DashboardPage** extends BasePage

The landing page after sign-in: net-worth cards, quick actions, recent activity.

shell: BankShell

welcomeMessage

statCards

netWorthValue

quickActions

recentTransactions

viewAllTransactionsButton

quickAction(action: QuickAction): Link

A quick-action tile, by the slug the application uses.

useQuickAction(action: QuickAction):
Promise<void>

Clicks a quick action and waits for its destination.

netWorth(): Promise<number>

The Total Net Worth figure, as a number. Currency parsing lives here rather than in each test: `$17,050.00` has a symbol, a thousands separator and two decimals, and re-deriving that regex in nine specs is nine chances to get it subtly wrong.

stats(): Promise<{ label: string; value: string }[]>

Every stat card's label and value, in display order.

FUNCTION **parseCurrency**(text: string): number

Turns a displayed money string into a number. Exported because six page objects need it and the alternative is six slightly different regexes. Handles the leading sign the application uses on credits and debits (`+$3,200.00` , `-$87.43`) and parenthesised negatives.

src/pages/bank/dropdown-panel.ts

24 lines · 1 exports

The `VISIBLE_LISTBOX` selector every custom dropdown in the application shares.

TypeScript

```
export const VISIBLE_LISTBOX = '[role="listbox"]:visible';
```

The application renders each dropdown panel as a `[role="listbox"]` and **leaves it in the DOM when it closes**, merely hidden. On the transfer page, once "From" has been used there are two listboxes: the closed one and the one being opened.

CAREFUL A plain `[role="listbox"]` therefore resolves to the **first** match — the hidden one — and the Dropdown component waits for it to become visible until it times out. The failure reads "waiting for `[role=listbox]` to be visible" while a perfectly good panel is open a few pixels away. That is about as misleading as a failure gets, and it cost a debugging cycle to find.

`:visible` is Playwright's own pseudo-class and resolves to whichever panel is open. It is preferred over per-dropdown container test ids because the application provides those only inconsistently.

WHEN YOU NEED TO CHANGE IT

- The application changes how it renders dropdown panels.

HOW TO CHANGE IT

- 1 Change the constant. Every dropdown in every page object picks it up.

WHY THE CHANGE BELONGS HERE

One shared constant with the reasoning attached, rather than the same subtle selector repeated in six files with the reasoning nowhere.

Related: `src/components/form/dropdown.ts` · `src/pages/bank/transfer.page.ts`

API SURFACE (1 EXPORT)

CONST `VISIBLE_LISTBOX`

The selector every custom dropdown in SecureBank shares. -----
----- WHY `:visible` IS LOAD-BEARING -----
----- The application renders each dropdown's panel as a `[role="listbox"]` and **leaves it in the DOM when it closes**, merely hidden. So on the transfer page, once the "From" dropdown has been used there are two listboxes: the closed one and the one being opened. A plain `[role="listbox"]` therefore resolves to the **first** match — the hidden one — and the Dropdown component waits for it to become visible until it times out. The failure message says "waiting for `[role=listbox]` to be visible" while a perfectly good panel is open a few pixels away, which is about as misleading as a failure can be. `:visible` is Playwright's own pseudo-class and resolves to the open panel, whichever it is. It is used in preference to a per-dropdown container test id because the application only provides those inconsistently — some panels have one, several do not.

src/pages/bank/index.ts

20 lines · 12 exports

Barrel for the SecureBank page objects.

WHEN YOU NEED TO CHANGE IT

- You add a page object.

HOW TO CHANGE IT

- 1 Export it here and construct it in the `bank` fixture.

WHY THE CHANGE BELONGS HERE

One import path, and one place to see every page the suite drives.

Related: `src/fixtures/bank.fixture.ts`

API SURFACE (12 EXPORTS)

RE-EXPORT **NavDestination** from `'./bank-shell'`

RE-EXPORT **LoginPage** from `'./login.page'`

RE-EXPORT **DashboardPage** from `'./dashboard.page'`

RE-EXPORT **AccountsPage** from `'./accounts.page'`

RE-EXPORT **AccountDetailPage** from `'./account-detail.page'`

RE-EXPORT **TransferPage** from `'./transfer.page'`

RE-EXPORT **SendMoneyPage** from `'./send-money.page'`

RE-EXPORT **BillPayPage** from `'./bill-pay.page'`

RE-EXPORT **TransactionsPage** from `'./transactions.page'`

RE-EXPORT **ApplyLoanPage** from `'./apply-loan.page'`

RE-EXPORT **NotificationsPage** from `'./notifications.page'`

RE-EXPORT **ProfilePage** from `'./profile.page'`

src/pages/bank/login.page.ts

146 lines · 1 exports

The sign-in page — the only page reachable without authenticating, and therefore the entry point for every other page object.

FAILURE AND SUCCESS SHARE ONE METHOD

TypeScript

```
async signIn(persona: Persona, options: { rememberMe?: boolean } = {}): Promise<void> { ... }
async signInSuccessfully(persona: Persona): Promise<void> { ... } // waits for the dashboard
async signInExpectingFailure(persona: Persona): Promise<string> { ... } // returns the message
```

`signIn` does not assert the outcome, so a login expected to fail uses exactly the same call. The two wrappers make the **intent** explicit at the call site without duplicating the form filling.

NOTE `signInSuccessfully` waits for the **URL**, not for a timeout. The application shows a "Signing in..." state first, and a test that continued at that point would assert against a page still in flight.

`publishedCredentials()` reads the application's own credentials table, which is what lets one test cross-check `src/data/personas.ts` against reality.

WHEN YOU NEED TO CHANGE IT

- The login form changes.
- A new field is added.

HOW TO CHANGE IT

- 1 Update the component declarations. `src/hooks/auth.setup.ts` delegates here, so the setup project follows automatically.

WHY THE CHANGE BELONGS HERE

Login is used by every other spec indirectly. Concentrating it here means a login change is one edit rather than a suite-wide failure.

Related: `tests/ui/login.spec.ts` · `src/hooks/auth.setup.ts`

API SURFACE (1 EXPORT)

CLASS `LoginPage` extends `BasePage`

The SecureBank sign-in page. It is the only page a test can reach without authenticating, which makes it the entry point for every other page object.

`usernameInput``passwordInput``rememberMeCheckbox``submitButton``forgotPasswordLink``errorBanner``errorMessage``credentialsPanel``credentialsTable``get passwordVisibilityToggle: Button`

The password-visibility control. Located by accessible name because it carries no test id, and because the name is itself the thing worth asserting — it flips between "Show password" and "Hide password", which is how a screen-reader user knows the state changed.

```
signIn(persona: Persona, options: { rememberMe?: boolean } = {}): Promise<void>
```

Fills the form and submits it. Does **not** assert the outcome — a login that is expected to fail uses exactly the same call.

```
signInSuccessfully(persona: Persona): Promise<void>
```

Signs in and waits for the dashboard. The wait is on the **URL**, not on a timeout: the application shows a "Signing in..." state first, and a test that continued at that point would assert against a page still in flight.

```
signInExpectingFailure(persona: Persona): Promise<string>
```

Signs in expecting refusal, and returns the message the app displayed.

`isPasswordMasked(): Promise<boolean>`

Whether the password field is currently masked.

```
publishedCredentials(): Promise< { username: string; password: string; description: string }[]>
```

The credentials the application publishes, read from its own table. Used by one test to cross-check `src/data/personas.ts` against reality — if the app adds or renames an account, that fixture is now wrong and the suite should say so rather than silently testing a stale list.

`src/pages/bank/notifications.page.ts`

76 lines · 1 exports

The notification centre and the unread badges it drives.

CAREFUL The unread count element is **removed from the DOM** once everything is read. A test that waited for it to say "0" would hang until its timeout — which is exactly why absence deserves a method (`unreadCount()`) rather than an inline locator in each test.

WHEN YOU NEED TO CHANGE IT

- The notification list changes.

HOW TO CHANGE IT

- 1
- Update the list component and its item selector.

WHY THE CHANGE BELONGS HERE

The badge and the page are two renderings of one number; the page object exposes both so a test can assert they agree.

Related: `tests/ui/account-settings.spec.ts` · `src/pages/bank/bank-shell.ts`

API SURFACE (1 EXPORT)

CLASS `NotificationsPage` extends `BasePage`

The notification centre, and the unread badges it drives.

`shell:` `BankShell`

`unreadCountText`

`markAllReadButton`

`list`

`itemCount(): Promise<number>`

How many notifications are shown, read and unread together.

`unreadCount(): Promise<number>`

The unread count. The element is **removed from the DOM** once everything is read, so its absence has to be read as zero. A test that waited for it to say "0" would hang until the timeout — a good example of why an "absent" state deserves a method rather than an inline locator.

`unreadItemCount(): Promise<number>`

How many items still offer a "mark as read" control.

`markFirstAsRead(): Promise<void>`

Marks the first unread item as read.

`markAllAsRead(): Promise<void>`

Marks everything as read and waits for the count to disappear.

`src/pages/bank/profile.page.ts`

134 lines · 1 exports

Profile, password change and security settings.

NOTE The edit affordance is **not a dialog** — it replaces the read-only display with an inline form. Every other "edit" in the application opens a modal, so a page object written from that pattern would wait for one that never appears. Checked against the running application rather than assumed.

Two-factor is a `button[role="switch"]` with `aria-checked` — the ARIA switch pattern, which the Toggle component reads directly.

WHEN YOU NEED TO CHANGE IT

- The profile form changes.

HOW TO CHANGE IT

- 1 Update the display and edit-form components together; they mirror each other.

WHY THE CHANGE BELONGS HERE

The inline-edit surprise is the reason this page object is worth reading before writing a test against it.

Related: `tests/ui/account-settings.spec.ts`

API SURFACE (1 EXPORT)

CLASS `ProfilePage` extends `BasePage`

Profile, password change and security settings. Notable because the "edit" affordance is **not a dialog** — it replaces the read-only display with an inline form. A page object that assumed a modal would wait forever, which is why this was checked against the running application rather than assumed from the pattern used elsewhere.

`shell: BankShell``display``username`

`firstName``lastName``email``phone``address``editButton``editForm``firstNameInput``lastNameInput``emailInput``phoneInput``addressInput``saveProfileButton``cancelEditButton``currentPasswordInput``newPasswordInput``confirmPasswordInput``savePasswordButton``twoFactorToggle``resetDataButton``startEditing(): Promise<void>`

Switches the profile section into edit mode.

```
updateProfile(changes: { firstName?: string;
lastName?: string; email?: string; }):
Promise<void>
```

Edits the profile and saves, returning to the read-only display.

src/pages/bank/send-money.page.ts

105 lines · 1 exports

External payments to a saved payee, and the add-payee dialog.

`isBlocked()` reports whether money movement is disabled, which is how the frozen-persona tests assert the block without duplicating the selector.

WHEN YOU NEED TO CHANGE IT

- The payee form changes.

HOW TO CHANGE IT

- 1
- Update the dialog components and `addPayee()`.

WHY THE CHANGE BELONGS HERE

Saving a payee only matters if it appears next time — which is why the page object exposes the payee dropdown rather than just the dialog.

Related: `tests/ui/money-movement.spec.ts`

API SURFACE (1 EXPORT)

CLASS `SendMoneyPage` extends `BasePage`

External payments to a saved payee, plus the "add payee" dialog.

`shell`: `BankShell`

`fromAccount`

`payee`

`addPayeeButton`

`amountInput`

`noteInput`

`reviewButton`

`cancelButton`

`addPayeeDialog`

`payeeNameInput`

`payeeBankInput`

`payeeRoutingInput`

`payeeAccountInput`

`savePayeeButton`

`cancelPayeeButton`

`addPayee(input: { name: string; bank: string; routing: string; account: string; }): Promise<void>`

Opens the dialog, fills it and saves. Returns without asserting.

`isBlocked(): Promise<boolean>`

Whether money movement is blocked — true for the frozen persona.

src/pages/bank/transactions.page.ts

124 lines · 1 exports

All activity across every account: search, account filter, type filter, three sortable columns, pagination and CSV export. The page that exercises the most of the component library.

A PARSER WITH A TRAP IN IT

TypeScript

```
async paginationState(): Promise<{ from: number; to: number; total: number }> {
  const text = await this.paginationInfo.getText();
  const match = /(\d+)\s*[\u2013\u2014]\s*(\d+)\s+of\s+(\d+)/.exec(text);
  ...
}
```

CAREFUL The dash in "Showing 1–10 of 18" is an **en dash**, not a hyphen. Matching only on `-` silently returns nothing — the sort of thing that costs an afternoon, which is why the character class contains both.

The type filter is a segmented `div[role="group"]` of buttons rather than native radios, so tests drive it by clicking the visible option.

WHEN YOU NEED TO CHANGE IT

- A filter or column is added.

HOW TO CHANGE IT

- 1 Add the component and a method that waits for the table to settle after the interaction.

WHY THE CHANGE BELONGS HERE

Pagination, sorting and filtering are where a table stops being a list and starts being a feature, and each needs its own accessor rather than a locator in every test.

Related: `tests/ui/transactions.spec.ts`

API SURFACE (1 EXPORT)

CLASS **TransactionsPage** extends `BasePage`

All activity across every account: search, filter, sort, paginate, export. The richest page in the application, and the one that exercises the most of the component library in a single place.

`shell:` `BankShell`

`searchInput`

`accountFilter`

`typeFilter``table``pagination``paginationInfo``previousPageButton``nextPageButton``downloadButton``emptyMessage``search(term: string): Promise<void>`

Types into search and waits for the debounce.

`descriptions(): Promise<string[]>`

Descriptions of every row currently displayed.

`dates(): Promise<string[]>`

Dates of every row currently displayed, as text.

`amounts(): Promise<number[]>`

Amounts of every row, parsed — sign included.

`sortBy(column: 'date' | 'category' | 'amount'): Promise<void>`

Clicks a sortable column header by its slug.

`paginationState(): Promise<{ from: number; to: number; total: number }>`Parses "Showing 1–10 of 20" into numbers. The dash is an **en dash**, not a hyphen — matching on `–` silently returns nothing, which is the sort of thing that costs an afternoon.`isEmpty(): Promise<boolean>`

True when the "no transactions match" message is showing.

`src/pages/bank/transfer.page.ts`

167 lines · 1 exports

Internal transfers. Models all **three** screens of the flow: the form, the confirmation modal and the success page.

RULE A test that stops at "the form submitted" has not tested a transfer. Modelling all three screens is what lets a test assert the reference number, the confirmed amounts and the resulting balances.

THE HAPPY PATH RETURNS THE ONE THING A CALLER CANNOT GET OTHERWISE

TypeScript

```

async completeTransfer(input: { from: string; to: string; amount: string }): Promise<string> {
  await this.fillForm(input);
  await this.review();
  await this.confirmDialog.waitForOpen();
  await this.confirmButton.click();
  await this.successHeading.waitForVisible();
  return (await this.referenceId.getText()).trim();
}

```

CAREFUL `to` must be filled **after** `from`: the application removes the selected source from the destination list, so the other order picks an option that is about to disappear.

That removal is also how the application prevents a same-account transfer — structurally, rather than by validating afterwards. The test asserts the mechanism the app actually uses rather than forcing an invalid selection to produce a message that never appears.

WHEN YOU NEED TO CHANGE IT

- The flow gains a step.
- The form gains a field.

HOW TO CHANGE IT

- 1 Add the components under the step they belong to; the three comment banners keep the file navigable.

WHY THE CHANGE BELONGS HERE

Money movement is the one action that has to be right in three separate places — validation, confirmation and the resulting balances — and only a page object that models all three lets a test check all three.

Related: `tests/ui/transfer.spec.ts` · `src/pages/bank/dropdown-panel.ts`

API SURFACE (1 EXPORT)

CLASS `TransferPage` extends `BasePage`

Internal transfers between the signed-in user's own accounts. The flow has three distinct screens, and the page object models all three because a test that stops at "the form submitted" has not tested a transfer: 1. the form → `review-transfer-btn` 2. a confirmation modal → `confirm-transfer-btn` 3. a success page → reference number and a summary

`shell`: `BankShell`

`fromAccount`

`toAccount``amountInput``dateTypeToday``dateTypeScheduled``reviewButton``cancelButton``errorMessage``confirmDialog``confirmButton``cancelConfirmButton``successHeading``referenceId``confirmedFrom``confirmedTo``confirmedAmount``backToDashboardButton``anotherTransferButton`

```
fillForm(input: { from?: string; to?: string;
amount?: string }): Promise<void>
```

Fills the form. Stops short of submitting, so a validation test and a happy-path test can share it. `to` is filled after `from` because the application removes the selected source from the destination list — filling them in the other order picks an option that is about to disappear.

```
review(): Promise<void>
```

Submits the form to reach the confirmation modal.

```
completeTransfer(input: { from: string; to:
string; amount: string }): Promise<string>
```

The whole happy path: fill, review, confirm, and land on the success page. Returns the reference number, because that is the one piece of information a caller cannot obtain any other way and is what a follow-up assertion needs.

```
validationError(): Promise<string | null>
```

The validation message the form is currently showing, or `null`.

```
availableBalance(): Promise<number>
```

The "Available: \$X" hint under the amount field, as a number.

API client (src/api)

src/api/api.client.ts

102 lines · 1 exports

A thin HTTP client for **test setup**, not for testing the API itself. GET, POST, PUT, PATCH, DELETE and a raw fetch, with the base URL, auth header and TLS policy taken from the validated configuration, and errors that include status and body.

SEED OVER HTTP, ASSERT IN THE UI

TypeScript

```
const order = await api.post<Order>(ENDPOINTS.orders.create, { items: [{ sku: 'A1', qty: 2 }] })
await ordersPage.goto();
await expect(ordersPage.rowFor(order.id)).toBeVisible();
```

NOTE Seeding through the API takes about 200 ms. Driving the same setup through the UI takes 20 seconds and couples the test to screens it is not testing.

WHEN YOU NEED TO CHANGE IT

- You need a new HTTP verb, different auth (cookie, API key, mTLS), retries, or request logging.

HOW TO CHANGE IT

- 1 Add the header or option in the context creation so every request inherits it.

```
extraHTTPHeaders: {
  'Content-Type': 'application/json',
  ...(token ? { Authorization: `Bearer ${token}` } : {}),
  'X-Tenant': config.tenant,
},
```

- 2 Wrap a flaky endpoint with the `retry` helper rather than adding retries to every call site.

WHY THE CHANGE BELONGS HERE

Restricting this client to setup keeps the boundary clear: UI tests verify the interface, and the API is used to arrange state quickly. Testing the API itself belongs in an API suite.

Related: `src/utils/retry.utils.ts`

API SURFACE (1 EXPORT)

CLASS `ApiClient`

Thin HTTP client for *test setup*, not for testing the API itself. UI tests should create their fixtures over HTTP — seeding a user through the API takes 200ms, whereas driving the signup form takes 20 seconds and couples every test to a screen it is not testing.

```
get(path: string, params?: Record<string, string  
| number | boolean>): Promise<T>
```

```
post(path: string, data?: unknown): Promise<T>
```

```
put(path: string, data?: unknown): Promise<T>
```

```
patch(path: string, data?: unknown): Promise<T>
```

```
delete(path: string): Promise<void>
```

```
raw(method: 'get' | 'post' | 'put' | 'delete',  
path: string, data?: unknown):  
Promise<APIResponse>
```

Raw response, for callers that need headers or a non-JSON body.

```
dispose(): Promise<void>
```

Reference: utilities, reporters & data

Cross-cutting helpers and outputs

Utilities (src/utils)

src/utils/ally.utils.ts

111 lines · 1 exports

Runs axe-core against a page or a subtree, attaches both a JSON and an HTML report to the test result, formats violations into a readable failure message, and fails the test by default when violations are found.

SCOPING A SCAN

TypeScript

```
await scanAccessibility(page, {
  include: ['main'],
  exclude: ['#third-party-widget'],
  tags: ['wcag2a', 'wcag2aa'],
  disableRules: ['color-contrast'], // temporary, with a ticket
});
```

WHEN YOU NEED TO CHANGE IT

- You change the WCAG level you hold yourself to, need to exclude a third-party region, or want a different report format.

HOW TO CHANGE IT

- 1 Change `DEFAULT_WCAG_TAGS` to raise or lower the bar for the whole suite.

```
export const DEFAULT_WCAG_TAGS: WcagTag[] = ['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa',
```

- 2 Disable a rule only with a linked ticket and a date, so the exception is temporary by construction.

WHY THE CHANGE BELONGS HERE

`failOnViolation` defaults to true on purpose: accessibility regressions should break the build, not accumulate in a report nobody opens.

Related: `src/fixtures/custom-matchers.ts` · `src/core/base.page.ts`

API SURFACE (1 EXPORT)

FUNCTION

scanAccessibility(page: Page, options: AllyScanOptions = {}, testInfo?: TestInfo): Promise<AllyReport>

Runs an axe-core scan against the current page (or a subtree of it) and attaches a readable report to the test result. `failOnViolation` defaults to true: accessibility regressions should break the build, not sit in a report nobody opens.

`src/utils/browser.utils.ts`

42 lines · 3 exports

Console capture: `captureConsole` records console errors, uncaught exceptions and failed requests into a container the `consoleErrors` fixture owns, so a test can assert the page logged nothing unexpected.

ASSERT THE PAGE LOGGED NOTHING UNEXPECTED

TypeScript

```
const capture = createConsoleCapture(); // the fixture owns the container
captureConsole(page, capture);          // it fills as the test runs

// ... drive the page ...
expect(capture.errors, capture.errors.join('\n')).toEqual([]);
```

WHEN YOU NEED TO CHANGE IT

- You need to record another signal from the page — a specific console channel, a resource-timing entry.

HOW TO CHANGE IT

- 1 Add the helper here and import it directly where it is needed. If every test needs it, promote it to a fixture instead.

WHY THE CHANGE BELONGS HERE

A page that throws in the console is usually broken even when the assertions pass. Capturing into a container the fixture owns means the check costs a test nothing to opt into.

WATCH OUT FOR

- `captureConsole(page, into)` records into an existing container — that is how the `consoleErrors` fixture avoids depending on `page`.

Related: `src/fixtures/base.fixture.ts`

API SURFACE (3 EXPORTS)

INTERFACE ConsoleCapture

```
errors: string[];
warnings: string[];
pageErrors: string[];
failedRequests: string[];
```

FUNCTION createConsoleCapture(): ConsoleCapture

An empty capture container, to be attached to a page later.

FUNCTION captureConsole(page: Page, into?: ConsoleCapture): ConsoleCapture

Starts recording console errors, uncaught exceptions and failed requests. The returned object mutates as the test runs — assert on it at the end. Pass an existing `capture` to record into a container created earlier; this is what lets a fixture own the container without depending on the page.

src/utils/data.utils.ts

54 lines · 3 exports

Seeded, reproducible test data. `seedFaker` makes a run repeatable from the test title, and `generateUser` builds a complete person. Generate rather than hard-code: a suite whose fixtures say `alice@example.com` fails the second time it runs against a uniqueness constraint.

REPRODUCIBLE DATA, DIFFERENT PER TEST

TypeScript

```
const faker = seedFaker(testInfo.title); // same title -> same data
const user = generateUser(faker);        // a complete, valid person
```

WHEN YOU NEED TO CHANGE IT

- Your domain needs generators (an account number with a checksum, a valid VAT id), or you find a new input that breaks things.

HOW TO CHANGE IT

- 1 Add the generator here so every test uses the same valid shape.

```
export function generateIban(country = 'GB'): string {
  return `${country}${faker.string.numeric(2)}${faker.string.alpha({ length: 4, casing:
  }}
```


WHY THE CHANGE BELONGS HERE

Generated data with unique markers is what lets parallel workers run the same test without colliding on a shared record — the most common cause of "passes alone, fails in parallel".

WATCH OUT FOR

- The `testData` fixture seeds faker from the test title, so within a test the sequence is deterministic and reproducible on retry.

Related: `src/fixtures/base.fixture.ts`

API SURFACE (3 EXPORTS)

INTERFACE `GeneratedUser`

```
firstName: string;
lastName: string;
fullName: string;
email: string;
username: string;
password: string;
phone: string;
company: string;
jobTitle: string;
dateOfBirth: string;
address: { street: string; city: string; state: string; zipCode: string; country: string; }
```

FUNCTION `seedFaker(seed: number): void`

Seeds faker so a failing run can be reproduced exactly.

FUNCTION `generateUser(overrides: Partial<GeneratedUser> = {}): GeneratedUser`

`src/utils/date.utils.ts`

58 lines · 3 exports

`toIsoDate` and `format` for writing dates into inputs and comparing them, and `humanizeDuration` for readable log and report output.

WHEN YOU NEED TO CHANGE IT

- You need a token the formatter does not support, or timezone-aware handling.

HOW TO CHANGE IT

- 1 Add the token to the replacement map and to the regular expression in `format`. For timezone-sensitive assertions, freeze the clock with Playwright's own clock API instead of computing offsets.

```
await page.clock.setFixedTime(new Date('2026-01-15T09:00:00Z'));
```

WHY THE CHANGE BELONGS HERE

Tests that compute "today" against a live clock fail at midnight and on daylight-saving boundaries. Freezing the page clock makes date-dependent UI deterministic.

Related: `src/components/form/date-picker.ts`

API SURFACE (3 EXPORTS)

FUNCTION `toIsoDate(date: Date): string`

ISO date portion only: 2026-08-31

FUNCTION `format(date: Date, pattern = 'YYYY-MM-DD'): string`

Formats a date with a token string: YYYY, MM, DD, HH, mm, ss, MMM, MMMM, D, M. Deliberately dependency-free and locale-independent.

FUNCTION `humanizeDuration(ms: number): string`

Human-readable elapsed time, for logs and reports.

`src/utils/file.utils.ts`

14 lines · 2 exports

Creating a directory if it is missing, and resolving a path against the test-data directory so callers can pass a name relative to `src/data` instead of a relative path that depends on where the caller lives.

THE WHOLE MODULE

TypeScript

```
ensureDir(config.paths.storage);           // used by auth.setup.ts
resolveDataPath('cases.json');             // -> <root>/src/data/cases.json
```

This module used to carry readers for JSON, CSV, Excel and NDJSON, plus temporary-file and download-stabilising helpers. No test ever called any of them, so they were removed along with the `csv-parse` and `xlsx-populate` dependencies they pulled in.

WHEN YOU NEED TO CHANGE IT

- A test needs to read a data file, or needs a generated file on disk.

HOW TO CHANGE IT

- 1 Add the reader here and route it through `resolveDataPath`, so a caller passes a name relative to `src/data` rather than a path relative to itself. Add it in the same change as the test that reads it, not before.
- 2 For size-boundary upload tests, generate the file at run time rather than committing a large binary.

API SURFACE (2 EXPORTS)

FUNCTION `ensureDir(dir: string): string`

FUNCTION `resolveDataPath(filePath: string): string`

Resolves a path against the test-data directory when it is not absolute.

`src/utils/logger.ts`

138 lines · 3 exports

Structured logging with console and file output, deliberately dependency-free. Five levels (`error`, `warn`, `info`, `debug`, `trace`), nested scopes, colour when attached to a terminal, and JSON-lines files at `test-results/logs/test-run.log` and `errors.log`.

SCOPED LOGGING

TypeScript

```
const log = createLogger('CheckoutPage');
log.info('Applying discount', { code: 'SAVE20' });
log.child('promo').debug('Server responded', { status: 200 }); // scope: CheckoutPage:promo
```

NOTE Errors and warnings go to stderr; everything else to stdout, so CI error streams stay signal-only.

WHEN YOU NEED TO CHANGE IT

- You need a new destination (a log aggregator), a different format, or an extra field on every line.

HOW TO CHANGE IT

- 1 Add a sink and write to it in `emit`. Keep every write inside a guard — logging must never be the reason a test fails.

```
class HttpSink {
  write(line: string): void {
    void fetch(process.env.LOG_ENDPOINT!, { method: 'POST', body: line }).catch(() => un
  }
}
```

WHY THE CHANGE BELONGS HERE

General-purpose logging libraries pull in stream polyfills that break across Node releases; a test framework cannot afford a logger that fails to load. Everything needed here is a few dozen lines.

WATCH OUT FOR

- Workers are separate processes, so log lines from parallel tests interleave. The `pid` field on every record is how you separate them.

Related: `src/config/env.config.ts` · `src/fixtures/base.fixture.ts`

API SURFACE (3 EXPORTS)

INTERFACE `Logger`

```
error(message: string, meta?: Record<string, unknown>): void;
warn(message: string, meta?: Record<string, unknown>): void;
info(message: string, meta?: Record<string, unknown>): void;
debug(message: string, meta?: Record<string, unknown>): void;
trace(message: string, meta?: Record<string, unknown>): void;
child(scope: string): Logger;
```

CONST `logger`

Framework-wide logger. Prefer `createLogger('Scope')` inside components.

FUNCTION `createLogger(scope: string): Logger`

`src/utils/network.utils.ts`

155 lines · 1 exports

The network control surface for a page or context: `mock` a response, `modifyResponse` to rewrite a real one, `abort` to simulate failure, `delay` to exercise loading states, block heavy assets or third-party trackers, capture and search requests, wait for a specific response, and toggle offline.

TESTING STATES THAT ARE HARD TO REPRODUCE FOR REAL

TypeScript

```
await network.mock('**/api/cart', { status: 500, body: { message: 'Server error' } });
await network.delay('**/api/search*', 3000);           // prove the spinner appears
await network.abort('**/api/recommendations');         // prove the page survives a dead widget
await network.setOffline(true);                        // offline banner
```

WHEN YOU NEED TO CHANGE IT

- You need a new interception pattern — conditional mocking, recording to a fixture file, replaying a HAR.

HOW TO CHANGE IT

- Add the method to the class so it is available through the `network` fixture everywhere.

```
async mockOnce<T>(pattern: string | RegExp, response: MockResponse<T>): Promise<void> {
  await this.target.route(pattern, async (route) => {
    await route.fulfill({ status: response.status ?? 200, body: JSON.stringify(response) });
    await this.target.unroute(pattern);
  });
}
```

WHY THE CHANGE BELONGS HERE

Error states, slow responses and third-party outages are extremely hard to reproduce against a real backend and trivial to reproduce here — which is what makes those paths testable at all.

WATCH OUT FOR

- The `network` fixture resets routes after each test, so a mock cannot leak into the next one.
- Blocking third parties speeds runs and removes a large source of flakiness, but do not block them in tests whose purpose is the integration.

Related: `src/fixtures/base.fixture.ts` · `src/types/index.ts`

API SURFACE (1 EXPORT)

CLASS NetworkHelper

Network control surface for a page or context: stub responses, simulate failures and latency, and record traffic for assertions.

```
mock(pattern: UrlMatcher, response: MockResponse<T>): Promise<void>
```

Replaces matching requests with a canned response.

```
modifyResponse(pattern: UrlMatcher, transform: (body: T, response: APIResponse) => T | Promise<T>): Promise<void>
```

Lets the request hit the server, then rewrites the response body.

<code>abort(pattern: UrlMatcher, errorCode: Parameters<Route['abort']>[0] = 'failed'): Promise<void></code>	Aborts matching requests — offline behaviour, dead CDN, blocked analytics.
<code>delay(pattern: UrlMatcher, ms: number): Promise<void></code>	Adds latency without changing the payload — for spinner/skeleton coverage.
<code>blockHeavyAssets(): Promise<void></code>	Blocks images/fonts/media to speed up non-visual runs.
<code>blockThirdParty(hosts: string[] = DEFAULT_BLOCKED_HOSTS): Promise<void></code>	Blocks common third-party trackers that add noise and flakiness.
<code>startCapture(filter?: UrlMatcher): void</code>	Starts recording every request for later assertion.
<code>get requests: readonly CapturedRequest[]</code>	
<code>findRequests(pattern: UrlMatcher): CapturedRequest[]</code>	
<code>clearCapture(): void</code>	
<code>reset(): Promise<void></code>	Removes every route handler installed on the target.
<code>waitForResponse(pattern: UrlMatcher, options: { timeout?: number; status?: number } = {}): Promise<Response></code>	Waits for the response of a specific call — e.g. the save that a click triggers.
<code>setOffline(offline: boolean): Promise<void></code>	Simulates going offline (context-level only).

src/utils/retry.utils.ts

22 lines · 1 exports

`waitUntil` — polls a condition until it holds or the timeout elapses, and fails with a message naming the condition. Used where Playwright has no web-first assertion to wait on, such as a progress bar reaching a value.

CAREFUL These are for genuinely flaky boundaries — a third-party widget, a download that occasionally 502s. They are not a substitute for a web-first assertion; prefer Playwright's auto-waiting and `expect.poll` first.

SELECTIVE RETRY

TypeScript

```
await retry(
  () => api.post('/reports', { type: 'monthly' }),
  { attempts: 3, delayMs: 500, retryOn: (error) => error.message.includes('503') },
  'monthly report generation',
);
```

WHEN YOU NEED TO CHANGE IT

- You need a different backoff strategy, jitter, or a shared default policy.

HOW TO CHANGE IT

- 1 Extend `RetryPolicy` in `src/types/index.ts` and honour the new field in `retry`.

WHY THE CHANGE BELONGS HERE

A retry that wraps every failure hides real bugs; `retryOn` is what keeps retrying honest by failing fast on errors that will never succeed.

Related: `src/types/index.ts`

API SURFACE (1 EXPORT)

FUNCTION

```
waitUntil(condition: () => Promise<boolean> | boolean, options: { timeout?: number; interval?: number; message?: string } = {}): Promise<void>
```

Polls `condition` until it returns true or the timeout elapses.

src/utils/string.utils.ts

5 lines · 1 exports

`normalizeText` — collapses the whitespace the DOM inserts, so an assertion compares the text a person sees rather than the text the markup happens to contain. It is the single most-used helper in the framework: every component that reads text routes through it.

WHEN YOU NEED TO CHANGE IT

- Your application formats values in a way the parsers do not handle — a different decimal separator, a suffix like "1.2k".

HOW TO CHANGE IT

- 1 Extend the parser here rather than in the assertion, so every test parses the same way.

```
export function parseCompactNumber(value: string): number {
  const match = /^(([\d.]+)([km])$/i.exec(normalizeText(value));
  if (!match) return Number(normalizeText(value).replace(/^\d./g, ''));
  return Number(match[1]) * (match[2].toLowerCase() === 'k' ? 1_000 : 1_000_000);
}
```

WHY THE CHANGE BELONGS HERE

Rendered text carries non-breaking spaces, line breaks and indentation. Normalising once removes an entire class of confusing string-comparison failures.

Related: `src/core/base.component.ts` · `src/fixtures/custom-matchers.ts`

API SURFACE (1 EXPORT)

FUNCTION `normalizeText(value: string | null | undefined): string`

Collapses all whitespace runs to single spaces and trims — DOM text is messy.

Reporters (src/reporters)

`src/reporters/summary.reporter.ts`

152 lines · 1 exports

A custom Playwright reporter that prints a compact end-of-run summary and writes `reports/summary.json` containing totals, every failure with its first error line, flaky tests, the five slowest tests and the full test list with tags and durations.

Expected failures are passes

THE OUTCOME, NOT THE RAW STATUS

TypeScript

```
const met = result.status === test.expectedStatus;
const outcome =
  met && result.status === 'failed' ? 'passed'
  : !met && result.status === 'passed' ? 'failed'
  : result.status;
```

A test annotated `test.fail()` is **expected** to fail, and Playwright records that in `test.expectedStatus`. Its raw `result.status` is still `failed`, so counting that directly reports a green run as red — which is exactly what happened once the suite began recording known defects this way.

The stored status is therefore the `*outcome*`: whether the result matched what was expected. An expected failure is a pass; an unexpectedly **passing** `test.fail()` is a failure — which is the whole point of the annotation, because it means the defect has been fixed and the annotation should be removed.

NOTE `expectedFailure` is recorded per test, so a pipeline can report "111 passed, 4 of them documented defects" rather than hiding the distinction.

CONSUMING THE SUMMARY IN CI

TypeScript

```
const summary = JSON.parse(fs.readFileSync('reports/summary.json', 'utf8'));
if (summary.totals.flaky > 0) core.warning(`${summary.totals.flaky} flaky tests`);
for (const failure of summary.failures) core.error(`${failure.title}: ${failure.error}`);
```

WHEN YOU NEED TO CHANGE IT

- A dashboard, Slack notification or PR comment needs another field — ownership, suite name, retry count, browser.

HOW TO CHANGE IT

- 1 Add the field in `onTestEnd` where the test result is available, then include it in the `summary` object in `onEnd`.

```
this.tests.push({
  // ...
  annotations: test.annotations.map((a) => a.type),
  browser: test.parent.project()?.use.browserName ?? 'unknown',
});
```

WHY THE CHANGE BELONGS HERE

CI needs machine-readable results. Answering "what broke and how slow are we" from a JSON file is far cheaper than downloading and unzipping an HTML report.

WATCH OUT FOR

- Passing `--reporter=...` on the command line replaces the configured reporters, so this one does not run on those invocations.

Related: `playwright.config.ts`

API SURFACE (1 EXPORT)

CLASS **SummaryReporter** implements Reporter

Console summary plus a machine-readable `summary.json`. Exists so CI can answer "what broke and how slow are we" without unzipping the HTML report: the JSON is what dashboards and PR comments consume.

```
onBegin(config: FullConfig, suite: Suite): void
```

```
onTestEnd(test: TestCase, result: TestResult):  
void
```

```
onEnd(result: FullResult): Promise<void>
```

```
printsToStdio(): boolean
```

Test data (src/data)

src/data/personas.ts

144 lines · 6 exports

The seven SecureBank accounts and the facts about the seeded dataset that tests assert against.

Why credentials are in the repository

The framework's rule is that credentials never enter version control, and it is not being bent here. These are **published on the application's own login page**, in a table headed "Test credentials", for anyone to read. They authenticate nothing and protect nothing.

Treating them as secrets would mean handing every developer a `.env` before they could run a single test against a public practice app — ceremony with no security benefit. `getUser()` in `env.config.ts` remains the only route for credentials in every other environment.

The personas are the point

PERSONA	BEHAVIOUR	WHAT IT MAKES TESTABLE
<code>standard</code>	Full access	Every happy path
<code>locked</code>	Sign-in refused as suspended	A distinct message from a wrong password
<code>frozen</code>	Signs in; money movement disabled	Transfer, send and bill-pay all blocked
<code>overdraft</code>	Negative balance	The overdraft indicator
<code>slow</code>	Slow loading	Real loading states

PERSONA	BEHAVIOUR	WHAT IT MAKES TESTABLE
<code>buggy</code> (<code>error_user</code>)	A planted defect in the loan total	Proving the suite can find a bug
<code>admin</code>	Additional view	Role-dependent chrome

Negative paths and degraded states are reachable by **logging in as somebody else**, rather than by mocking a failure and hoping the mock resembles reality. That is unusual and worth exploiting.

SEED — dataset facts, read from the running app

NOTE Every value in `SEED` was read from the application, not from its documentation. One of them was originally wrong: `totalTransactions` was recorded as 20 from a session that had already performed a transfer — each completed transfer adds a debit and a matching credit. The true figure in a fresh context is 18. **Seed values must be read from a clean context**, or they encode whatever the person exploring happened to do first.

WHEN YOU NEED TO CHANGE IT

- The application reseeds.
- A persona is added or changes behaviour.

HOW TO CHANGE IT

- 1 Update the constant. Nine specs read from `SEED`, so one edit covers all of them.
- 2 Run the fixture-integrity test, which reads the app's own credentials table and compares it with this file.

```
npx playwright test -g "personas fixture matches"
```

WHY THE CHANGE BELONGS HERE

A hand-written copy of a table the application renders itself will drift. Keeping the copy in one file, and testing it against the source, is what stops that drift being silent.

WATCH OUT FOR

- Read seed values from a fresh browser context. State lives in `localStorage`, so an exploratory session pollutes them.

Related: `tests/ui/login.spec.ts` · `src/fixtures/bank.fixture.ts`

API SURFACE (6 EXPORTS)

TYPE **PersonaTrait**

The behaviours the application deliberately exhibits per account.

```
| 'full-access' /** Login is refused with a suspension message. */ | 'locked' /** Signs in,
```

INTERFACE **Persona**

```
readonly username: string;  
readonly password: string;  
readonly trait: PersonaTrait;  
readonly description: string;
```

CONST **PERSONAS**

Keyed by trait rather than by username, so a test reads as an intention — `personas.frozen` — instead of as a lookup somebody has to decode.

TYPE **PersonaName**

```
keyof typeof PERSONAS
```

CONST **ALL_PERSONAS**

Every persona, for the test that cross-checks this file against the app.

CONST **SEED**

Facts about the seeded dataset that tests assert against. Kept here rather than inline so that when the application reseeds, one file changes instead of nine specs. Every value was read from the running application, not from its documentation.

src/data/README.md

23 lines

Explains what belongs in the data directory: static fixtures only, with anything unique per run generated at runtime so parallel workers never collide. Also records the rule that credentials and production data never live here.

WHEN YOU NEED TO CHANGE IT

- You add a data file or change the data conventions.

HOW TO CHANGE IT

- 1 Add the file to the table with a one-line purpose so the directory stays self-describing.
-

WHY THE CHANGE BELONGS HERE

A data directory without rules becomes a graveyard of stale records that quietly couple unrelated tests to each other.

Related: `src/utls/file.utls.ts` · `src/utls/data.utls.ts`

Reference: tests & repository docs

Where specs live and the conventions they follow

Tests folder

`tests/ally/.gitkeep`

Placeholder for the `accessibility` project's specs — axe scans across key pages and states.

WHEN YOU NEED TO CHANGE IT

- You add an accessibility spec.

HOW TO CHANGE IT

1

Scan meaningful states, not just the landing page: forms with errors showing, modals open, menus expanded.

WHY THE CHANGE BELONGS HERE

Most accessibility defects appear in interactive states, which a scan of the initial render never reaches.

Related: `src/utils/ally.utils.ts`

`tests/ally/bank.ally.spec.ts`

187 lines

Automated axe scans across all ten pages and both themes, with the application's existing violations baselined.

The baseline approach, and why it is not a climbdown

SecureBank has two genuine WCAG 2 AA violations, found by this suite. We do not control the application, so a suite that simply failed on them would be red forever — and a permanently red suite is one people stop reading, which loses every *future* violation too.

RULE	IMPACT	WHERE
<code>color-contrast</code>	serious	dashboard, transactions, apply-loan, notifications

RULE	IMPACT	WHERE
<code>aria-prohibited-attr</code>	serious	notifications

The contrast failure is a green (`#009966`) on white at 14px measuring 3.65:1 against the 4.5:1 WCAG 1.4.3 requires — close enough to look fine and far enough to fail, which is exactly what only a tool catches.

RULE The known rules are excluded from the gating scans, so **any new violation fails the build**, while a separate never-failing test reports the full backlog every run. That is the standard way to adopt accessibility testing on an application that already has issues; the alternative — "fix everything before you can test anything" — is how it never gets adopted at all.

A third defect is recorded here with `test.fail()`: three controls on the transfer page have no accessible name. axe does not flag them, which is precisely the class of issue an automated scan misses and a hand-written check catches.

NOTE Honest limits: axe detects roughly a third of WCAG issues. It cannot judge whether a focus order makes sense or whether an error message is comprehensible. The keyboard tests in `navigation.spec.ts` cover part of what it misses.

WHEN YOU NEED TO CHANGE IT

- A baselined violation is fixed.
- A page is added.

HOW TO CHANGE IT

- 1 Remove the rule from `KNOWN_VIOLATIONS` once it passes. **Shrink this list; never grow it.** A rule removed and passing in progress the suite then protects.
- 2 Add the slug to `PAGES`; the per-page test is generated from it.

WHY THE CHANGE BELONGS HERE

Scanning ten pages is cheap because `checkAccessibility` lives on `BasePage`. What costs judgement is deciding what to gate on, and that decision is written down here.

Related: `src/utils/all.ts` · `tests/ui/navigation.spec.ts`

tests/README.md

123 lines

The rules for writing specs: import from `@fixtures/index`, tag every test, one behaviour per test, no selectors in specs, seed state through the API, and never use `waitForTimeout`. Also documents the folder layout and naming.

WHEN YOU NEED TO CHANGE IT

- A convention changes, or a new folder or tag is introduced.
-

HOW TO CHANGE IT

- 1 Update the rule here and in the review checklist so the two never disagree.
-

WHY THE CHANGE BELONGS HERE

Conventions that live next to the code get read; conventions in a wiki do not.

Related: `src/fixtures/index.ts`

tests/ui/.gitkeep

Placeholder keeping `tests/ui/` in version control. Functional specs live here, mirroring the application's feature areas.

WHEN YOU NEED TO CHANGE IT

- You add your first spec — delete the placeholder then.
-

HOW TO CHANGE IT

- 1 Create `tests/ui/<feature>/<feature>.spec.ts` and remove the `.gitkeep`.
-

WHY THE CHANGE BELONGS HERE

Mirroring feature areas keeps ownership obvious and makes `--grep` by path useful.

Related: `tests/README.md`

tests/ui/account-settings.spec.ts

141 lines

Notifications and profile — the two pages the application's catalogue does not cover.

Included because "all UI" means all of it, and because between them they exercise three widget types nothing else does: a list with per-item actions, an inline edit form and an ARIA switch.

The notification tests assert **three renderings of one number** agree — the page count, the per-item controls and the sidebar badge. A stale badge is a classic defect and is invisible from the page itself.

The profile test asserts explicitly that editing produces an inline form and **no** `[role="dialog"]` — the surprise on that page, and the thing a page object written from the pattern used elsewhere would get wrong.

All three password fields are checked for `type="password"`. A confirm field rendering as plain text is a genuine and common oversight.

WHEN YOU NEED TO CHANGE IT

- Either page changes.

HOW TO CHANGE IT

- 1 Update the page object.

WHY THE CHANGE BELONGS HERE

The pages nobody specifies are the pages nobody tests, which is exactly why they are worth covering.

Related: `src/pages/bank/notifications.page.ts` · `src/pages/bank/profile.page.ts`

tests/ui/accounts.spec.ts

189 lines

Covers TC-ACC-001..006 across three personas, plus the add dialog and the detail page.

The overdraft and frozen personas exist precisely so those states are reachable **without inventing them**. TC-ACC-003 asserts not merely that a balance is negative but that the interface *says so* — a number the user cannot interpret is not a warning.

A second documented defect

CAREFUL The account dialog has `role="dialog"` and an accessible name, but **no** `aria-modal="true"`. Without it a screen reader is not told to confine the user to the dialog, so they can arrow out into the page behind it while it is open — a confusing state sighted users never see. Recorded with `test.fail()`; a companion test asserts what the dialog does do correctly.

The cancel test asserts nothing was persisted. A dialog that saves on cancel is a genuine and surprisingly common defect, and only an explicit before/after comparison catches it.

WHEN YOU NEED TO CHANGE IT

- The account form or list changes.

HOW TO CHANGE IT

- 1 Update `AccountsPage`; these tests describe behaviour.

WHY THE CHANGE BELONGS HERE

Three personas in one spec is what turns "the list renders" into coverage of the states a real customer can actually be in.

Related: `src/pages/bank/accounts.page.ts`

`tests/ui/apply-loan.spec.ts`

160 lines

Covers TC-LOAN-001..006 — including **reproducing the application's planted defect**.

The most valuable test in the repository

SecureBank deliberately ships a bug for `error_user`: the total beneath the loan history omits the most recent loan. TC-LOAN-004 asks for it to be reproduced, and this is the one test that proves the framework can **find** something rather than confirm things work.

WRITTEN AS IT WOULD BE AGAINST A REAL SYSTEM

TypeScript

```
const displayed = await bank.applyLoan.displayedTotal();
const expected = await bank.applyLoan.expectedActiveTotal();

expect(displayed, `shows ${displayed} but the active and pending loans sum to ${expected}`)
  .toBeCloseTo(expected, 2);
```

Measured: `standard_user` shows \$39,300 and its active loans sum to \$39,300. `error_user` has identical rows but shows \$24,300 — short by exactly the \$15,000 newest pending loan.

RULE A control test asserts the same comparison **passes** for a healthy account. Without it, a failing total for `error_user` could just as easily mean the assertion is wrong. That control is what makes the planted-defect result trustworthy.

NOTE Getting the comparison right took two attempts. The total is labelled "Total (Active/Pending)" and the history paginates at five rows, so comparing it against the visible page is wrong twice over — it misses later pages and includes closed loans. `expectedActiveTotal()` walks the pages and filters by status.

WHEN YOU NEED TO CHANGE IT

- The defect is fixed — this test will then fail and the annotation should be removed.

HOW TO CHANGE IT

- 1 Remove the `test.fail()` line. That is the whole point of recording it this way.

WHY THE CHANGE BELONGS HERE

A suite that has never caught anything is a suite nobody trusts. This is the demonstration.

Related: `src/pages/bank/apply-loan.page.ts` · `src/data/personas.ts`

tests/ui/dashboard.spec.ts

118 lines

Covers the application's own TC-DASH-001..006, plus the stat cards and every quick action.

SecureBank publishes a catalogue of intended test cases at `/bank/test-cases`. Each test here names the case it satisfies, so coverage can be checked against the app's own specification rather than against somebody's memory.

The assertion worth copying

TC-DASH-002 COMPARES TWO INDEPENDENT RENDERINGS

TypeScript

```
const dashboardTotal = await signedIn.dashboard.netWorth();

await signedIn.dashboard.shell.navigateTo('accounts');
const accountsTotal = await signedIn.accounts.totalBalance();

expect(dashboardTotal).toBe(accountsTotal);
```

The interesting assertion is not "is it \$17,050" — that would only restate the fixture. It is that two independently rendered views of the same underlying data **agree**. A rounding bug or a stale cache in either place breaks this and nothing else.

The theme test toggles **twice**. A control that only works in one direction is a common defect, and a single click would never reveal it.

WHEN YOU NEED TO CHANGE IT

- The dashboard gains a card or a tile.

HOW TO CHANGE IT

- 1 Add the slug to `QuickAction` in the page object; the sweep test then covers it automatically.

WHY THE CHANGE BELONGS HERE

The dashboard is a summary of data rendered elsewhere, which makes cross-checking the more valuable assertion than checking any single number.

Related: `src/pages/bank/dashboard.page.ts`

tests/ui/login.spec.ts

141 lines

Authentication: the happy path, a wrong password, a suspended account, password visibility, remember-me, and a fixture-integrity check.

These specs deliberately do **not** use the `signedIn` fixture — they are about getting signed in, so they must start signed out. Because Playwright only constructs a fixture a test names, that costs nothing.

The test that keeps the fixtures honest

READING THE APP'S OWN CREDENTIALS TABLE

TypeScript

```
const published = await bank.login.publishedCredentials();

expect(published.length).toBe(ALL_PERSONAS.length);
for (const persona of ALL_PERSONAS) {
  const match = published.find((row) => row.username === persona.username);
  expect(match, `the app no longer publishes "${persona.username}"`).toBeDefined();
  expect(match?.description).toBe(persona.description);
}
```

`src/data/personas.ts` is a hand-written copy of a table the application renders itself, and copies drift. This reads the real table and compares — so if the app adds, renames or repurposes an account, the suite says so instead of silently testing a stale list.

The password-visibility test asserts the **accessible name** flips between "Show password" and "Hide password", not just the input type. That is how a screen-reader user knows the state changed, and it is invisible to a purely visual check.

WHEN YOU NEED TO CHANGE IT

- The login form or its messages change.

HOW TO CHANGE IT

- 1 Most changes belong in `LoginPage`, not here. These tests describe behaviour, not markup.

WHY THE CHANGE BELONGS HERE

Login is the gate to every other spec. A distinct message for a suspended account versus a wrong password is a real product decision worth protecting: a locked-out user needs to know it is not their typing.

Related: `src/pages/bank/login.page.ts` · `src/data/personas.ts`

tests/ui/money-movement.spec.ts

143 lines

Covers TC-SEND-001..006 and TC-BILL-001..006 — the two external payment flows.

Two flows in one file because they share a shape: pick a source account, pick or create a recipient, enter an amount, review. Splitting them would duplicate the reasoning without separating anything.

TC-SEND-004 asserts a saved payee **appears in the dropdown afterwards**. Asserting the dialog closed would prove nothing at all.

Both flows are asserted blocked for the frozen persona — disabled controls, not merely rejected submissions, which is the correct treatment for a blocked account.

WHEN YOU NEED TO CHANGE IT

- Either payment form changes.

HOW TO CHANGE IT

- 1 Update the page object; check which fields are required by submitting without them.

WHY THE CHANGE BELONGS HERE

A "saved" record that cannot be found again has not been saved in any sense the user cares about.

Related: `src/pages/bank/send-money.page.ts` · `src/pages/bank/bill-pay.page.ts`

tests/ui/navigation.spec.ts

149 lines

The application shell: every sidebar destination, the brand link, the unread badge, responsive behaviour and the skip link.

A **generated test per destination** rather than one loop inside a single test. Ten separate results mean a broken link names itself in the report; one looping test would just say "navigation failed".

A documented accessibility defect

CAREFUL WCAG 2.4.1 requires a keyboard user to be able to skip repeated navigation. SecureBank ships the correct markup — the skip link is first in the DOM, targets a real element and works when focused — but the **first Tab bypasses it** and lands on the brand link, so a keyboard user can never reach it. Verified directly against the running application.

RECORDED WITH TEST.FAIL(), NOT SKIPPED OR DELETED

TypeScript

```
test('the skip link should be the first thing Tab reaches @a11y @known-issue', async ({ signedIn }) => {
  test.fail(true, 'Known defect: the first Tab focuses the brand link (WCAG 2.4.1).');
  await signedIn.dashboard.page.keyboard.press('Tab');
  await expect(signedIn.dashboard.page.locator(':focus')).toHaveAttribute('data-testid', 'skip-link');
});
```

The expectation stays in the suite written as it *should* behave; the run stays green so nobody learns to ignore a permanently red test; and the moment somebody fixes it the test **fails**, telling us to remove the annotation. A skipped test would go quiet forever; a deleted one would lose the finding.

NOTE A separate passing test asserts what the skip link does do correctly. Note it checks the **bounding box**, not `toBeHidden()`: the `sr-only` pattern clips the link to 1x1 rather than hiding it, because a `display: none` element cannot be focused — so Playwright reports it as visible and `toBeHidden()` would fail for the wrong reason.

WHEN YOU NEED TO CHANGE IT

- A destination is added to the sidebar.

HOW TO CHANGE IT

- 1 Add a row to the `DESTINATIONS` table; the per-destination test is generated from it.

WHY THE CHANGE BELONGS HERE

Chrome that appears on every page deserves one spec, not an assertion repeated in ten.

Related: `src/pages/bank/bank-shell.ts` · `tests/a11y/bank.a11y.spec.ts`

tests/ui/transactions.spec.ts

206 lines

Covers TC-TXN-001..006: loading, search, empty state, account filter, sorting, pagination, the type filter and the CSV export.

An assertion that had to be corrected

NOTE An earlier draft asserted that clicking sort twice produced the reverse of the first page. It failed against entirely correct behaviour: sorting applies to all 18 transactions while only 10 are shown, so descending page 1 holds the largest ten and ascending page 1 the smallest — two different slices, not a reversal. Comparing the **directions** is the assertion that actually catches a sort control ignoring the second click.

The search term is taken from the data itself rather than hard-coded, so the test does not depend on a description that may not exist tomorrow.

The empty-state test asserts the message, not just zero rows: a table rendering nothing looks identical to one that failed to load, and the user cannot tell which happened.

The CSV test uses Playwright's download event — the only way to prove a download happened, since there is no DOM to assert on — and reads the stream to check the file is not empty.

WHEN YOU NEED TO CHANGE IT

- A filter, column or page size changes.

HOW TO CHANGE IT

- 1 Update `SEED` for a page-size change; nine specs read from it.

WHY THE CHANGE BELONGS HERE

Pagination and sorting are where a table stops being a list and starts being a feature, and both have boundaries that only an explicit test visits.

Related: `src/pages/bank/transactions.page.ts` · `src/data/personas.ts`

tests/ui/transfer.spec.ts

190 lines

Covers TC-XFER-001..006: form loading, validation, the confirmation step, the success page, and the resulting balances.

The most valuable suite here, because a transfer is the one action in a banking application that moves real money and has to be right in three separate places.

TC-XFER-006 — BOTH SIDES AND THE TOTAL

TypeScript


```
expect(fromAfter).toBeCloseTo(fromBefore - amount, 2);
expect(toAfter).toBeCloseTo(toBefore + amount, 2);
expect(fromAfter + toAfter).toBeCloseTo(fromBefore + toBefore, 2);
```

Checking only the source would miss money that left one account and never arrived; checking only the total would miss it going to the wrong place. The starting position is read **from the application**, not from the fixture, so the test stays correct if the seed data changes.

TC-XFER-004 asserts the confirmation summary echoes what was requested. An application that confirms a different amount from the one submitted is the worst possible defect in this domain, and only that assertion catches it.

NOTE TC-XFER-002 asserts the destination list **excludes** the chosen source. The application prevents a same-account transfer structurally rather than by validating afterwards — a better design, and asserting the mechanism the app really uses is more honest than forcing an invalid selection to produce a message that never appears.

WHEN YOU NEED TO CHANGE IT

- The transfer flow changes.

HOW TO CHANGE IT

- 1 Update `TransferPage`. If a step is added, model it there first.

WHY THE CHANGE BELONGS HERE

Money movement is where a UI suite earns its keep. A cancelled confirmation must be a genuine no-op, and that is asserted too.

Related: `src/pages/bank/transfer.page.ts`

`tests/visual/.gitkeep`

Placeholder for the `visual` project's specs. Baselines are written to `__screenshots__/<project>/` beside the spec.

WHEN YOU NEED TO CHANGE IT

- You add a visual spec.

HOW TO CHANGE IT

- 1 Add the spec here — the `visual` project is the only one that runs this folder, and every other project ignores it.

WHY THE CHANGE BELONGS HERE

Isolating visual tests keeps pixel comparison on one fixed rendering stack, where it is meaningful.

Related: `playwright.config.ts`

`tests/visual/bank.visual.spec.ts`

107 lines

Visual regression on the login page, the dashboard cards, the sidebar in both themes, the transfer form and the empty transactions state.

What makes a visual test worth having rather than an annoyance

1. **Stabilise before capturing.** Animations, carets and — here — live monetary figures all change between runs. Every balance is masked; what remains under test is the `*layout*`.
2. **Capture the smallest meaningful region.** A full-page snapshot fails when anything anywhere changes, so it tells you a page changed without telling you what. Component-level snapshots point at the thing that moved.

Masking is not skipping: the numbers are covered and the card layout is compared. Masking the volatile part is what makes the stable part testable at all.

The sidebar is captured in **both themes**. A palette regression that only affects dark mode is invisible to a light-mode baseline.

NOTE The login page is the one page captured whole, because it has no balances and no dates — nothing that legitimately varies.

WHEN YOU NEED TO CHANGE IT

- A deliberate design change makes a baseline stale.

HOW TO CHANGE IT

- 1 Refresh baselines deliberately, never as a reflex.

```
npm run test:visual:update
```

- 2 Review the resulting image diff before committing it — that review is the entire value of the test.

WHY THE CHANGE BELONGS HERE

Layout, spacing and alignment are what a screenshot is good at and an assertion is bad at. Everything else should be an assertion.

Repository documentation

docs/ARCHITECTURE.md

140 lines

The concise markdown version of the design rationale, for reading on GitHub without building the documentation site.

WHEN YOU NEED TO CHANGE IT

- A structural decision changes.

HOW TO CHANGE IT

- 1 Update this file and the corresponding architecture page content in `scripts/docs/content/guides.mjs`.

WHY THE CHANGE BELONGS HERE

Reviewers read markdown in pull requests; the generated site is for reference. Keeping both is cheap when the content is short.

Related: `scripts/docs/content/`

docs/COMPONENTS.md

157 lines

The markdown summary of every component and its key methods, for quick lookup in an editor or on GitHub.

WHEN YOU NEED TO CHANGE IT

- You add or rename a component.

HOW TO CHANGE IT

- 1 Add the row here and the entry in `scripts/docs/content/files.components-*.mjs`; the generated API surface updates itself.

WHY THE CHANGE BELONGS HERE

A one-screen table is the fastest way to answer "does the framework already have something for this?"

README.md

178 lines

The front door: quick start, the folder map, how a test looks, the command table, environment handling, reporting outputs and links to the deeper documentation.

WHEN YOU NEED TO CHANGE IT

- A command, folder or workflow changes.

HOW TO CHANGE IT

- 1 Keep it short and link outwards. Detail belongs in the generated documentation, which cannot drift from the code.

WHY THE CHANGE BELONGS HERE

A README that tries to be complete goes stale. A README that orients and links stays useful.

Related: `docs/ARCHITECTURE.md` · `docs/COMPONENTS.md`

Playbooks

I need to change something — where do I go, what do I do, and why there?

Task-oriented runbooks. Each answers the same three questions: **where** to go, **what** to do, and **why** the change belongs there rather than somewhere else.

1. Point the framework at a real application

- 1 **Where:** `src/config/environments.ts`. Set the real URLs for each environment.

```
dev: {  
  name: 'dev',  
  baseUrl: 'https://dev.yourapp.com',  
  apiBaseUrl: 'https://dev.yourapp.com/api',  
  retries: 1,  
  workers: undefined,  
  ignoreHTTPSErrors: true,  
},
```

- 2 **Where:** `.env`. Set `TEST_ENV` and the credentials for each role.

```
TEST_ENV=dev  
STANDARD_USER=qa.standard@yourapp.com  
STANDARD_PASSWORD=...
```

- 3 **Where:** `src/hooks/auth.setup.ts`. Replace the placeholder selectors in `signIn` with your login page's. This is the only file in the framework that must know your markup.

- 4 **Verify:** run the setup project alone before anything else.

```
npx playwright test --project=setup
```

Why here: URLs are non-secret configuration and belong in version control; credentials are secrets and belong in the environment; the login flow is application-specific and is deliberately isolated to one function so nothing else in the framework depends on your markup.

2. Add a page object for a new screen

- 1 **Where:** add `src/pages/<feature>.page.ts`, modelled on an existing page such as `src/pages/bank/transfer.page.ts`.

- 2 **What:** set `path` and `readyIndicator`, declare components as readonly fields, and write business actions.

```
export class InvoicesPage extends BasePage {
  protected readonly path = '/invoices';
  protected readonly readyIndicator: SelectorLike = '[data-testid="invoices-root"]';

  private readonly factory = ui(this.page);
  readonly grid = this.factory.dataGrid('[data-testid="invoice-grid"]', { name: 'Invoices' });
  readonly search = this.factory.input('[data-testid="search"]', { name: 'Search' });

  async searchFor(term: string): Promise<void> {
    await this.search.typeAndSettle(term);
    await this.grid.waitForData();
  }
}
```

- 3 **Where:** add it to the `pages` fixture in `src/fixtures/bank.fixture.ts` so specs can reach it.

Why here: the ready indicator is what makes navigation deterministic; declaring components as fields means a selector change is one line and editor autocomplete lists the whole page surface.

3. The component library does not cover my widget

- 1 **First:** check whether an existing component only needs configuration. Most do — `Dropdown`, `DataGrid`, `DatePicker`, `Autocomplete` and the rest take their selectors as options.

```
ui.dropdown('#country', { panelSelector: '.MuiAutocomplete-popover', optionSelector: '.MuiAutoccc
```

- 2 **Then:** if the behaviour differs, subclass and override only the method that differs.

```
class HoverDropdown extends Dropdown {
  override async open(): Promise<void> { /* hover instead of click */ }
}
```

- 3 **Otherwise:** create a new component under the right group in `src/components/`, extending `BaseComponent`.

```
export class Timeline extends BaseComponent {
  protected override get componentType(): string { return 'Timeline'; }

  async getEvents(): Promise<string[]> {
    return this.step('read events', async () => {
      return (await this.locator.locator('.event').allInnerTexts()).map((t) => t.trim());
    });
  }
}
```

- 4 **Then:** export it from `src/components/index.ts` and add a factory method in `component.factory.ts`.
- 5 **Finally:** document it — add an entry to the matching module in `scripts/docs/content/` and run `npm run docs`.

Why here: a widget behaviour written in a page object is invisible to every other page. Written as a component, it is available everywhere, reports itself in traces, and inherits all the base behaviour for free.

4. A test is flaky

- 1 **Diagnose first.** Open the trace from the HTML report; the failing step is named after the component and action.

```
npm run report
```

- 2 **If the wait is wrong:** fix it in the component, not the test. Add a condition-based wait rather than a delay.
- 3 **If the timeout is genuinely too short:** raise the named budget in `src/config/timeouts.ts` — never hard-code a number at the call site.
- 4 **If the data collides:** use the `testData` fixture, which seeds faker per test, so parallel workers cannot share a record.
- 5 **If a third party is unreliable:** stub it with the `network` fixture, or block it.

```
await network.mock('**/api/recommendations', { body: [] });  
await network.blockThirdParty();
```

- 6 **If the app is genuinely slow:** that is a product finding. Record it rather than hiding it behind a longer timeout.

Why here: a wait added to a test fixes one test; a wait added to the component fixes every current and future use of that element. Retries in `playwright.config.ts` mask flakiness — they are a safety net, not a fix.

5. Add a new environment

- 1 **Where:** `src/types/index.ts` — add the name to `EnvironmentName`. TypeScript will now point at everything that must be updated.
- 2 **Where:** `src/config/environments.ts` — add the entry with URLs, retries, workers and TLS policy.
- 3 **Where:** `src/config/env.config.ts` — add the value to the `TEST_ENV` enum in the schema.
- 4 **Where:** `.github/workflows/playwright.yml` — add it to the `workflow_dispatch` choice list.

5 Verify:

```
TEST_ENV=preprod npm run test:smoke
```

Why here: the union type is the mechanism that turns "add an environment" from a search-and-hope exercise into a compiler-guided checklist.

6. Add a configuration variable

1 **Where:** `src/config/env.config.ts` — add it to the zod schema with a fallback.

```
MAX_UPLOAD_MB: numeric(25),
```

2 **What:** expose it on the exported `config` object in the shape callers should see.

```
maxUploadMb: raw.MAX_UPLOAD_MB,
```

3 **Where:** `.env.example` — document it with a safe placeholder.

4 **Where:** the CI workflow, if it must be set there.

Why here: reading `process.env` at the point of use reintroduces exactly the failure mode this file exists to prevent — an unparsed string, a missing value, a suite silently pointed at the wrong host.

7. Add a browser, device or specialised suite

1 **Where:** `playwright.config.ts`, the `projects` array. Copy an existing entry.

```
{
  name: 'galaxy-s23',
  use: { ...devices['Galaxy S9+'] },
  dependencies: ['setup'],
  testIgnore: ['**/visual/**', '**/a11y/**'],
},
```

2 **Where:** `.github/workflows/playwright.yml` — add it to the matrix if it should run in CI.

3 **Where:** `package.json` — add a script if people will run it directly.

Why here: `dependencies: ['setup']` is what gives the new project authenticated sessions; without it its tests start signed out and fail in a way that looks like an application bug.

8. Seed test data instead of clicking through the UI

1 **Where:** the service or fixture that owns the call — the framework has no central route table, so a route lives with the code that uses it.

- 2 **Where:** the test, or a fixture if several tests need it. A fixture also gives you automatic cleanup.

```
order: async ({ api }, use) => {
  const order = await api.post<Order>(ENDPOINTS.orders.create, { items: [{ sku: 'A1', qty: 1 }] })
  await use(order);
  await api.delete(ENDPOINTS.orders.byId(order.id));
},
```

Why here: seeding over HTTP takes about 200 ms; the same setup through the UI takes 20 seconds and couples the test to screens it is not testing. When those screens break, the wrong test fails.

9. Add an assertion the whole team will reuse

- 1 **Where:** `src/fixtures/custom-matchers.ts`.
- 2 **What:** return `pass` plus a `message` function covering both directions — the negated message is what makes a matcher pleasant to debug.
- 3 **Alternative:** if the assertion is about one element type, put it on that component as `expectSomething()` instead.

Why here: a named matcher produces a failure message that is diagnosable from a CI log alone, which `expect(true).toBe(false)` never is.

10. Change the reporting outputs

- 1 **Where:** `playwright.config.ts` — the `reporters` array, for adding or removing a reporter.
- 2 **Where:** `src/reporters/summary.reporter.ts` — to add a field to `reports/summary.json` that a dashboard or PR comment needs.
- 3 **Where:** `.github/workflows/playwright.yml` — to change what is uploaded or retained.

Why here: `summary.json` is the machine-readable contract with CI. Extending it there keeps every consumer working; parsing the HTML report instead would break on the next Playwright upgrade.

11. Update visual baselines

- 1 **Confirm the change is intended.** Open the diff images in the HTML report first.
- 2 **Regenerate**, ideally in the container so the images match CI.

```
npm run test:visual:update
# or: docker compose run --rm ui-tests --project=visual --update-snapshots
```

- 3 **Review the images in the pull request** exactly as you would review code.

Why here: baselines are OS- and browser-specific. Regenerating on a laptop and running in CI produces diffs on every run, which is why the container exists.

12. Handle an iframe, a shadow root or a new tab

ALL THREE, WITHOUT SPECIAL CASES

TypeScript

```
// iframe: a scope, not a component
const payment = new Frame(page, '#payment-iframe');
await payment.waitForLoaded();
const card = ui(payment.locator).input('#card-number');

// shadow DOM: open roots are pierced automatically
const picker = ui.shadowHost('my-date-picker');
await picker.clickInShadow('.today'); // only needed for closed roots

// new tab
const report = await ui.link('#open-report').clickAndGetNewTab();
```

Why here: treating a frame as a scope rather than a component is what keeps iframe support out of all 37 component classes.

13. Onboard a new engineer

1. Read **Overview** and **Architecture** here (about twenty minutes).
2. Run `npm ci && npx playwright install --with-deps`, then `npm run test:ui`.
3. Read `src/core/base.component.ts` — it explains more about the framework than any other file.
4. Read `src/pages/bank/transfer.page.ts`, then build a page object for one screen the same way.
5. Write one `@smoke` test using only page-object methods.
6. Read **Conventions** before opening the pull request.

Conventions

Rules, naming, tags, anti-patterns and the review checklist

RULE **Record a defect you cannot fix with `test.fail()`, never with `test.skip()` or a deletion.** The expectation stays in the suite written as it should behave, the run stays green so nobody learns to ignore it, and the test turns red the moment somebody fixes the defect. A skipped test goes quiet forever; a deleted one loses the finding. Tag it `@known-issue`.

RULE **Explore the application before writing a page object.** Dump its test ids, open its widgets, submit its forms incomplete and read the messages that come back. Every non-obvious detail in this suite — a listbox that stays in the DOM when closed, a badge that disappears at zero, three required fields where one was expected — was found that way, and each would have cost more to discover through a failing test.

RULE **Baseline recurring accessibility violations; annotate one-off ones.** A permanently red a11y suite is one people stop reading, which loses every future violation too. Exclude the known rules by name so anything **new** fails the build, and report the backlog in a test that never fails. Shrink the list; never grow it.

Rules the framework enforces, the anti-patterns they prevent, and a review checklist.

Hard rules

RULE	WHY	ENFORCED BY
Import <code>test / expect</code> from <code>@fixtures/index</code>	Otherwise the spec has no fixtures or matchers.	Review
No selectors in specs	A selector change would touch every test that used it.	Review
No <code>waitForTimeout</code> in tests	Fixed sleeps are slow when short and flaky when long.	Review
No <code>process.env</code> outside <code>env.config.ts</code>	Unvalidated configuration fails silently and late.	Review
No business assertions in page objects	Tests must state their own expectations.	Review
Every test carries a tag	CI slices the suite by tag.	Review

RULE	WHY	ENFORCED BY
Every page object declares a ready indicator	Deterministic navigation.	TypeScript (abstract member)
No floating promises	An un-awaited action produces a phantom pass.	ESLint (error)
Explicit return types on exported functions	Signatures stay stable and readable.	ESLint (<code>--max-warnings=0</code>)
No <code>test.only</code> on a branch	It would silently skip the entire suite in CI.	<code>forbidOnly</code> in CI

Naming

THING	CONVENTION	EXAMPLE
Spec file	<code><feature>.spec.ts</code> under <code>tests/ui/<area>/</code>	<code>tests/ui/checkout/discount.spec.ts</code>
Page object	<code><Feature>Page</code> in <code><feature>.page.ts</code>	<code>CheckoutPage</code>
Component	Noun naming the element type	<code>DataGrid</code> , <code>RichTextEditor</code>
Component options	<code><Component>Options</code>	<code>DropdownOptions</code>
Component instance	What a user would call it	<code>applyPromoButton</code> , not <code>btn2</code>
Test title	Behaviour plus tag	<code>applies a discount code @smoke</code>
Utility file	<code><area>.utils.ts</code>	<code>network.utils.ts</code>

Tags

TAG	MEANING	RUNS
<code>@smoke</code>	Critical path; must always pass.	Every pull request
<code>@regression</code>	Full functional coverage.	Nightly
<code>@a11y</code>	Accessibility scan.	Accessibility job
<code>@visual</code>	Screenshot comparison.	Visual job

TAG	MEANING	RUNS
<code>@slow</code>	Known long-running.	Nightly
<code>@flaky</code>	Quarantined, with a ticket.	Excluded from gating runs

Anti-patterns

INSTEAD OF...	DO THIS	BECAUSE
<code>await</code> <code>page.waitForTimeout(3000)</code>	A condition-based wait in the component	Sleeps are slow when short and flaky when long.
<code>page.locator('#btn-4').click()</code> in a spec	A page-object method	The spec should read as behaviour.
A hard-coded user in a shared record	the <code>testData</code> fixture	Parallel workers collide.
Signing in through the UI in every test	Storage state	Minutes of wall-clock time per run.
<code>if (await x.isVisible()) { ... }</code> branching	Deterministic setup	Conditional tests hide failures.
Raising a timeout to fix flakiness	Fixing the wait	The bug is still there, just slower.
A full-page screenshot for one component	<code>compareElement</code>	Full-page baselines fail for any reason.
<code>expect(await x.count()).toBe(3)</code>	<code>await</code> <code>expect(x).toHaveCount(3)</code>	Web-first assertions retry; plain ones do not.

Pull-request checklist

- `npm run validate` passes (typecheck, lint with zero warnings, format).
- New element behaviour lives in a component, not a page object or a test.
- New selectors are configurable options, not hard-coded strings, when the component serves several UI libraries.
- Every new public method is wrapped in `this.step()` and has an explicit return type.
- New configuration is in the zod schema **and** in `.env.example`.
- New tests carry a tag and use no raw selectors.
- Test data is generated or API-seeded, never a shared fixed record.
- Documentation entry added for any new file, and `npm run docs` succeeds.

Troubleshooting

Failure modes, flakiness and debugging tools

Traps specific to this application

SYMPTOM	CAUSE	FIX
<code>waiting for [role="listbox"] to be visible</code> while a panel is clearly open	A closed dropdown leaves its panel in the DOM, so an unqualified selector resolves to the hidden one.	Use <code>VISIBLE_LISTBOX</code> from <code>src/pages/bank/dropdown-panel.ts</code> .
A dialog never closes after submitting	A required field was not filled. The application validates on submit and shows the message inside the dialog.	Read the dialog text — it names the missing field. The add-account and add-biller dialogs each have a field with no test id.
<code>strict mode violation</code> on <code>bank-sidebar</code>	The mobile drawer renders a second copy of the sidebar.	Scope to the overlay with <code>shell.mobileNavLink()</code> . <code>.first()</code> picks the hidden desktop copy and can never be visible.
A wait for the unread count "0" times out	The badge is removed from the DOM at zero rather than showing "0".	Use <code>unreadCount()</code> , which reads absence as zero.
<code>toBeHidden()</code> fails on a visually hidden element	<code>sr-only</code> clips to 1x1 rather than hiding, because a hidden element cannot be focused.	Assert the bounding box instead.
A seeded-value assertion is off by a small amount	The value was read from a session that had already performed a transfer.	Re-read it from a fresh browser context and update <code>SEED</code> .

Failure modes, what they mean, and where to fix them.

Setup and environment

SYMPTOM	CAUSE	FIX
<code>Invalid environment configuration:</code> on startup	A variable failed zod validation.	The message names the variable — fix it in <code>.env</code> or CI secrets.
<code>Missing credentials for role "admin"</code>	<code>ADMIN_USER</code> / <code>ADMIN_PASSWORD</code> not set.	Set them in <code>.env</code> locally or as repository secrets in CI.

SYMPTOM	CAUSE	FIX
Every test fails as unauthenticated	Saved storage states expired.	Delete <code>storage/*.json</code> and re-run; the <code>setup</code> project regenerates them.
<code>setup</code> project fails at login	Placeholder selectors in <code>auth.setup.ts</code> do not match your login page.	Replace them — this is the one file that must know your markup.
Modules load with empty exports; tooling fails in bizarre ways	A corrupted dependency install.	<code>rm -rf node_modules && npm ci</code> .
Playwright or ESLint crashes on an unusual Node version	Non-LTS Node changes module resolution.	Use the version in <code>.nvmrc</code> (Node 22).

Flaky tests

SYMPTOM	LIKELY CAUSE	WHERE TO FIX
Passes alone, fails in parallel	Shared test data.	Use the <code>testData</code> fixture.
Passes locally, fails in CI	CI is slower, or the viewport differs.	Fix the wait in the component; check the project viewport.
Fails intermittently on click	Element still animating or moving.	Set <code>waitForStable: true</code> on that component.
Toast assertion fails intermittently	The toast auto-dismissed before the read.	<code>Alert.waitForMessage()</code> captures during the visibility window.
Autocomplete selects the wrong option	Assertion ran before the debounce settled.	<code>typeAndSettle()</code> or configure <code>debounceMs</code> .
Grid row not found	Virtualised row not rendered.	<code>DataGrid.scrollToRow()</code> .
Visual diff on every run	Dynamic region not masked.	Add the selector to <code>DEFAULT_MASK_SELECTORS</code> or <code>data-visual-ignore</code> .

Reading a failure

- 1 Read the error message. Component failures name the component, the selector and the underlying cause.
- 2 Open the trace — `npm run report`, then the failing test. Steps are named after business actions.

- 3 Check the attachments: `failure-url.txt`, `dom-snapshot.html` and `browser-diagnostics.json` (console errors, page errors, failed requests).
- 4 Check `test-results/logs/test-run.log` for the structured log of the run; the `pid` field separates parallel workers.
- 5 Reproduce locally with the same seed — generated data is derived from the test title, so it repeats.

```
npx playwright test --grep "applies a discount code" --headed --debug
```

Debugging tools

TOOL	COMMAND	BEST FOR
UI mode	<code>npm run test:ui</code>	Watch mode, time-travel, locator picking.
Debug mode	<code>npm run test:debug</code>	Stepping through with the inspector.
Headed	<code>npm run test:headed</code>	Watching the real interaction.
Trace viewer	<code>npm run report</code>	Post-mortem of a CI failure.
Codegen	<code>npm run codegen</code>	Discovering resilient locators.
<code>component.highlight()</code>	in code	Confirming which element a locator resolves to.
<code>component.getState()</code>	in code	One-call snapshot of everything observable.
<code>LOG_LEVEL=trace</code>	env var	Seeing every framework action.

Performance

PROBLEM	FIX
Suite is slow overall	Increase workers; shard in CI; ensure storage-state auth is used.
Individual tests are slow	Seed through the API instead of the UI; block third-party assets.
CI is slow	Increase the shard count in the matrix — wall-clock time stays flat as the suite grows.
Browser install dominates CI time	The browser cache keyed on <code>package-lock.json</code> is already configured; confirm it is hitting.

API index

Every exported symbol and where it lives

Every exported symbol in the framework — 157 in total — extracted from the source. Click a file to jump to its reference entry.

SYMBOL	KIND	DEFINED IN	SUMMARY
<code>AllYScanOptions</code>	INTERFACE	<code>src/types/index.ts</code>	
<code>Accordion</code>	CLASS	<code>src/components/navigation/accordion.ts</code>	Accordion / collapsible sections.
<code>AccordionOptions</code>	INTERFACE	<code>src/components/navigation/accordion.ts</code>	
<code>AccountDetailPage</code>	CLASS	<code>src/pages/bank/account-detail.page.ts</code>	A single account's page: balance plus a filterable, sortable transaction table.
<code>AccountsPage</code>	CLASS	<code>src/pages/bank/accounts.page.ts</code>	The account list, plus the add/edit dialog it opens.
<code>Alert</code>	CLASS	<code>src/components/feedback/alert.ts</code>	Toast / snackbar / inline alert.
<code>AlertOptions</code>	INTERFACE	<code>src/components/feedback/alert.ts</code>	
<code>ALL_PERSONAS</code>	CONST	<code>src/data/personas.ts</code>	Every persona, for the test that cross-checks this file against the app.
<code>ApiClient</code>	CLASS	<code>src/api/api.client.ts</code>	Thin HTTP client for *test setup*, not for testing the API itself.
<code>AppConfig</code>	TYPE	<code>src/config/env.config.ts</code>	
<code>ApplyLoanPage</code>	CLASS	<code>src/pages/bank/apply-loan.page.ts</code>	Loan applications and their history.

SYMBOL	KIND	DEFINED IN	SUMMARY
<code>architecture</code>	CONST	<code>scripts/docs/content/guides.mjs</code>	
<code>Autocomplete</code>	CLASS	<code>src/components/form/autocomplete.ts</code>	Typeahead / autocomplete: type a fragment, wait for asynchronous suggestions, pick one.
<code>AutocompleteOptions</code>	INTERFACE	<code>src/components/form/autocomplete.ts</code>	
<code>BankShell</code>	CLASS	<code>src/pages/bank/bank-shell.ts</code>	The application chrome that surrounds every signed-in page: the top bar and the left sidebar.
<code>BaseComponent</code>	CLASS	<code>src/core/base.component.ts</code>	The root of the component model.
<code>BasePage</code>	CLASS	<code>src/core/base.page.ts</code>	Base for every page object.
<code>BillPayment</code>	CLASS	<code>src/pages/bank/bill-payment.page.ts</code>	Bill payment: pick a biller, an amount and a date.
<code>BoundingBox</code>	INTERFACE	<code>src/types/index.ts</code>	
<code>Breadcrumb</code>	CLASS	<code>src/components/navigation/breadcrumb.ts</code>	Breadcrumb trail — position within the hierarchy, and navigation up it.
<code>BreadcrumbOptions</code>	INTERFACE	<code>src/components/navigation/breadcrumb.ts</code>	
<code>Button</code>	CLASS	<code>src/components/form/button.ts</code>	Any clickable control: <code><button></code> , <code><input type="submit"></code> , <code></code> or a div that a framework pretends is a button.
<code>Canvas</code>	CLASS	<code>src/components/media/canvas.ts</code>	<code><canvas></code> — signature pads, drawing tools, charts and games.
<code>captureConsole</code>	FUNCTION	<code>src/utils/browser.utils.ts</code>	Starts recording console errors, uncaught exceptions and failed requests.
<code>CapturedRequest</code>	INTERFACE	<code>src/types/index.ts</code>	

SYMBOL	KIND	DEFINED IN	SUMMARY
<code>Card</code>	CLASS	<code>src/components/data/card.ts</code>	Card / tile — product cards, dashboard widgets, result items.
<code>CardOptions</code>	INTERFACE	<code>src/components/data/card.ts</code>	
<code>Carousel</code>	CLASS	<code>src/components/media/carousel.ts</code>	Image/content carousel with arrows, indicators and (often) autoplay.
<code>CarouselOptions</code>	INTERFACE	<code>src/components/media/carousel.ts</code>	
<code>Chart</code>	CLASS	<code>src/components/advanced/chart.ts</code>	SVG chart (D3, Highcharts, Chart.js in SVG mode, Recharts).
<code>ChartOptions</code>	INTERFACE	<code>src/components/advanced/chart.ts</code>	
<code>Checkbox</code>	CLASS	<code>src/components/form/checkbox.ts</code>	Checkbox — native <code><input type="checkbox"></code> or an ARIA checkbox.
<code>ClickOptions</code>	TYPE	<code>src/types/index.ts</code>	
<code>ColorPicker</code>	CLASS	<code>src/components/form/color-picker.ts</code>	<code><input type="color"></code> and custom colour swatch pickers.
<code>ComponentOptions</code>	INTERFACE	<code>src/types/index.ts</code>	
<code>config</code>	CONST	<code>src/config/env.config.ts</code>	The single, validated, strongly-typed view of runtime configuration.
<code>ConsoleCapture</code>	INTERFACE	<code>src/utils/browser.utils.ts</code>	
<code>conventions</code>	CONST	<code>scripts/docs/content/guides.mjs</code>	
<code>createConsoleCapture</code>	FUNCTION	<code>src/utils/browser.utils.ts</code>	An empty capture container, to be attached to a page later.

SYMBOL	KIND	DEFINED IN	SUMMARY
<code>createLogger</code>	FUNCTION	<code>src/utils/logger.ts</code>	
<code>DashboardPage</code>	CLASS	<code>src/pages/bank/dashboard.page.ts</code>	The landing page after sign-in: net-worth cards, quick actions, recent activity.
<code>DataGrid</code>	CLASS	<code>src/components/data/data-grid.ts</code>	Enterprise data grid (AG Grid, MUI DataGrid, Kendo, PrimeNG).
<code>DataGridOptions</code>	INTERFACE	<code>src/components/data/data-grid.ts</code>	
<code>DatePicker</code>	CLASS	<code>src/components/form/date-picker.ts</code>	Date picker, covering both interaction paths: typing into the input, and navigating the calendar popup month by month.
<code>DatePickerOptions</code>	INTERFACE	<code>src/components/form/date-picker.ts</code>	
<code>detectLanguage</code>	FUNCTION	<code>scripts/docs/highlight.mjs</code>	
<code>Draggable</code>	CLASS	<code>src/components/advanced/drag-drop.ts</code>	Draggable element.
<code>Dropdown</code>	CLASS	<code>src/components/form/dropdown.ts</code>	Custom (non-native) dropdown / combobox / listbox — React-Select, MUI, Ant Design, Kendo and friends.
<code>DropdownOptions</code>	INTERFACE	<code>src/components/form/dropdown.ts</code>	
<code>DropZone</code>	CLASS	<code>src/components/advanced/drag-drop.ts</code>	A drop zone: where draggables land.
<code>ElementState</code>	INTERFACE	<code>src/types/index.ts</code>	The observable state of any UI element, used by assertions and diagnostics.
<code>ensureDir</code>	FUNCTION	<code>src/utils/file-utils.ts</code>	
<code>EnvironmentConfig</code>	INTERFACE	<code>src/types/index.ts</code>	

SYMBOL	KIND	DEFINED IN	SUMMARY
<code>EnvironmentName</code>	TYPE	<code>src/types/index.ts</code>	
<code>esc</code>	FUNCTION	<code>scripts/docs/renderer.mjs</code>	
<code>expect</code>	CONST	<code>src/fixtures/custom-matchers.ts</code>	Domain-specific assertions.
<code>fileAnchor</code>	FUNCTION	<code>scripts/docs/renderer.mjs</code>	
<code>FileUpload</code>	CLASS	<code>src/components/form/file-upload.ts</code>	File upload in all three flavours: a plain <code><input type="file"></code> , a hidden input behind a styled button, and a drag-and-drop zone.
<code>FileUploadOptions</code>	INTERFACE	<code>src/components/form/file-upload.ts</code>	
<code>FillOptions</code>	TYPE	<code>src/types/index.ts</code>	
<code>Form</code>	CLASS	<code>src/components/form/form.ts</code>	Composite component for a whole <code><form></code> .
<code>format</code>	FUNCTION	<code>src/utils/date-utils.ts</code>	Formats a date with a token string: YYYY, MM, DD, HH, mm, ss, MMM, MMMM, D, M.
<code>Frame</code>	CLASS	<code>src/components/media/frame.ts</code>	iframe wrapper.
<code>GeneratedUser</code>	INTERFACE	<code>src/utils/data-utils.ts</code>	
<code>generateUser</code>	FUNCTION	<code>src/utils/data-utils.ts</code>	
<code>getEnvironment</code>	FUNCTION	<code>src/config/environments.ts</code>	
<code>getUser</code>	FUNCTION	<code>src/config/env.config.ts</code>	Resolves credentials for a role, failing loudly at the point of use rather than silently logging in as <code>undefined</code> .
<code>glossary</code>	CONST	<code>scripts/docs/content/guides.mjs</code>	

SYMBOL	KIND	DEFINED IN	SUMMARY
<code>highlight</code>	FUNCTION	<code>scripts/docs/highlight.mjs</code>	Highlights a snippet.
<code>humanizeDuration</code>	FUNCTION	<code>src/utils/date.utils.ts</code>	Human-readable elapsed time, for logs and reports.
<code>Image</code>	CLASS	<code>src/components/media/image.ts</code>	<code></code> — including the load-failure and lazy-loading cases.
<code>InfiniteScroll</code>	CLASS	<code>src/components/advanced/infinite-scroll.ts</code>	Infinite-scroll / lazy-loaded list.
<code>InfiniteScrollOptions</code>	INTERFACE	<code>src/components/advanced/infinite-scroll.ts</code>	
<code>inline</code>	FUNCTION	<code>scripts/docs/renderer.mjs</code>	Inline markup: <code>code</code> , bold, text.
<code>LANGUAGE_LABEL</code>	CONST	<code>scripts/docs/highlight.mjs</code>	
<code>Link</code>	CLASS	<code>src/components/navigation/link.ts</code>	Anchor element, including download and new-tab behaviours.
<code>ListOptions</code>	INTERFACE	<code>src/components/data/list.ts</code>	
<code>ListView</code>	CLASS	<code>src/components/data/list.ts</code>	Ordered/unordered lists, <code>role="list"</code> , and card collections.
<code>Loader</code>	CLASS	<code>src/components/feedback/progress.ts</code>	Spinner / skeleton loader — an element whose whole job is to disappear.
<code>logger</code>	CONST	<code>src/utils/logger.ts</code>	Framework-wide logger.
<code>Logger</code>	INTERFACE	<code>src/utils/logger.ts</code>	
<code>LoginPage</code>	CLASS	<code>src/pages/bank/login.page.ts</code>	The SecureBank sign-in page.
<code>LogLevel</code>	TYPE	<code>src/types/index.ts</code>	

SYMBOL	KIND	DEFINED IN	SUMMARY
<code>MediaPlayer</code>	CLASS	<code>src/components/media/media-player.ts</code>	<code><video></code> and <code><audio></code> .
<code>Menu</code>	CLASS	<code>src/components/navigation/menu.ts</code>	Navigation menu / menubar / context menu, including nested submenus.
<code>MenuOptions</code>	INTERFACE	<code>src/components/navigation/menu.ts</code>	
<code>MockResponse</code>	INTERFACE	<code>src/types/index.ts</code>	
<code>Modal</code>	CLASS	<code>src/components/feedback/modal.ts</code>	Modal dialog / drawer / confirmation popup.
<code>ModalOptions</code>	INTERFACE	<code>src/components/feedback/modal.ts</code>	
<code>MultiSelect</code>	CLASS	<code>src/components/form/multi-select.ts</code>	Multi-select / tag input: several values selected at once, usually rendered as removable chips.
<code>MultiSelectOptions</code>	INTERFACE	<code>src/components/form/multi-select.ts</code>	
<code>NavDestination</code>	TYPE	<code>src/pages/bank/bank-shell.ts</code>	Every sidebar destination, as the application slugs them.
<code>NetworkHelper</code>	CLASS	<code>src/utils/network.utils.ts</code>	Network control surface for a page or context: stub responses, simulate failures and latency, and record traffic for assertions.
<code>normalizeText</code>	FUNCTION	<code>src/utils/string.utils.ts</code>	Collapses all whitespace runs to single spaces and trims — DOM text is messy.
<code>NotificationsPage</code>	CLASS	<code>src/pages/bank/notifications.page.ts</code>	The notification centre, and the unread badges it drives.
<code>overview</code>	CONST	<code>scripts/docs/content/guides.mjs</code>	Long-form documentation pages.

SYMBOL	KIND	DEFINED IN	SUMMARY
Pagination	CLASS	src/components/data/pagination.ts	Pagination control: page numbers, next/previous, and page-size selection.
PaginationOptions	INTERFACE	src/components/data/pagination.ts	
parseCurrency	FUNCTION	src/pages/bank/dashboard.page.ts	Turns a displayed money string into a number.
Persona	INTERFACE	src/data/personas.ts	
PersonaName	TYPE	src/data/personas.ts	
PERSONAS	CONST	src/data/personas.ts	Keyed by trait rather than by username, so a test reads as an intention — <code>personas.frozen</code> — instead of as a lookup somebody has to decode.
PersonaTrait	TYPE	src/data/personas.ts	The behaviours the application deliberately exhibits per account.
playbooks	CONST	scripts/docs/content/guides.mjs	
ProfilePage	CLASS	src/pages/bank/profile.page.ts	Profile, password change and security settings.
ProgressBar	CLASS	src/components/feedback/progress.ts	Determinate/indeterminate progress bar.
RadioGroup	CLASS	src/components/form/radio-group.ts	A set of mutually-exclusive radio options, addressed by value, label or index.
Rating	CLASS	src/components/form/rating.ts	Star / heart rating control.
RatingOptions	INTERFACE	src/components/form/rating.ts	
renderBlocks	FUNCTION	scripts/docs/renderer.mjs	

SYMBOL	KIND	DEFINED IN	SUMMARY
<code>renderFileEntry</code>	FUNCTION	<code>scripts/docs/render.mjs</code>	Renders one documented file: purpose, change guidance and API.
<code>resolveDataPath</code>	FUNCTION	<code>src/utils/fileUtils.ts</code>	Resolves a path against the test-data directory when it is not absolute.
<code>resolveLocator</code>	FUNCTION	<code>src/core/locator.factory.ts</code>	Resolves any <code>SelectorLike</code> into a Locator against a scope, applying the iframe and shadow-DOM options that components declare.
<code>RichTextEditor</code>	CLASS	<code>src/components/advanced/rich-text-editor.ts</code>	WYSIWYG editor (Quill, TinyMCE, CKEditor, ProseMirror, Slate).
<code>RichTextEditorOptions</code>	INTERFACE	<code>src/components/advanced/rich-text-editor.ts</code>	
<code>scanAccessibility</code>	FUNCTION	<code>src/utils/allyUtils.ts</code>	Runs an axe-core scan against the current page (or a subtree of it) and attaches a readable report to the test result.
<code>Scope</code>	TYPE	<code>src/core/locator.factory.ts</code>	
<code>SEED</code>	CONST	<code>src/data/personas.ts</code>	Facts about the seeded dataset that tests assert against.
<code>seedFaker</code>	FUNCTION	<code>src/utils/dataUtils.ts</code>	Seeds faker so a failing run can be reproduced exactly.
<code>Select</code>	CLASS	<code>src/components/form/select.ts</code>	Native <code><select></code> .
<code>SelectorLike</code>	TYPE	<code>src/types/index.ts</code>	Anything a component can be built from: a raw CSS/XPath selector, an already-resolved Locator, or a factory that derives one from a scope.
<code>SendMoneyPage</code>	CLASS	<code>src/pages/bank/send-money.page.ts</code>	External payments to a saved payee, plus the "add payee" dialog.
<code>ShadowHost</code>	CLASS	<code>src/components/advanced/shadow-dom.ts</code>	Web component with a shadow root.
<code>Slider</code>	CLASS	<code>src/components/form/slider.ts</code>	Range slider — native <code><input type="range"></code> or <code>role="slider"</code> .

SYMBOL	KIND	DEFINED IN	SUMMARY
<code>slug</code>	FUNCTION	<code>scripts/docs/re nder.mjs</code>	
<code>SortDirect ion</code>	TYPE	<code>src/types/index .ts</code>	
<code>Stepper</code>	CLASS	<code>src/components/ navigation/stepp er.ts</code>	Multi-step wizard / stepper: current step, completion state, and traversal.
<code>StepperOpt ions</code>	INTERFACE	<code>src/components/ navigation/stepp er.ts</code>	
<code>SummaryRep orter</code>	CLASS	<code>src/reporters/s ummary.reporter. ts</code>	Console summary plus a machine-readable <code>summary.json</code> .
<code>Table</code>	CLASS	<code>src/components/ data/table.ts</code>	HTML table.
<code>TableOptio ns</code>	INTERFACE	<code>src/components/ data/table.ts</code>	
<code>TableRow</code>	TYPE	<code>src/types/index .ts</code>	
<code>Tabs</code>	CLASS	<code>src/components/ navigation/tabs. ts</code>	Tab strip with associated panels (<code>role="tablist"</code> / <code>tab</code> / <code>tabpanel</code>).
<code>TabsOption s</code>	INTERFACE	<code>src/components/ navigation/tabs. ts</code>	
<code>test</code>	CONST	<code>src/fixtures/au th.fixture.ts</code>	Authentication via saved storage state.
<code>test</code>	CONST	<code>src/fixtures/ba nk.fixture.ts</code>	
<code>test</code>	CONST	<code>src/fixtures/ba se.fixture.ts</code>	The framework's test object.
<code>TestUser</code>	INTERFACE	<code>src/types/index .ts</code>	
<code>TextArea</code>	CLASS	<code>src/components/ form/text- area.ts</code>	Multi-line text field.

SYMBOL	KIND	DEFINED IN	SUMMARY
<code>TextInput</code>	CLASS	<code>src/components/form/text-input.ts</code>	Single-line inputs of every flavour (text, email, password, number, tel, url, search) plus <code>contenteditable</code> fields.
<code>TIMEOUTS</code>	CONST	<code>src/config/timeouts.ts</code>	Centralised timeout budget (milliseconds).
<code>Toggle</code>	CLASS	<code>src/components/form/toggle.ts</code>	Toggle / switch control — <code>role="switch"</code> , a styled checkbox, or a div pair.
<code>toIsoDate</code>	FUNCTION	<code>src/utils/date-utils.ts</code>	ISO date portion only: 2026-08-31
<code>Tooltip</code>	CLASS	<code>src/components/feedback/tooltip.ts</code>	Tooltip / popover.
<code>TooltipOptions</code>	INTERFACE	<code>src/components/feedback/tooltip.ts</code>	
<code>TransactionsPage</code>	CLASS	<code>src/pages/bank/transactions.page.ts</code>	All activity across every account: search, filter, sort, paginate, export.
<code>TransferPage</code>	CLASS	<code>src/pages/bank/transfer.page.ts</code>	Internal transfers between the signed-in user's own accounts.
<code>TreeOptions</code>	INTERFACE	<code>src/components/data/tree.ts</code>	
<code>TreeView</code>	CLASS	<code>src/components/data/tree.ts</code>	Tree view / file explorer — <code>role="tree"</code> with expandable <code>treeitem</code> nodes.
<code>troubleshooting</code>	CONST	<code>scripts/docs/content/guides.mjs</code>	
<code>ui</code>	FUNCTION	<code>src/components/component.factory.ts</code>	Creates a UI factory bound to a page, frame or container locator.
<code>UiFactory</code>	CLASS	<code>src/components/component.factory.ts</code>	Ergonomic factory bound to a scope (a page, a frame or a container locator).
<code>UserRole</code>	TYPE	<code>src/types/index.ts</code>	

SYMBOL	KIND	DEFINED IN	SUMMARY
<code>VISIBLE_LI</code> <code>STBOX</code>	CONST	<code>src/pages/bank/dropdown-panel.ts</code>	The selector every custom dropdown in SecureBank shares.
<code>waitUntil</code>	FUNCTION	<code>src/utils/retry.utils.ts</code>	Polls <code>condition</code> until it returns true or the timeout elapses.
<code>WcagTag</code>	TYPE	<code>src/types/index.ts</code>	
<code>workedExample</code>	CONST	<code>scripts/docs/content/guides.mjs</code>	

Glossary

Terms used in the code and in this document

TERM	MEANING HERE
Persona	A demo account whose behaviour the application deliberately varies — locked, frozen, overdrawn, or carrying a planted defect. How negative paths are reached without mocking.
Planted defect	A bug the application ships on purpose, for a suite to find. SecureBank has one: the loan history total omits the newest loan for <code>error_user</code> .
Documented defect	A real defect the suite cannot fix, recorded with <code>test.fail()</code> so the expectation survives, the run stays green, and a fix turns the test red.
Baselined violation	An accessibility rule already failing across the application, excluded by name from the gating scans so that any <i>*new*</i> violation still fails the build.
Seed value	A fact about the starting dataset, held in <code>SEED</code> and read from a clean browser context — never from an exploratory session.

Terms used throughout this documentation and in the code.

TERM	MEANING
Component	A class wrapping one kind of UI element, extending <code>BaseComponent</code> . Owns selectors and interaction behaviour.
Page object	A class representing one screen, extending <code>BasePage</code> . Owns the URL, the ready indicator and business actions.
Fixture	A Playwright dependency injected into a test. Test-scoped fixtures are rebuilt per test; worker-scoped fixtures once per worker.
Project	A named Playwright run configuration — a browser, a device, or a specialised suite such as <code>visual</code> .
Ready indicator	The element whose presence proves a page has finished rendering. Every page object must declare one.
Storage state	A saved set of cookies and origin storage that replays an authenticated session without logging in again.
Scope	What a component or factory is bound to: a page, a frame locator, or a container locator.
SelectorLike	A CSS/XPath string, an existing Locator, or a function deriving one from a scope.
Web-first assertion	An assertion that retries until it passes or times out (<code>await expect(locator).toBeVisible()</code>).

TERM	MEANING
Virtualised grid	A grid that renders only the visible window of rows; off-screen rows are absent from the DOM.
Blob report	Playwright's intermediate report format, produced per shard and merged into one HTML report.
Trace	A recorded timeline of a test — DOM snapshots, actions, network — replayable in the trace viewer.
Flaky test	A test that passed only on retry. Tracked in <code>reports/summary.json</code> .
Tag	A marker in a test title (<code>@smoke</code>) used by <code>--grep</code> to slice the suite.
Named timeout budget	A constant in <code>timeouts.ts</code> used instead of a literal, so waits can be tuned as a class.